



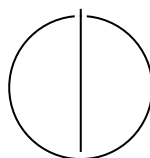
SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

**Flow Matching Predictive Control with
Constraints**

TODO: Author





SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

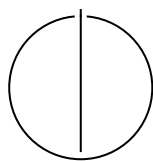
TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

**Flow Matching Predictive Control with
Constraints**

**Flow-Matching-basierte prädiktive
Regelung unter Nebenbedingungen**

Author:	TODO: Author
Examiner:	TODO: Supervisor
Supervisor:	TODO: Advisor
Submission Date:	TODO: Submission date



I confirm that this master's thesis is my own work and I have documented all sources and material used.

Munich, TODO: Submission date

TODO: Author

Acknowledgments

Abstract

Diffusion planners capture multimodal demonstrations, and Diffusion Predictive Control with Constraints (DPCC) enforces constraints by projecting denoising iterates onto the feasible set. However, their long denoising sequence is fixed during training and repeated at every control step. This thesis replaces diffusion with deterministic transport along an ordinary differential equation. Three objectives are studied: instantaneous-velocity matching, analytic average-velocity matching and consistency-interpolated average-velocity matching, whose target mixes the instantaneous velocity with the network’s own prediction; the integration budget is selected at deployment. On the obstacle-avoidance benchmark of DPCC, the aim is to preserve success and constraint satisfaction while reducing network evaluations, the time to compute one action and the control steps required to reach the goal. Constraint enforcement forms a second contribution. DPCC projects the current iterate; endpoint projection predicts and projects the final trajectory, then guides later sampling steps towards it. The analysis identifies when the budget and activation threshold permit guidance. After evaluation on the DPCC benchmark, the controller is extended to two more complex settings built for this thesis: vision-conditioned object alignment and control of an underactuated, open-loop unstable quadrotor. On obstacle avoidance, the average-velocity models lead: consistency-interpolated average-velocity matching dominates the diffusion model of DPCC with its own network, holding its success within the constraints in fewer control steps at a small fraction of its computation, and analytic average-velocity matching gives up little of that success for the same saving; no configuration of endpoint projection improves on them there. On vision-conditioned alignment, analytic average-velocity matching dominates the diffusion model again, and at the larger budget that task needs, endpoint projection is the better projector on the constraints. On the quadrotor, projection carries over to a new plant and to constraints in three dimensions; there, simple generated demonstrations make instantaneous-velocity matching enough, and the tracking controller decides whether a plan is flown.

Contents

Acknowledgments	iv
Abstract	v
1. Introduction	1
1.1. Motivation	1
1.2. Problem Statement	1
1.3. Research Questions	2
1.4. Contributions	2
1.5. Outline	5
2. Background	6
2.1. Learning Trajectory Policies from Demonstration	6
2.2. Diffusion Models and Diffusion Predictive Control	6
2.3. Flow Matching	7
2.4. Few-Step Generative Models	7
2.5. Constraint Enforcement in Generative Sampling	8
2.6. Vision-Conditioned Policies	9
2.7. Quadrotor Dynamics and Control	9
3. Related Work	11
3.1. Diffusion Planners and Policies	11
3.2. Flow Matching and Few-Step Policies	11
3.3. Accelerating Generative Planners	12
3.4. Constrained and Safe Generative Planning	12
3.5. Vision-Conditioned Imitation Learning	13
3.6. Learning-Based Planning and Control for Aerial Vehicles	13
3.7. Positioning of This Thesis	13
4. Method	16
4.1. Problem Formulation	16
4.1.1. The Control Problem	16
4.1.2. The Plan Variable	16

4.1.3.	Notation	17
4.2.	System Overview	17
4.3.	Plant Dynamics and Control	19
4.3.1.	The Manipulator and Its First-Order Model	20
4.3.2.	The Quadrotor	20
4.3.3.	From Plan to Setpoint	21
4.3.4.	Manipulator: Differential Inverse Kinematics	23
4.3.5.	Quadrotor: Cascaded Geometric Tracking Control	23
4.3.6.	Sampling-Based Predictive Control	26
4.4.	Generative Models	27
4.4.1.	Diffusion and the DPCC Baseline	27
4.4.2.	Flow Matching	28
4.4.3.	Analytic Average-Velocity Matching	30
4.4.4.	Consistency-Interpolated Average-Velocity Matching	33
4.4.5.	A Common Interface	33
4.4.6.	Backbone and Conditioning	34
4.5.	Constraint Projection	36
4.5.1.	The Projection Problem	36
4.5.2.	Per-Step Projection (DPCC)	38
4.5.3.	Endpoint Projection	39
4.5.4.	Steps at Which Projection Guides Sampling	42
4.5.5.	Candidate Selection	43
4.6.	Environments	43
4.6.1.	Obstacle Avoidance	44
4.6.2.	Vision-Conditioned Alignment	45
4.6.3.	Quadrotor Scenes	45
5.	Experimental Setup	49
5.1.	Simulated Platforms	49
5.2.	Tasks and Environments	51
5.2.1.	Obstacle Avoidance	51
5.2.2.	Vision-Conditioned Alignment	54
5.2.3.	Quadrotor Benchmark	58
5.3.	Datasets	62
5.3.1.	Obstacle Avoidance	62
5.3.2.	Vision-Conditioned Alignment	62
5.3.3.	Quadrotor Benchmark	63
5.4.	Metrics	66
5.4.1.	Obstacle Avoidance	66

5.4.2.	Vision-Conditioned Alignment	68
5.4.3.	Quadrotor Benchmark	68
5.4.4.	Aggregation and Variability	70
5.5.	Baselines	70
5.5.1.	Obstacle Avoidance	71
5.5.2.	Vision-Conditioned Alignment	72
5.5.3.	Quadrotor Benchmark	75
5.6.	Training and Evaluation Protocol	76
5.6.1.	Compute Environment	76
5.6.2.	Training	76
5.6.3.	Evaluation	78
5.6.4.	Guiding Steps of Endpoint Projection	82
6.	Results	84
6.1.	Obstacle Avoidance	84
6.1.1.	Generative Models	84
6.1.2.	Projection Methods	89
6.1.3.	Conclusion	97
6.2.	Vision-Conditioned Alignment	99
6.2.1.	Generative Models	99
6.2.2.	Projection Methods	103
6.2.3.	Conclusion	110
6.3.	Quadrotor Benchmark	112
6.3.1.	UAV-pillars	113
6.3.2.	UAV-corridor	118
6.3.3.	UAV-s-curve	128
6.4.	Comparison across Environments	136
6.4.1.	Generative Models	136
6.4.2.	Projection Methods	137
6.4.3.	Conclusion	138
7.	Conclusion	141
7.1.	Summary	141
7.2.	Answers to the Research Questions	142
8.	Discussion	144
8.1.	Limitations	144
8.2.	Towards Deployment	145
8.3.	Future Work	146

Appendix	148
A. Supplementary Material	149
A.1. Training-Time Sampling Laws	149
A.2. Quadrotor Dimensions	149
B. Extended Results	151
B.1. D3IL-avoiding: Executed Paths under Per-Step Projection	151
B.2. D3IL-aligning: Executed Paths under Each Projection Method	152
B.3. UAV-corridor: The Flights along the Corridor	155
B.4. UAV-corridor under Every Selection Rule	156
B.5. D3IL-avoiding at Twenty Episodes per Seed	161
C. Compute Environment	164
Abbreviations	166
List of Figures	167
List of Tables	168
Bibliography	170

1. Introduction

1.1. Motivation

A robot that plans with a generative model buys expressiveness with computation. Diffusion planners [Jan+22; Chi+23] represent multi-modal demonstration data [Jia+24] well and, when paired with a constrained projection step [RRS25], can respect dynamics and safety at deployment. The price is the sampling loop: producing one plan costs tens to hundreds of network evaluations — the number of function evaluations (NFE) of the sampler — and in a receding-horizon controller that cost is paid again at every timestep.

This cost is not incidental but *structural*, and the structure is what makes it addressable. In a diffusion model the number of steps is a property of the trained noise schedule [HJA20]; reducing it after training degrades the sample, and recovering quality needs a different sampler [SME21]. In a deterministic transport model [Lip+23] the same quantity is a property of the ordinary differential equation (ODE) solver, and is chosen at inference. This thesis moves the step budget from training time to inference time and measures what the move costs.

Once sampling is cheap, constraint enforcement becomes the dominant cost, and how constraints are enforced becomes a question in its own right.

1.2. Problem Statement

The object of study is a closed-loop controller in which, at each timestep, a conditional generative model proposes a trajectory over a fixed horizon, a constraint mechanism makes that trajectory admissible, and the first action is executed. This thesis holds that architecture fixed and varies two parts of it: the generative model and the projection method.

Flow matching in the title names the family of generative models that transport noise to data along an ordinary differential equation. The three generative models studied here belong to it and differ in the velocity their network learns: *instantaneous-velocity matching*, *analytic average-velocity matching* and *consistency-interpolated average-velocity matching*. The study covers two observation modalities, state and camera images, and

two embodiments, a manipulator and a quadrotor.

The generative models and projection methods are first compared with the diffusion baseline on the obstacle-avoidance benchmark of Diffusion Predictive Control with Constraints (DPCC) [RRS25; Jia+24], with its setup unchanged, and are then tested in a vision-conditioned alignment task and a quadrotor benchmark built for this thesis. Chapter 5 describes the evaluation.

1.3. Research Questions

- RQ1 — Generative model.** With a matched temporal U-Net backbone, dataset and evaluation protocol, can the flow-based models reach the diffusion baseline’s task success and constraint satisfaction with fewer sampling steps and less planning time per control step? How do the three objectives compare as the sampling budget changes?
- RQ2 — Constraints.** Given one fixed feasible set and one solver, does endpoint projection reach the goal with fewer constraint violations, fewer control steps or less computation than the per-step projection of DPCC? For which step budgets and activation thresholds does it guide subsequent sampling, and when does it reduce to projection of the finished sample?
- RQ3 — Transfer.** Obstacle avoidance is planar, observed as a state, kinematic at the planning level, and solvable by a classical planner with a collision constraint. Do the answers to RQ1 and RQ2 hold when the observation becomes camera images, when the task requires aligning an object through contact, and when the platform is an underactuated, open-loop unstable quadrotor moving through three-dimensional obstacle fields — settings whose dynamics and perception are closer to those of a physical deployment?

1.4. Contributions

This thesis makes six contributions, in two stages. The first four are made on the obstacle-avoidance benchmark of DPCC [RRS25; Jia+24], with its setup unchanged, against its diffusion model and its projection: three generative models take the diffusion model’s place one after another, and a second projection method is set beside the inherited one. The last two take what that benchmark establishes — the generative models and both projection methods, in the configurations that hold there — into two settings built for this thesis: a task observed through cameras, and a plant that flies.

On the benchmark of DPCC.

1. **Flow matching in place of diffusion.** The diffusion model of DPCC [RRS25] is replaced by instantaneous-velocity matching, the flow-matching objective [Lip+23; Lip+24], which learns a velocity field and generates a trajectory by integrating an ordinary differential equation. Backbone, projection, data and evaluation protocol are kept, and the number of network evaluations becomes a choice made at deployment. At the primary DPCC protocol, it matches the baseline’s success with constraint satisfaction while using fewer control steps and less time per action (Subsubsection 6.1.2.1).
2. **Analytic average-velocity matching.** Analytic average-velocity matching [Gen+25a] learns the average velocity over an interval instead of the instantaneous velocity, with a regression target constructed analytically through a Jacobian–vector product, so that a single network evaluation can cover the whole integration. It replaces instantaneous-velocity matching within the same architecture. At the selected avoiding configuration, it reduces control steps and planning time with a small loss in success with constraint satisfaction relative to the best baseline rule (Subsubsection 6.1.2.1).
3. **Consistency-interpolated average-velocity matching.** Consistency-interpolated average-velocity matching [Zha+25b] also learns the average velocity, with a target interpolated by a consistency step ratio between the instantaneous velocity and a stop-gradient prediction of the network itself; the ratio is annealed during training. Both average-velocity objectives achieve state-of-the-art one- and two-step image generation [Gen+25a; Zha+25b]; this thesis evaluates both as planners inside a constrained controller. On the avoiding benchmark it matches the baseline’s success with constraint satisfaction while reducing both control steps and planning time (Subsubsection 6.1.2.1).
4. **Endpoint projection against DPCC’s per-step projection.** The central contribution of DPCC is a model-based projection applied to each denoising step [RRS25]. Endpoint projection [LAA25] instead projects the trajectory the sampler is predicted to reach and steers the sampler towards it. Both are implemented on the same feasible set and solver and compared, together with the step budget below which endpoint projection reduces to projection after sampling. At one network evaluation, the budget that suffices on obstacle avoidance, it has no step to guide, and no configuration of it improves on per-step projection at that budget; at the budgets where it guides, it is as reliable as per-step projection or more, and cheaper for the two average-velocity models. At the larger budgets the alignment

task needs, it keeps more contexts free of violations than per-step projection in most comparisons, at a comparable time per action, and on UAV-corridor the two methods trade against each other (Subsection 6.4.2).

Beyond it.

5. **A vision-conditioned alignment task.** Once established on the state-based benchmark of DPCC, all generative models and both projection methods are tested on a more complex task built for this thesis from the alignment task of D3IL, in which the robot observes two camera views. The generative network is given a visual encoder, and the projection is given the dynamics rows and the constraint set of the new task. Analytic average-velocity matching dominates the diffusion baseline again: in the evaluated training contexts it moves the object closer to its target than the other objectives and the baseline, and keeps more contexts free of violations at a fraction of the baseline’s time per action (Section 6.2).
6. **A quadrotor benchmark.** The controller is carried to an underactuated, open-loop unstable plant: the manipulator with inverse kinematics is replaced by the Skydio X2 quadrotor of MuJoCo Menagerie [ZTM22], commanded through its rotor thrusts by a cascaded geometric tracking controller [LLM10], and the stack is tested from state observations in three scenes that ask three different questions. UAV-pillars is D3IL-avoiding flown by the quadrotor: its field is scaled into an arena of pillars, and the planner of the first four contributions flies there unchanged, so the scene reads what the vehicle adds to a plan D3IL-avoiding has already judged. UAV-corridor is the three-dimensional test: its two test-time constraints leave the horizontal plane, and a plan has to descend or climb to satisfy them. UAV-s-curve is the scene of two sharp turns, and it is reported for what those turns do to the tracking controller. This is where the constraint mechanism is asked to hold on a plant that requires continuous stabilisation: an arm can hold a setpoint through joint feedback, while a quadrotor must regulate both position and attitude in flight (Section 2.7). The scenes, their constraint sets, the flown demonstrations of the corridor and the s-curve, and the control loop were built for this thesis in MuJoCo [TET12], inside the environment code of D3IL; MuJoCo MPC [How+22], a sampling-based predictive controller, is the second tracking controller, used in the controller comparison alone. On UAV-pillars the projected flights keep the declared constraints as in D3IL-avoiding, and collisions with pillars outside those constraints end flights and lower the success. On UAV-corridor the projection carries over to three dimensions, removing most violations of constraints no demonstration satisfies, and on its straight-line demonstrations

the three objectives cannot be told apart. On UAV-s-curve the same plans succeed or fail by the controller that flies them, and the cascaded geometric controller is the better of the two (Section 6.3).

1.5. Outline

Chapter 2 introduces generative trajectory models, constraint enforcement and the two control platforms. Chapter 3 positions the comparison among generative planners and constrained control methods. Chapter 4 develops the models, projection procedures and execution loops. Chapter 5 specifies the tasks, training and evaluation protocols. Chapter 6 reports the model and projection comparisons and their transfer to visual and aerial tasks. Chapter 7 summarises the findings and answers the research questions, and Chapter 8 states the limits of the work, what a deployment would add to it, and future work.

2. Background

This chapter introduces the concepts behind each contribution of Section 1.4. Their mathematical form is given in Chapter 4.

2.1. Learning Trajectory Policies from Demonstration

The setting is a dataset of expert trajectories, a conditional model over horizon-length trajectory segments [Jan+22; Chi+23], and receding-horizon execution: the model plans a short horizon, the first action is executed, and the model plans again. The benchmarks come from [Jia+24], whose demonstrations are multimodal. Planning over a horizon permits constraints that link future states and actions. A single action can also be projected onto an admissible set. The generative model represents alternative demonstrated plans; the effect of a reduced sampling budget is evaluated in Chapter 6.

2.2. Diffusion Models and Diffusion Predictive Control

A denoising diffusion model [Soh+15; HJA20] learns a distribution by reversing a fixed process that gradually adds Gaussian noise to the data. A network is trained to predict the noise added at each step of a noise schedule; generation starts from pure noise and removes it step by step. Diffuser [Jan+22] applies this to whole trajectories, and diffusion policies [Chi+23] brought it to visuomotor control.

Two properties follow from the construction. The number of denoising steps is tied to the schedule the network was trained on, so it is fixed when training ends; fewer steps require a different sampler [SME21] or a distilled model. The stochastic reverse sampler adds noise at nonterminal steps, so plans drawn for the same observation can differ.

DPCC [RRS25] makes such a planner respect state and action constraints that are absent from the demonstrations. Between denoising steps it projects the intermediate trajectory onto the set of trajectories that satisfy the constraints and a model of the dynamics, and justifies the projection as guidance towards that set. It shrinks the constraint sets by a margin that absorbs the mismatch between the dynamics model

and the real system, and it draws several candidate plans per control step and selects one.

2.3. Flow Matching

Flow matching [Lip+23; Lip+24] describes generation as transport: a time-dependent velocity field moves samples from a noise distribution to the data distribution, and a sample is produced by integrating an ordinary differential equation along that field. The field is learned by regression. Because the field of the whole distribution is intractable, training regresses the velocities of simple per-sample paths between a noise draw and a data point, which are known in closed form and yield the same gradient [Lip+23]. With a straight-line path, as in rectified flow [LGL23], the regression target is the difference between the data point and the noise, and no trajectory has to be simulated during training. Flow matching has achieved state-of-the-art results across images, video, audio, speech and biological structures [Lip+24].

Two consequences matter for control. The number of integration steps is a property of the solver, not of training, so it is chosen at deployment. And the integration adds no noise: a plan is a deterministic function of its initial noise and the observation.

2.4. Few-Step Generative Models

Few-step generation is studied in both diffusion and transport models. Diffusion models can use accelerated samplers [SME21] or distillation [Wan+24]. For flow matching, the integration budget is an inference argument. The geometry of the learned field affects how accurately a small number of steps approximates its flow; optimal-transport paths [Lip+23] and rectified flow [LGL23] study this geometry.

A step budget chosen at deployment is still not automatically a good one. Straight per-sample paths average into a curved field, and coarse integration steps along a curved field are inaccurate however well the network is trained [Gen+25a, Sec. 3]. The objectives below change what the network learns so that a large step stays accurate.

Analytic average-velocity matching [Gen+25a] learns the average velocity over a time interval instead of the instantaneous velocity. One evaluation of an accurate average velocity covers the whole interval, so a plan can be generated in a single step. The average velocity is defined by an integral that cannot be evaluated during training; the method differentiates that definition into an identity relating the average velocity to the instantaneous velocity and to its own time derivative, and computes the derivative with a Jacobian–vector product (JVP). When the interval shrinks to a point, its target becomes the instantaneous-velocity target.

A subsequent analysis [Zha+25b] decomposes this objective into a trajectory flow-matching term and a trajectory-consistency term, and builds a family of objectives that anneals from the first to the second. Its target interpolates, by a consistency step ratio, between the instantaneous velocity and a stop-gradient prediction of the network itself; this thesis refers to it as *consistency-interpolated average-velocity matching*. Both average-velocity objectives achieve state-of-the-art one- and two-step image generation [Gen+25a; Zha+25b].

The DPCC reverse sampler uses its trained schedule, whereas the flow-based samplers choose their step budget at deployment. The average-velocity objectives are designed to improve accuracy at small budgets; their performance on the control tasks is measured in Chapter 6.

2.5. Constraint Enforcement in Generative Sampling

Model predictive control [ML99] optimises over a finite horizon, executes the first action and optimises again. A learned planner fits this loop, but its plans carry no guarantee for constraints the demonstrations never contained. Three approaches exist. Guidance adds the gradient of a constraint cost to the sampler [Car+25], which makes violations less likely without excluding them. Projection maps a sample onto the feasible set, after sampling [Pow+23] or at every step [CBF24; RRS25]; the projection of DPCC includes a model of the dynamics, so an exact feasible solution satisfies that model. Optimisation-based methods steer the sampler itself.

One such method [LAA25] steers a flow-matching sampler by treating sampling as an optimal control problem. A control input is added to the learned velocity field; its effort is penalised, so the controlled sampler stays close to the learned one; and the constraints are imposed on the final sample only, not on the path that leads to it. Solving that problem exactly would require simulating the rest of the trajectory at every step. The method avoids this in three moves. It splits the problem, as model predictive control (MPC) does, into one short problem per sampling step. It replaces the unknown final sample by its expected value given the current state, which for a flow-matching model follows in closed form from the state and the learned velocity. And it optimises directly over that predicted final sample instead of over the next state, so the constraints act on a plan and not on its image through the network. The corrected prediction is then mapped back into the next state of the sampler. At the final step the prediction and the state coincide, so an exact feasible projection yields a constraint-satisfying final sample. An optional terminal cost can further shape the sample, and the optimisation is applied only in the later sampling steps, where the prediction of the final sample is accurate.

The two approaches therefore differ in where the constraints act. Per-step projection corrects the sampler’s current iterate, which early in sampling is still far from any plan; this thesis calls it *iterate projection*. The optimal-control formulation corrects the plan the sampler is heading towards; this thesis calls it *endpoint projection*. Plans generated with neither are *unguided*.

2.6. Vision-Conditioned Policies

A policy that observes images compresses each camera view into a feature vector with a convolutional encoder and conditions the generative model on that vector. D3IL [Jia+24] provides such a vision pipeline for its tasks, following Diffusion Policy [Chi+23]: a ResNet-18 encoder [He+16] processes each camera view. The features are then concatenated with the time embedding of the network and reach its layers as a feature-wise conditional bias — the bias half of feature-wise linear modulation, feature-wise linear modulation (FiLM) [Per+17, Sec. 3], whose general form scales and shifts each channel of a layer from a conditioning vector. Image-generation transformers can operate on image-patch tokens [PX22]; trajectory transformers instead represent a sequence of planned states or actions.

2.7. Quadrotor Dynamics and Control

A fully actuated manipulator in position control holds its configuration through joint feedback; a planned sequence of end-effector positions becomes joint commands through inverse kinematics (IK). A quadrotor is different. Its four rotors produce a thrust along one body axis and three moments, so it controls four of its six degrees of freedom directly and must tilt in order to translate, and hover is an unstable equilibrium that requires continuous feedback.

A position-controlled arm can hold its last setpoint through the joint controller, although tracking error and collisions still depend on the commanded motion. A quadrotor must continuously regulate thrust and attitude even to hold a fixed position. Position corrections require tilting and translation, and a loss of attitude control can end the flight. This difference in tracking behaviour motivates testing the same planning framework on both plants.

Such an underactuated, open-loop unstable vehicle is controlled in cascade: an outer loop turns a position error into a desired thrust vector, which fixes a desired attitude, and an inner loop tracks that attitude. Geometric controllers formulate the attitude error directly on the rotation group and thereby avoid the singularities of local coordinates [LLM10]. Model predictive control is an alternative that optimises the

2. Background

rotor commands over a short horizon against a simulation of the vehicle; predictive sampling in MuJoCo MPC does so without derivatives, by rolling out perturbed control sequences and keeping the best [How+22]. The vehicle and its scenes are simulated in MuJoCo, a rigid-body physics engine for contact-rich robotic simulation [TET12].

3. Related Work

3.1. Diffusion Planners and Policies

Diffusion was first used to plan whole trajectories [Jan+22] and then to learn visuomotor policies [Chi+23]. Motion planning diffusion constructs cost functions for safe robot motion and biases the sampler towards them [Car+25]. DPCC combines a trajectory diffusion model with a model-based projection at every denoising step and with constraint tightening [RRS25]. Its planner runs as MPC: a one-dimensional temporal U-Net generates a state-action plan over a fixed horizon, a batch of candidate plans is drawn, the plan is made dynamically feasible against an Euler-integrated model of the plant, and one action is executed before the next plan is generated [RRS25, Sec. 5]. It is both the codebase this thesis starts from and the baseline it is measured against.

3.2. Flow Matching and Few-Step Policies

Flow matching [Lip+23; Lip+24] and rectified flow [LGL23] replace the denoising chain by an ordinary differential equation, and robot policies have adopted them to act in few steps. FlowPolicy uses consistency flow matching on 3-D point clouds [Zha+24]; ManiFlow combines flow matching with consistency training to generate dexterous actions in one or two steps from visual, language and proprioceptive inputs [Yan+25a]; Streaming Flow Policy treats the action trajectory itself as the flow trajectory [Jia+25b]. VFP captures multimodality with a variational latent prior [Zha+25a], real-time chunking enables smooth asynchronous execution of flow policies [BGL25], π_0 is a vision-language-action flow model for general robot control [Bla+24], and flow policies have also been trained with policy gradients [Yi+26]. Most of these policies enforce no hard constraints on the generated trajectory.

On the generative-modelling side, MeanFlow learns average velocities for one-step generation [Gen+25a], Improved MeanFlow recasts its objective as a loss on the instantaneous velocity with a network that predicts the average velocity [Gen+25b], and α -Flow unifies trajectory flow matching, shortcut models and MeanFlow in one family trained with a curriculum [Zha+25b].

3.3. Accelerating Generative Planners

Implicit samplers shorten the chain of a trained diffusion model [SME21], and distillation compresses it further: One-Step Diffusion Policy distils a pre-trained diffusion policy into a single-step action generator [Wan+24]. Both accelerate a model after it is trained. Consistency models [Son+23] and MeanFlow [Gen+25a] instead learn a few-step generator directly, and with flow matching the step budget remains a choice at deployment.

3.4. Constrained and Safe Generative Planning

Constraints on generated trajectories are enforced by guidance, by projection, or by acting inside the sampler. Guidance biases the sampler with the gradient of a cost [Car+25]. Projection maps a sample onto the feasible set after sampling [Pow+23] or at every step [CBF24; RRS25]. Among the methods that act inside the sampler, SafeFlowMatcher combines flow matching with control barrier functions [YJH25]; Physics-Constrained Flow Matching enforces arbitrary nonlinear constraints in pretrained flow-based models without retraining [Utk+25]; SAD-Flow generates safe, admissible and dynamically consistent trajectories [Hua+25]; UniConFlow incorporates equality and inequality constraints in one constrained flow-matching framework [Yan+25b]; XFlowMP is a task-conditioned generative motion planner built on Schrödinger bridges [NV25]; and SafeFlowMPC couples flow matching with online optimisation on a real seven-degree-of-freedom manipulator, generating joint-position trajectories that a real-time solver projects onto the safe set inside the sampling loop [Oel+26, Sec. III].

HardFlow reformulates hard-constrained sampling of flow-matching models as trajectory optimisation and projects the plan the sampler is predicted to reach [LAA25]. Its derivation decomposes the sampling problem the way MPC decomposes a horizon, into one subproblem per sampling step, and the resulting procedure is stated as an algorithm [LAA25, Alg. 1]. It applies this optimisation only in the later sampling steps. At a small step budget that convention leaves no step before the final one, and the method reduces to projecting a finished sample — a case that, to the author’s knowledge, has not been characterised.

HardFlow is evaluated on the same obstacle-avoidance task as DPCC: it follows the setup and the constraint geometry of [RRS25], counts projection after every sampling step — the projection of DPCC — among its baselines, and reports, beside the safety rate and the computation time, the number of steps to reach the target on the trials that end without a collision [LAA25, Sec. VII-A, Table II]. Its differences from this thesis therefore lie in the control loop rather than in the task: it plans a horizon of

sixteen steps and executes eight of them before replanning, fixes the step budget at ten Euler steps, states its plant as a linear model fitted to the training data, and solves the resulting program with an interior-point solver [LAA25, App. C]. Its planner is likewise a temporal U-Net [Jan+22].

3.5. Vision-Conditioned Imitation Learning

D3IL provides manipulation tasks with diverse human demonstrations, including vision-based variants [Jia+24], and X-IL, from the same group, explores the design space of imitation-learning policies [Jia+25a]. Visuomotor diffusion policies condition the denoising network on image features [Chi+23]. Image-generation transformers use patch tokens [PX22], while policy transformers can represent action sequences. Flow-matching policies follow the same path: VITA flows directly from visual representations to latent actions [Gao+26], and π_0 conditions a flow model on images and language [Bla+24].

3.6. Learning-Based Planning and Control for Aerial Vehicles

Imitation learning for aerial vehicles covers language-guided short-range flight, for which UAV-Flow Colosseo provides a real-world benchmark [Wan+25], and trajectory planning: CGD guides diffusion policies for unmanned aerial vehicle (UAV) trajectories with constraints never encountered in training [Kon+24]. Below the planner, geometric tracking control on the special Euclidean group tracks position and attitude without the singularities of local coordinates. Its position-controlled flight mode states the thrust magnitude and moment vector that track a position reference, the attitude they imply, and the stability of the resulting closed loop [LLM10, Eqs. (19)–(23), Prop. 3]; the cascade from a commanded force to individual rotor thrusts through a fixed allocation matrix is the structure of [MK11]. These are the equations the controller of Subsection 4.3.5 implements. Predictive sampling offers derivative-free model predictive control in simulation [How+22]. The quadrotor model used here is the Skydio X2 from MuJoCo Menagerie [ZTM22].

3.7. Positioning of This Thesis

The methods above differ less in their generative model than in the loop around it: whether a plan is executed once or inside a receding horizon, what the plan commands, where the constraints act, and which backbone carries the comparison. Table 3.1 states these for the works this thesis builds on and is measured against.

3. Related Work

Table 3.1.: The closest related work by the loop it runs in, not by its generative model. “Executed” is the number of planned steps applied before the next plan is generated. None of the sources states the parameter count of its network.

Work	Constraints	Loop and plan	Backbone and capacity
Diffuser [Jan+22]	cost guidance, no hard constraint	plan of states and actions, replanned	temporal U-Net, size not reported
Diffusion Policy [Chi+23]	none	receding horizon over a chunk of actions	U-Net or transformer, size not reported
DPCC [RRS25]	projection of every denoising iterate, tightened	receding horizon, horizon 8, 1 executed, Euler plant model	temporal U-Net, size not reported
HardFlow [LAA25]	projection of the predicted final sample, later steps only	receding horizon, horizon 16, 8 executed, fitted linear plant model	temporal U-Net, size not reported
SafeFlowMPC [Oel+26]	projection onto the safe set inside the sampling loop	receding horizon over joint-position trajectories	flow-matching policy, size not reported
This thesis	one feasible set, applied to the iterate or to the predicted endpoint	receding horizon, horizon 8, 1 executed, Euler plant model	matched temporal U-Net backbone; capacity in Section 5.6

Fast generative sampling, hard constraint enforcement, visual conditioning and aerial embodiment have each been studied, but in a receding-horizon controller the cost of fast sampling and the cost of constraint enforcement interact. The comparison here measures that interaction within the same planning framework. This thesis places instantaneous-velocity matching and the two average-velocity objectives inside the constrained controller of DPCC and compares its per-step projection with endpoint projection [LAA25] on the same feasible set, first on the benchmark of DPCC with its setup unchanged. It then carries the resulting controller to a vision-conditioned alignment task and to a quadrotor benchmark, both built for this thesis, and adds the step-budget condition under which endpoint projection reduces to projecting a finished sample.

4. Method

The method starts from DPCC and changes two of its parts: the generative model and the projection. The planning problem is formulated first, and the controller is then described from the plant upwards: the two plants and the controllers that drive them, the generative models and their network, the constraint projection, and the environments with their constraint sets.

4.1. Problem Formulation

4.1.1. The Control Problem

The plant is a discrete-time system with state $s_t \in \mathcal{S}$ and action $a_t \in \mathcal{A}$,

$$s_{t+1} = f(s_t, a_t) + w_t, \quad \|w_t\|_2 \leq \gamma, \quad (4.1)$$

where f is a known approximate model and w_t collects model mismatch. A goal g is reached when an indicator $\phi(s_t, g)$ equals one. Available is a dataset of N demonstrations produced by an unknown stochastic expert; the demonstrations are *multimodal*, meaning several distinct ways of reaching the goal are present, which is why a generative model rather than a regressor is used. At test time the system must in addition satisfy *novel* constraints

$$s_t \in \mathcal{S}_t \subseteq \mathcal{S}, \quad a_t \in \mathcal{A}_t \subseteq \mathcal{A}, \quad \forall t, \quad (4.2)$$

which are not assumed to hold in the demonstrations. This is the problem statement of [RRS25, Sec. 3], restated here because every claim in Chapter 6 is made against it.

4.1.2. The Plan Variable

At control step t the model proposes a horizon of H state–action pairs,

$$x = (s_{t:t+H-1|t}, a_{t:t+H-1|t}) \in \mathbb{R}^n, \quad n = Hd, \quad (4.3)$$

where $s_{t+i|t}$ denotes the prediction for time $t + i$ made at time t , and d is the per-step transition width. Every environment in this thesis plans $H = 8$ steps, so a plan is a

block of $8 \times d$ numbers, which the network reads as a length-8 signal of d channels (Subsection 4.4.6) and the projection as 8 horizon blocks (Subsection 4.5.1); d is 6, 9 and 12 in the three environments (Table 4.3). The generative model is conditional on a context c comprising the current observation and any task-specific conditioning. Known current-state entries of x are imposed by inpainting after each sampler step and masked in the training loss. The visual policy additionally conditions on image features; the box and target pose are observed through those images rather than supplied as known goal coordinates. The feasible set is the intersection of the plan-level constraints with the dynamics rows,

$$\mathcal{Z}_f = \{x \mid s_{t'|t} \in \tilde{\mathcal{S}}_{t'}, a_{t'|t} \in \mathcal{A}_{t'}, s_{t'+1|t} = f(s_{t'|t}, a_{t'|t}), \forall t'\}, \quad (4.4)$$

and the closed loop applies the first action $a_{t|t}$ of the selected plan and re-solves at $t + 1$.

4.1.3. Notation

This field’s papers overload t , u and x : control time, transport time, the velocity field and the control input all compete for the same letters. Table 4.1 fixes one convention for the thesis. Two of its decisions carry through every chapter. The plan is x , not τ as in [RRS25], which frees τ for transport time; and τ runs *data-at-one*, so the diffusion index k of Subsection 4.4.1 counts down to the data while the transport index k of Subsection 4.4.2 counts up to it. Every equation reproduced from a source that uses the opposite direction has been mapped, and the mapping is noted at the point of use.

Three further collisions arise once the control stack is written down, and the table resolves them by decoration rather than by renaming, so that each subfield keeps its own idiom. The generative velocity field is v and the drone’s linear velocity is $\dot{p}$ — never v . The body moment is τ_b , always bold and always subscripted; an undecorated τ is always transport time. The indexed α_k and $\bar{\alpha}_k$ belong to the diffusion schedule; the unindexed α is the consistency step ratio. The symbol g in the task indicator denotes a goal, while the scalar g in the quadrotor equations denotes gravitational acceleration. And $J(q)$ is the manipulator Jacobian while J_b is the drone’s inertia tensor; the two never appear in the same equation.

4.2. System Overview

At each control step, the observation conditions B candidate plans drawn from independent noise. The generative sampler advances each candidate, and the selected projection procedure constrains its iterates or predicted endpoint. Candidate selection

Table 4.1.: Notation used throughout. Vectors are bold lowercase, matrices bold uppercase, per the institute’s convention.

Symbol	Meaning	Range / type
t	control timestep (physical time)	$\mathbb{N}$
H	planning horizon: the number of steps in a plan; $H = 8$ throughout	$\mathbb{N}$
$\mathbf{s}_t, \mathbf{a}_t$	state, action	$\mathcal{S}, \mathcal{A}$
$\mathbf{s}_{t+i t}$	prediction for $t + i$ made at t	
$\mathbf{x}$	the plan, flattened	$\mathbb{R}^n$
$\mathbf{c}$	task-specific observation and conditioning	
τ	transport time; 0 prior, 1 data	$[0, 1]$
r	the second (start-side) transport time	$[0, \tau]$
h	interval length $\tau - r$	$[0, 1]$
k	sampler step index	$0, \dots, K$
K	sampling-step budget: denoising steps of the reverse process for the diffusion baseline, fixed with its schedule at training; steps of the ODE solver for the flow-based models, chosen at deployment	$\mathbb{N}$
NFE	network evaluations per candidate plan	$\mathbb{N}$
$\beta_k, \bar{\alpha}_k, \sigma_k$	diffusion schedule quantities	
v_θ	instantaneous velocity network	$\mathbb{R}^n$
u_θ	average velocity network	$\mathbb{R}^n$
ϵ_θ	noise-prediction network (baseline)	$\mathbb{R}^n$
$\mathcal{Z}_f$	feasible set, dynamics included	$\subset \mathbb{R}^n$
$\Pi_{\mathcal{Z}_f}$	projection (4.48)	
η	constraint activation threshold	$[0, 1]$
B	number of candidate plans	$\mathbb{N}$
γ	model-mismatch bound	$\mathbb{R}_{>0}$
<i>low-level control (Section 4.3)</i>		
$\mathbf{p}, \dot{\mathbf{p}}$	measured position, linear velocity	$\mathbb{R}^3$
$\mathbf{p}_{\text{des}}$	commanded setpoint (the plan’s integral)	$\mathbb{R}^3$
$\mathbf{q}$	manipulator joint positions	$\mathbb{R}^7$
$J(\mathbf{q})$	manipulator Jacobian	$\mathbb{R}^{6 \times 7}$
$\mathbf{R}, \boldsymbol{\omega}$	body attitude, body rates	$SO(3), \mathbb{R}^3$
$T, \boldsymbol{\tau}_b$	collective thrust, body moment	$\mathbb{R}, \mathbb{R}^3$
J_b	body inertia (never the Jacobian)	$\mathbb{R}^{3 \times 3}$
t_s	sampling time of the dynamics model	$\mathbb{R}_{>0}$

then chooses one plan. Its first position increment updates the commanded setpoint, which the low-level controller tracks before the next observation is acquired:

observation and initial candidates $\rightarrow$ sampling with projection $\rightarrow$ candidate selection
 $\rightarrow$ first increment $\rightarrow$ setpoint and tracking $\rightarrow$ next observation.

The generative objective and projection procedure are the two varied components. The models use matched temporal U-Net backbones and the same task data and plan representation. Training settings and the evaluated projection thresholds are specified in Section 5.6; architecture matching alone does not make the training recipes identical.

Table 4.2.: Provenance of the planning and control components.

Component	Inherited basis	Work in this thesis
Planning loop	DPCC receding-horizon execution, normalisation, candidate selection and model-based projection [RRS25]	Integration of the flow-based samplers
Generative models	Published instantaneous- and average-velocity objectives [Lip+23; Gen+25a; Zha+25b]	Conditional trajectory models with matched temporal U-Net backbones
Endpoint projection	Predicted-endpoint control formulation [LAA25]	Projection on the task’s feasible set and analysis of the guiding-step count
Visual task	D3IL task and demonstrations; visual encoder design [Jia+24; Chi+23]	Camera-conditioned models and task constraints
Aerial tasks	MuJoCo vehicle; geometric and predictive tracking [ZTM22; LLM10; How+22]	Scenes, planar transfer, spatial demonstrations and planning/control integration

4.3. Plant Dynamics and Control

Two plants are controlled: a manipulator, whose end-effector position the generative model plans, and a quadrotor, whose position it plans. The projection uses a first-order model of the manipulator; the quadrotor is described by its rigid-body dynamics. Underneath both sits a low-level controller, because the generative model plans position increments and never an actuator command: the manipulator uses differential IK and a joint-space proportional–derivative (PD) law, the quadrotor a cascaded geometric tracking controller or, as a baseline, sampling-based predictive control.

4.3.1. The Manipulator and Its First-Order Model

The projection of Subsection 4.5.1 needs a concrete dynamics model f for its rows (4.49). The inherited system does not use an identified model of the manipulator and its low-level controller; it uses a first-order (explicit Euler) integrator over the commanded quantity, and pushes everything it omits into the mismatch term w_t of (4.1). For the two-dimensional manipulator benchmark, where the state stacks a commanded and a measured planar position and the action is a planar increment, that model is

$$s_{t+1} = s_t + \begin{bmatrix} a_t^T & a_t^T \end{bmatrix}^T t_s + w_t, \quad (4.5)$$

i.e. *both* halves of the state are advanced by the *same* action [RRS25, Sec. 6.1]. Three things follow, and all three are inherited by every environment in this thesis.

- **The model carries no knowledge of the robot.** f knows nothing of the arm’s kinematics, its impedance controller or its integration error. What it omits appears as the mismatch w_t , whose bound γ is estimated from unconstrained rollouts, and the tightening of (4.60) absorbs it. Feasibility is therefore a statement about the model and its mismatch bound, not about the plant.
- $t_s = 1$, **because the action is a displacement and not a velocity.** The network’s output is a per-step Cartesian increment, so the Euler update is already in the right units and no physical timestep enters the constraint. Setting t_s to the simulator’s integration step — a natural-looking mistake — would rescale the entire feasible set.
- **Anchoring both the commanded and the measured channel is what couples the plan to the plant.** If only the measured position were constrained, the projection could return a plan whose commanded channel is unreachable; if only the commanded one were, the plan would be admissible in a space the robot never occupies. Section 4.3 shows the same two-channel structure in all three environments, and Subsection 4.5.1 gives the row form.

Remark 4.1 (Scope of the dynamics model). The integrator constrains Cartesian plan increments. Joint limits, actuator dynamics and tracking errors are represented only through the mismatch term. The same plan representation supports both embodiments; executed constraint satisfaction is evaluated on the simulated plant.

4.3.2. The Quadrotor

The vehicle is the rigid body of [LLM10, Eqs. (2)–(5)]: with mass m , body inertia J_b , attitude $R \in SO(3)$ and body rates ω . Expressed in the simulation’s z-up world frame

with thrust along positive body-z, its equations are

$$m\ddot{\mathbf{p}} = -mg\mathbf{e}_3 + T\mathbf{Re}_3, \quad \dot{\mathbf{R}} = \mathbf{R}\hat{\boldsymbol{\omega}}, \quad \mathbf{J}_b\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times \mathbf{J}_b\boldsymbol{\omega} = \boldsymbol{\tau}_b, \quad (4.6)$$

where $\hat{(\cdot)}$ is the hat map. The gravity and thrust signs reverse those of the source's downward-axis convention.

The four rotor thrusts produce the collective thrust T along the body axis $\mathbf{Re}_3$ and the body moment $\boldsymbol{\tau}_b$. With four inputs for six degrees of freedom the vehicle is underactuated: it translates only by tilting its thrust vector, and hover is an unstable equilibrium. The generative model never commands these inputs; it plans positions, and the controller of Subsection 4.3.5 turns them into rotor thrusts.

4.3.3. From Plan to Setpoint

In every case the generative model replans at each control step, emits a *position increment*, and never emits an actuator command; everything below the increment is classical control — a differential IK loop feeding a joint-space PD law on the two manipulator environments, and a cascaded *geometric* tracking controller, or optionally the sampling-based tracker of MuJoCo MPC, on the quadrotor.

Write $\Delta\mathbf{p}_t$ for the first action of the selected plan at control step t . All three environments run the same two-line recursion,

$$\mathbf{p}_{\text{des},t+1} = \mathbf{p}_{\text{des},t} + \Delta\mathbf{p}_t, \quad \mathbf{p}_{\text{des},0} = \mathbf{p}_0, \quad (4.7)$$

$$\mathbf{o}_t = \left(\mathbf{p}_{\text{des},t}, \mathbf{p}_t, [\dot{\mathbf{p}}_t] \right), \quad (4.8)$$

where $\mathbf{o}_t$ is the observation the model conditions on at the next step and the bracketed velocity block is present for UAV-corridor and UAV-s-curve; UAV-pillars uses the planar observation through the map in Subsection 4.6.3. Two properties follow.

- (4.7) is a **free-running integrator driven by the model's own past outputs**. $\mathbf{p}_{\text{des}}$ is not read back from the plant. Past increments accumulate in the commanded setpoint. The next plan can respond to the tracking error because both commanded and measured positions are observed.
- (4.8) **feeds the commanded setpoint back alongside the measured state**, so the model conditions on its own history. The gap $\mathbf{p}_{\text{des},t} - \mathbf{p}_t$ is exactly the tracking error the low-level controller is working on, and it is visible to the planner — which is why the two channels are both constrained in (4.54).

Table 4.3.: The same skeleton, instantiated three times. The plan width d is the width of the plan variable (4.3) and $\mathbf{o}$ its observation block; t_s is the Euler sampling time of (4.5); “dynamics rows” counts the channel pairs of (4.54); the two intervals are simulated time (4.12). UAV-pillars flies the first column: planner, plan, observation and dynamics rows are those of D3IL-avoiding, and only the plant is the quadrotor (Subsection 4.6.3). The quantities measured on these loops are defined in Section 5.4.

	D3IL-avoiding	D3IL-aligning	UAV-corridor, UAV-s-curve
embodiment	manipulator, planar	manipulator, spatial	quadrotor
observation	state, 4-D	2 RGB views + 6-D state	state, 9-D
action	$\Delta \mathbf{p} \in \mathbb{R}^2$	$\Delta \mathbf{p} \in \mathbb{R}^3$	$\Delta \mathbf{p} \in \mathbb{R}^3$
plan width d	6	9	12
$\mathbf{o}$ layout	$(\mathbf{p}_{\text{des}}, \mathbf{p})$	$(\mathbf{p}_{\text{des}}, \mathbf{p})$	$(\mathbf{p}_{\text{des}}, \mathbf{p}, \dot{\mathbf{p}})$
t_s	1	1	1
dynamics rows	4	6	6
low level	differential IK $\rightarrow$ joint PD	same	geometric cascade, or MuJoCo MPC
plan interval	one plan per environment step	one plan per environment step	0.03 s simulated
inner interval	n_{sub} simulator substeps	n_{sub} simulator substeps	0.01 s simulated

4.3.4. Manipulator: Differential Inverse Kinematics

The environment is commanded with an absolute Cartesian pose. The evaluation loop forms it from the increment and the *commanded* channel of the observation — not from the measured one, which is what makes (4.7) the free-running recursion it is — and holds the end-effector orientation and, on the planar benchmark, the height fixed.

Below that setpoint sits the differential IK loop. Let $\mathbf{q} \in \mathbb{R}^7$ be the joint positions of the Panda, $\mathbf{J}(\mathbf{q})$ the end-effector Jacobian, and let the task-space error be a proportional term on position and orientation,

$$\boldsymbol{\xi}(\mathbf{q}) = \begin{bmatrix} k_p \odot \text{clip}(\mathbf{p}_{\text{des}} - \mathbf{p}(\mathbf{q}), \pm \epsilon_p) \\ k_q \odot \text{clip}(\mathbf{e}_{\text{rot}}(\mathbf{q}), \pm \epsilon_r) \end{bmatrix}, \quad (4.9)$$

with $\mathbf{p}(\mathbf{q})$ from forward kinematics and $\mathbf{e}_{\text{rot}}$ the quaternion error to the commanded orientation. The joint increment is the weighted, damped least-squares solution with a null-space term that pulls the arm towards a rest posture,

$$\dot{\mathbf{q}}_{\text{null}} = k_{\text{null}} (\mathbf{q}_{\text{rest}} - \mathbf{q}), \quad (4.10)$$

$$\dot{\mathbf{q}} = \mathbf{W} \mathbf{J}^T \left(\mathbf{J} \mathbf{W} \mathbf{J}^T + \lambda \mathbf{Id}_6 \right)^{-1} (\boldsymbol{\xi} - \mathbf{J} \dot{\mathbf{q}}_{\text{null}}) + \dot{\mathbf{q}}_{\text{null}}, \quad (4.11)$$

after which the singular values of $\mathbf{J} \mathbf{W} \mathbf{J}^T + \lambda \mathbf{Id}_6$ are clipped into $[\sigma_{\min}, \sigma_{\max}]$ before the solve, $\|\dot{\mathbf{q}}\|$ is capped, and $\mathbf{q} \leftarrow \text{clip}(\mathbf{q} + \eta_{\text{ik}} \dot{\mathbf{q}}, \mathbf{q}_{\min}, \mathbf{q}_{\max})$ is iterated n_{ik} times per control step. The resulting joint target, its finite-difference velocity and acceleration are handed to a joint-space PD law, and *that* is what produces the torques the simulator integrates.

The parameters are inherited from the benchmark and are listed once because the loop is otherwise not reproducible: $k_p = (200, 200, 800)$, $k_q = (30, 30, 30)$, $k_{\text{null}} = 40$, $\lambda = 10^{-12}$, $\mathbf{W} = \mathbf{Id}_7$, $[\sigma_{\min}, \sigma_{\max}] = [10^{-2}, 10^2]$, $n_{\text{ik}} = 3$, $\eta_{\text{ik}} = 10^{-3}$, clips $\epsilon_p = 0.01$ m and $\epsilon_r = 0.1$, and joint PD gains $(120, 120, 120, 120, 50, 30, 10) / (10, 10, 10, 10, 6, 5, 3)$.

Two consequences matter for the thesis. (4.11) is a *local* solve run three times per step, so the arm does not exactly reach $\mathbf{p}_{\text{des}}$ within one step — the residual is precisely the $\mathbf{w}_t$ that (4.5) pushes into the mismatch bound and (4.60) then absorbs. And the null-space term makes the map from setpoint to joint configuration history-dependent, so two identical plans executed from different postures do not produce identical joint trajectories. They request the same Cartesian reference, while the executed trace also depends on tracking error.

4.3.5. Quadrotor: Cascaded Geometric Tracking Control

On UAV-corridor and UAV-s-curve, the aerial chain runs on two intervals of simulated time: a plan is queried every n_{dec} -th physics step, the interval at which the expert

demonstrations were recorded, and the low-level controller runs at every physics step, with the setpoint held constant in between,

$$n_{\text{dec}} \Delta t_{\text{phys}} = 0.03 \text{ s}, \quad \Delta t_{\text{phys}} = 0.01 \text{ s} \Rightarrow n_{\text{dec}} = 3. \quad (4.12)$$

The simulation waits for the planner, so these intervals hold in simulated time for any planning time. UAV-pillars uses the separate control period specified in Section 5.6.

The controller is the *position-controlled flight mode* of [LLM10, Eqs. (19)–(23)], instantiated on (4.6) in a z-up frame with the thrust along $+\mathbf{b}_3$, and closed by the rotor allocation of [MK11]. It runs in four steps.

Outer loop — position to thrust vector. With the tracking errors $\mathbf{e}_p = \mathbf{p} - \mathbf{p}_{\text{des}}$ and $\mathbf{e}_v = \dot{\mathbf{p}} - \dot{\mathbf{p}}_{\text{des}}$ [LLM10, Eqs. (17)–(18)], a diagonal PD with acceleration feed-forward gives the required world-frame force

$$\mathbf{a}_{\text{cmd}} = -\mathbf{K}_p \mathbf{e}_p - \mathbf{K}_d \mathbf{e}_v + \ddot{\mathbf{p}}_{\text{des}}, \quad \mathbf{F} = m (\mathbf{a}_{\text{cmd}} + g \mathbf{e}_3), \quad (4.13)$$

which is the bracketed term of [LLM10, Eq. (23)]. The gain matrices are diagonal, with a stiffer vertical channel.

Attitude extraction. The desired rotor-disk normal points along $\mathbf{F}$ and the remaining degree of freedom is fixed by the commanded yaw ψ_{des} , which pins the free axis left open by [LLM10, Eqs. (22)–(23)]:

$$\mathbf{b}_{3,\text{des}} = \frac{\mathbf{F}}{\|\mathbf{F}\|}, \quad \mathbf{b}_{2,\text{des}} = \frac{\mathbf{b}_{3,\text{des}} \times \mathbf{x}_c}{\|\mathbf{b}_{3,\text{des}} \times \mathbf{x}_c\|}, \quad \mathbf{b}_{1,\text{des}} = \mathbf{b}_{2,\text{des}} \times \mathbf{b}_{3,\text{des}}, \quad \mathbf{R}_{\text{des}} = [\mathbf{b}_{1,\text{des}} \quad \mathbf{b}_{2,\text{des}} \quad \mathbf{b}_{3,\text{des}}], \quad (4.14)$$

with $\mathbf{x}_c = (\cos \psi_{\text{des}}, \sin \psi_{\text{des}}, 0)^T$.

Inner loop — geometric attitude control. The attitude error is the coordinate-free error on $SO(3)$ of [LLM10, Eq. (21)], and the moment is its [LLM10, Eq. (20)]:

$$\mathbf{e}_R = \frac{1}{2} \left(\mathbf{R}_{\text{des}}^T \mathbf{R} - \mathbf{R}^T \mathbf{R}_{\text{des}} \right)^\vee, \quad \boldsymbol{\tau}_b = -\mathbf{K}_R \mathbf{e}_R - \mathbf{K}_\omega \mathbf{e}_\omega + \boldsymbol{\omega} \times \mathbf{J}_b \boldsymbol{\omega}, \quad (4.15)$$

where $(\cdot)^\vee$ is the inverse hat map. The commanded angular velocity is held at zero, so $\mathbf{e}_\omega = \boldsymbol{\omega}$ and the attitude feed-forward of [LLM10, Eq. (20)] vanishes; the full tracking law assumed by [LLM10, Prop. 3] therefore differs from this zero-angular-reference implementation.

Allocation. The collective thrust is the world force projected on the current body-z axis with a positive lower bound, and the wrench is distributed over the four rotors by a fixed geometry matrix:

$$T = \max\left(\mathbf{F}^\top \mathbf{R} \mathbf{e}_3, T_{\text{floor}}\right), \quad \mathbf{M} \mathbf{u} = \begin{bmatrix} T \\ \boldsymbol{\tau}_b \end{bmatrix}, \quad \mathbf{M}_{:,i} = [1 \quad r_{i,y} \quad -r_{i,x} \quad \kappa_i]^\top, \quad (4.16)$$

with $\mathbf{r}_i$ the i -th rotor site position and κ_i its yaw-torque gear ratio, both read from the model file, so (4.16) holds for an arbitrary rotor placement [LLM10, Eq. (1)][MK11]. When $\mathbf{u} = \mathbf{M}^{-1}[T; \boldsymbol{\tau}_b]$ leaves the actuator box, the command is rescaled thrust-first: the mean is held and the torque component is shrunk by the largest factor that keeps every rotor in range, but never below one half. A final componentwise clipping enforces the actuator bounds; it can change both the achieved collective thrust and moment. Parameters of record: $\mathbf{K}_p = \text{diag}(4, 4, 8)$, $\mathbf{K}_d = \text{diag}(3, 3, 4)$, $\mathbf{K}_R = \text{diag}(70, 70, 4)$, $\mathbf{K}_\omega = \text{diag}(2.5, 2.5, 1)$, $u_{\text{hover}} = mg/4$, $u_{\text{max}} = \max(2u_{\text{hover}}, 6)$, $u_{\text{min}} = 0$, $T_{\text{floor}} = 0.1 mg$, $g = 9.81 \text{ m/s}^2$; m and $\mathbf{J}_b$ are read from the vehicle model. The loop has no integral term: the four gains act on position, velocity, attitude and angular rate, and the inner loop is the coordinate-free error on $SO(3)$ of (4.15), closed by the static allocation (4.16). The thesis therefore calls it a cascaded geometric tracking controller; pid in the run tags is an artefact token.

The velocity setpoint is synthesised, not planned. (4.13) needs $\dot{\mathbf{p}}_{\text{des}}$, which the plan does not supply — it supplies displacements. Three setpoint constructions are implemented:

$$\dot{\mathbf{p}}_{\text{des}} = \begin{cases} \frac{\Delta \mathbf{p}_t}{n_{\text{dec}} \Delta t_{\text{phys}}} & \text{displacement-derived velocity,} \\ \mathbf{0} & \text{zero velocity feed-forward,} \\ \frac{\Delta \mathbf{p}_t}{\|\Delta \mathbf{p}_t\|} \bar{v} & \text{constant-speed, with } \bar{v} = \overline{\|\Delta \mathbf{p}\|} / (n_{\text{dec}} \Delta t_{\text{phys}}) \text{ from the dataset.} \end{cases} \quad (4.17)$$

The denominator of the displacement-derived policy is the fixed dataset interval, not measured inference time. The constant-speed policy uses the dataset’s mean displacement magnitude over that same interval; the zero-velocity policy has no velocity feed-forward. Their average commanded speeds therefore need not agree.

The cascaded geometric controller uses zero velocity feed-forward in the reported aerial evaluations. MuJoCo MPC is evaluated separately in the s-curve controller comparison (Section 6.3). With $\dot{\mathbf{p}}_{\text{des}} = \mathbf{0}$, position error drives the outer loop while its velocity feedback damps the motion. Tracking lag is measured on the executed flights.

4.3.6. Sampling-Based Predictive Control

A sampling-based predictive controller is implemented behind the same interface, so it can be swapped for the geometric cascade without touching the plan layer. It is the *Predictive Sampling* planner of MuJoCo MPC [How+22], used as published; only the task is ours. Three pieces define it.

The objective. The planner minimises the finite-horizon sum $J = \sum_i c(\mathbf{x}_i, \mathbf{u}_i)$ over a rollout of the true simulator dynamics [How+22, Eq. (3)], with the running cost a weighted sum of twice-differentiable norms of *residuals* — quantities that are small when the task is solved [How+22, Eq. (4)]. The task defined here has two residual terms, both under a quadratic norm and at risk-neutral setting ($R = 0$ in [How+22, Eq. (5)], so the risk transform is the identity):

$$J = \sum_{i=1}^{H_{\text{mjpc}}} \left(\underbrace{\|\mathbf{p}_i - \mathbf{p}_{\text{des}}\|_2^2}_{r_1, w_1=1} + \underbrace{w_v \|\dot{\mathbf{p}}_i\|_2^2}_{r_2, w_2=w_v} \right), \quad (4.18)$$

where H_{mjpc} counts rollout steps and $\mathbf{p}_{\text{des}}$ is the reference. The two terms penalise position error and translational speed.

The plan and its parameterisation. The plan is a time-indexed spline over control knots $\boldsymbol{\theta}$ rather than a raw action sequence [How+22, §3.3]; the zero-order-hold interpolant is used here, with one knot per horizon step, so the spline degenerates to a piecewise-constant action sequence.

The search. Each call performs a fixed number of iterations of Algorithm 4 of [How+22]: draw $N - 1$ perturbations $\boldsymbol{\theta}_i \sim \mathcal{N}(\boldsymbol{\theta}, \sigma^2)$, roll out all N candidates — the incumbent included — from the measured state, and keep $\boldsymbol{\theta} \leftarrow \arg \min_i J(\boldsymbol{\theta}^{(i)})$. It is derivative-free and zero-order; its authors present it as an elementary baseline. Between control steps the plan is shifted forward and re-sampled, which is the warm-start of [How+22, Alg. 1] and the reason a few iterations per step suffice.

Prediction model. Contacts are disabled in the planner’s copy of the model. The tracker therefore plans against the vehicle’s free-flight dynamics only, and obstacle avoidance remains entirely the projection’s job. The constraint mechanism stays one identifiable component, and exchanging the tracker leaves the way constraints are enforced unchanged. Defaults: $N = 16$, rollout duration $T_{\text{mjpc}} = 0.3$ s, with $H_{\text{mjpc}} = T_{\text{mjpc}} / \Delta t_{\text{phys}}$, five improvement iterations per call, $\sigma = 0.3$, $w_v = 0.1$.

The controller comparison in Section 6.3 distinguishes flights tracked by the geometric cascade from those tracked by MuJoCo MPC. The per-scene choices are recorded in Section 5.6.

4.4. Generative Models

The diffusion model of DPCC is the baseline. Instantaneous-velocity matching, analytic average-velocity matching and consistency-interpolated average-velocity matching replace it behind one interface, each with its own training target and integration loop.

4.4.1. Diffusion and the DPCC Baseline

Let $\mathbf{x} \in \mathbb{R}^n$ be the flattened plan of (4.3). A denoising diffusion probabilistic model [Soh+15; HJA20] introduces latent variables $\mathbf{x}^1, \dots, \mathbf{x}^K$ through a fixed Gaussian forward process

$$q(\mathbf{x}^k | \mathbf{x}^{k-1}) = \mathcal{N}(\sqrt{1 - \beta_k} \mathbf{x}^{k-1}, \beta_k \mathbf{Id}_n), \quad k = 1, \dots, K, \quad (4.19)$$

whose step sizes $\beta_{1:K} \in (0, 1)^K$ are set by a *noise schedule*. Because the transitions are Gaussian, the marginal is available in closed form,

$$q(\mathbf{x}^k | \mathbf{x}^0) = \mathcal{N}(\sqrt{\bar{\alpha}_k} \mathbf{x}^0, (1 - \bar{\alpha}_k) \mathbf{Id}_n), \quad \alpha_k = 1 - \beta_k, \quad \bar{\alpha}_k = \prod_{i=1}^k \alpha_i, \quad (4.20)$$

and the schedule is chosen so that $q(\mathbf{x}^K | \mathbf{x}^0) \approx \mathcal{N}(\mathbf{0}, \mathbf{Id}_n)$. In the standard formulation, generation reverses (4.19) with a learned Markov chain

$$p_\theta(\mathbf{x}^{k-1} | \mathbf{x}^k, k, \mathbf{c}) = \mathcal{N}(\boldsymbol{\mu}_\theta(\mathbf{x}^k, k, \mathbf{c}), \sigma_k^2 \mathbf{Id}_n), \quad p(\mathbf{x}^K) = \mathcal{N}(\mathbf{0}, \mathbf{Id}_n), \quad (4.21)$$

with $\mathbf{c}$ the conditioning context and the variance pinned to the forward posterior, $\sigma_k^2 = \beta_k(1 - \bar{\alpha}_{k-1}) / (1 - \bar{\alpha}_k)$. The mean is learned indirectly by predicting the noise that was added [HJA20], in the form restated by [RRS25, Sec. 4]:

$$\mathcal{L}(\theta) = \mathbb{E}_{k \sim \mathcal{U}\{1, K\}, \mathbf{x}^0 \sim q, \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{Id}_n)} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_k} \mathbf{x}^0 + \sqrt{1 - \bar{\alpha}_k} \boldsymbol{\epsilon}, k) \right\|_2^2. \quad (4.22)$$

At sampling time the noise prediction is turned back into a mean in two steps — an estimate of the clean sample, and the forward posterior evaluated at that estimate:

$$\hat{\mathbf{x}}^0(\mathbf{x}^k, k) = \frac{1}{\sqrt{\bar{\alpha}_k}} \mathbf{x}^k - \sqrt{\frac{1}{\bar{\alpha}_k} - 1} \boldsymbol{\epsilon}_\theta(\mathbf{x}^k, k, \mathbf{c}), \quad (4.23)$$

$$\boldsymbol{\mu}_\theta(\mathbf{x}^k, k, \mathbf{c}) = \frac{\beta_k \sqrt{\bar{\alpha}_{k-1}}}{1 - \bar{\alpha}_k} \hat{\mathbf{x}}^0 + \frac{(1 - \bar{\alpha}_{k-1}) \sqrt{\bar{\alpha}_k}}{1 - \bar{\alpha}_k} \mathbf{x}^k. \quad (4.24)$$

In the inherited DPCC sampler, K indexes the trained schedule. Changing that configuration requires a separately trained baseline checkpoint; the reduced-budget baseline comparisons in Subsubsection 6.1.2.1 use such checkpoints. An accelerated

sampler such as DDIM can instead reuse a trained diffusion network [SME21], but is a different sampling procedure. The flow-based models studied here expose the integration budget directly at inference.

The standard reverse chain is stochastic at nonterminal steps; no fresh noise is injected at the last step. The evaluated baseline also clips its clean-sample estimate before computing the posterior mean. Its initialization and reverse-noise settings are given in Section 5.6; (4.21) states the standard background model.

The instance this thesis is measured against uses the cosine schedule of [ND21] with $K = 20$ and a temporal U-Net denoiser [Jan+22], conditioned on the current state by inpainting rather than by classifier or classifier-free guidance [DN21; HS22], following [RRS25, Sec. 6.1].

4.4.2. Flow Matching

Transport, and the field that generates it

Let $\tau \in [0, 1]$ be the *transport time*, with $\tau = 0$ at the prior and $\tau = 1$ at the data — the convention used throughout this thesis and by this work’s implementation.¹ A time-dependent velocity field $v_\tau : \mathbb{R}^n \rightarrow \mathbb{R}^n$ defines a flow Φ through the ordinary differential equation

$$\frac{d}{d\tau} x_\tau = v_\tau(x_\tau), \quad x_0 \sim p_0, \quad (4.25)$$

and the pushforward $p_\tau = (\Phi_{0 \rightarrow \tau})_\# p_0$ is the probability path it generates, characterised by the continuity equation $\partial_\tau p_\tau + \nabla \cdot (p_\tau v_\tau) = 0$ [Lip+23, Sec. 2]. Sampling is therefore integration: draw $x_0 \sim p_0$ and solve (4.25) to $\tau = 1$.

The objective, and why it is tractable

Flow matching [Lip+23, Eq. (5)] regresses a network v_τ^θ onto the field that generates a prescribed path:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{\tau \sim \mathcal{U}[0,1], x \sim p_\tau} \left\| v_\tau^\theta(x) - v_\tau(x) \right\|_2^2. \quad (4.26)$$

This is not computable: the marginal field v_τ is an integral over the data distribution. The construction that makes it usable is to build the path as a mixture of *per-sample* paths. Conditioning on a data point x_1 , one designs $p_\tau(x | x_1)$ and a conditional field

¹The opposite convention (noise at $\tau = 1$) is equally common; [Gen+25a] uses it. Every equation reproduced here has been mapped into the data-at-one convention, and the sign changes this induces are noted where they occur.

$v_\tau(\mathbf{x} \mid \mathbf{x}_1)$ that generates it; the marginals are then

$$p_\tau(\mathbf{x}) = \int p_\tau(\mathbf{x} \mid \mathbf{x}_1) q(\mathbf{x}_1) d\mathbf{x}_1, \quad v_\tau(\mathbf{x}) = \int v_\tau(\mathbf{x} \mid \mathbf{x}_1) \frac{p_\tau(\mathbf{x} \mid \mathbf{x}_1) q(\mathbf{x}_1)}{p_\tau(\mathbf{x})} d\mathbf{x}_1, \quad (4.27)$$

and the marginal field so defined does generate the marginal path [Lip+23, Theorem 1]. The *conditional* objective

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{\tau \sim \mathcal{U}[0,1], \mathbf{x}_1 \sim q, \mathbf{x} \sim p_\tau(\cdot \mid \mathbf{x}_1)} \left\| v_\tau^\theta(\mathbf{x}) - v_\tau(\mathbf{x} \mid \mathbf{x}_1) \right\|_2^2 \quad (4.28)$$

is an unbiased surrogate: it has the same gradient in θ as (4.26) [Lip+23, Theorem 2]. Training is therefore *simulation-free* — no trajectory is ever integrated to form a target.

The linear path

This thesis uses the linear (optimal-transport displacement, “rectified”) path [Lip+23; LGL23], with the noise–data coupling drawn independently:

$$\mathbf{x}_\tau = (1 - \tau) \mathbf{x}_0 + \tau \mathbf{x}_1, \quad v_\tau(\mathbf{x}_\tau \mid \mathbf{x}_0, \mathbf{x}_1) = \mathbf{x}_1 - \mathbf{x}_0, \quad (4.29)$$

i.e. the special case $\sigma_{\min} = 0$ of [Lip+23, Eq. (20)–(23)], read in the data-at-one direction. Substituting (4.29) into (4.28) gives the objective this work trains:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{\tau, \mathbf{x}_1 \sim q, \mathbf{x}_0 \sim p_0} \left\| v_\theta((1 - \tau) \mathbf{x}_0 + \tau \mathbf{x}_1, \tau, \mathbf{c}) - (\mathbf{x}_1 - \mathbf{x}_0) \right\|_2^2. \quad (4.30)$$

The regression target is a difference of two draws. Nothing in it depends on a schedule, and nothing in it depends on how many steps the sampler will later take.

Sampling, and the two consequences that matter here

With v_θ trained, a plan is produced by integrating (4.25). With the explicit Euler scheme at a uniform grid $\tau_k = k/K$,

$$\mathbf{x}_{\tau_{k+1}} = \mathbf{x}_{\tau_k} + \frac{1}{K} v_\theta(\mathbf{x}_{\tau_k}, \tau_k, \mathbf{c}), \quad k = 0, \dots, K-1, \quad (4.31)$$

which costs K network evaluations per candidate for unguided Euler sampling. Here K denotes sampling steps; endpoint lookahead adds evaluations as described in Subsection 4.5.3. The initialization distribution used at inference is specified with the experiment settings in Section 5.6. Two consequences follow.

The budget moved. K appears in (4.31) and nowhere in (??). It is a solver argument, chosen after training and changeable per deployment, in contrast to (4.19)–(4.22) where it indexes the schedule the model was fitted to. What limits small K is now discretisation

error of (4.25), which is a property of the field’s curvature rather than of the training objective — and that is the quantity analytic average-velocity matching addresses (Subsection 4.4.3).

The sampler became deterministic. (4.31) injects no noise: the plan is a deterministic function of x_0 and c for a fixed sampler. Its transport therefore adds no further random draws. The candidate plans of Subsection 4.5.5 therefore differ in x_0 , not in sampler noise.

Implementation

Instantaneous-velocity matching uses the interpolant of (4.29) and the regression loss of (??), with the known current-state entries masked. Sampling is (4.31). Relative to the baseline this deletes the schedule buffers, the noise-to-sample conversion (4.23) and the posterior mean (4.24), and it deletes the noise injected at every reverse step. The conditioning mask, the loss reduction and the backbone are untouched.

The transport time is not drawn uniformly, and the three models do not draw it alike. Instantaneous-velocity matching puts its mass towards $\tau = 0$, the noise end of the path, where the interpolant carries least of the data and the field is hardest to fit; the two average-velocity objectives draw the pair (τ, r) with its mass towards $\tau = 1$, the data end. Their sampled pairs define the interval used by the average-velocity target. The distributions and their parameters are configuration values and are given with the training protocol in Section 5.6.

4.4.3. Analytic Average-Velocity Matching

Average velocity

Following [Gen+25a, Eq. (3)], define the average velocity along the trajectory through x_r over $[r, \tau]$. Write $h = \tau - r$ and use the start and interval length as the field’s time arguments:

$$u(x_r, r, h) \triangleq \frac{1}{h} \int_r^{r+h} v(x_\zeta, \zeta) d\zeta = \frac{x_{r+h} - x_r}{h}, \quad h > 0. \quad (4.32)$$

The exact field is induced by v , with continuous extension $u(x_r, r, 0) = v(x_r, r)$. Additivity gives, for $r < s < \tau$,

$$(\tau - r)u(x_r, r, \tau - r) = (s - r)u(x_r, r, s - r) + (\tau - s)u(x_s, s, \tau - s).$$

The exact field traverses the interval in one step. Replacing it by a learned approximation gives

$$\hat{x}_1 = x_0 + u_\theta(x_0, 0, 1), \quad (4.33)$$

where $\hat{x}_1$ approximates the endpoint of the exact flow. The interval is a network input; no trajectory integral is evaluated at inference.

The average-velocity identity

Differentiate $(\tau - r)u(x_r, r, \tau - r) = \int_r^\tau v \, d\zeta$ along the trajectory with respect to r , holding τ fixed. The product rule gives the start-anchored form of [Gen+25a, Eqs. (4)–(6)],

$$u(x_r, r, h) = v(x_r, r) + h \frac{d}{dr} u(x_r, r, \tau - r), \quad h = \tau - r. \quad (4.34)$$

Here $dx_r/dr = v$ and $dh/dr = -1$, so

$$\frac{d}{dr} u(x_r, r, \tau - r) = (\partial_x u) v(x_r, r) + \partial_r u - \partial_h u. \quad (4.35)$$

This is a JVP with tangent $(v, +1, -1)$ in the inputs (x_r, r, h) . The plus sign in (4.34) follows from differentiating the lower limit, whereas [Gen+25a, Eq. (6)] uses an end-anchored field in the opposite time direction.

The implemented target substitutes the conditional velocity $v = x_1 - x_0$ of (4.29) and stops its gradient:

$$\begin{aligned} \mathcal{L}_{\text{avg}}(\theta) &= \mathbb{E} \|u_\theta(x_r, r, h) - \text{sg}(u_{\text{tgt}})\|_2^2, \\ u_{\text{tgt}} &= v + h((\partial_x u_\theta)v + \partial_r u_\theta - \partial_h u_\theta). \end{aligned} \quad (4.36)$$

The derivative enters as a fixed regression target, so the parameter update does not differentiate through the JVP [Gen+25a, Eqs. (7)–(11)].

Three properties of (4.36) are used later in this thesis.

- **It contains instantaneous-velocity matching as the case $r = \tau$.** The second term of u_{tgt} carries the factor $(\tau - r)$, so at $r = \tau$ the target collapses to v and (4.36) becomes (?). Analytic average-velocity matching is thus a strict generalisation of instantaneous-velocity matching, and the fraction of training samples drawn with $r \neq \tau$ is a hyperparameter of the objective: at zero it is instantaneous-velocity matching [Gen+25a, Tab. 1a]. The same collapse applies, for a different reason, to the consistency-interpolated objective through its explicit zero-interval branch (Subsection 4.4.4).
- **The interval is a network input, so the budget is chosen at inference.** As in Subsection 4.4.2, K appears only in the sampler. Unlike Subsection 4.4.2, small K is supported by learning the average displacement over an interval. The exact average would remove integration error over that interval; the network introduces approximation error.

- **The objective differentiates through the network, which constrains the architecture.** Everything inside the closure that (4.35) differentiates must support forward-mode differentiation. Image features enter the differentiated function as fixed conditioning (Subsection 4.4.6).

The squared residual e is adaptively reweighted as $\varrho(e) = e / \text{sg}((e + c)^p)$, following [Gen+25a, Sec. 4.3]. For $p = 1$, its displayed value is $e / (e + c)$ and approaches a ceiling at large residuals. The time pair is drawn from a logit-normal distribution [Ess+24], with the larger draw assigned to τ and the smaller to r .

Implementation: the start-anchored identity

The network inputs (x_r, r, h) use the same start-anchored convention as (4.32). Writing the identity in these coordinates gives

$$u = v + h((\partial_x u)v + \partial_r u - \partial_h u). \quad (4.37)$$

The two-head implementation combines an average-velocity target with an instantaneous-velocity loss. The target and loss are

$$u_{\text{tgt}} = \text{sg}[v + h \text{JVP}(u_\theta; (x_r, r, h), (v, +1, -1))], \quad v = x_1 - x_0, \quad (4.38)$$

$$\mathcal{L}(\theta) = \mathbb{E} \left[\varrho(\|u_\theta(x_r, r, h) - u_{\text{tgt}}\|_2^2) + \varrho(\|v_\theta(x_r, r, h) - \text{sg}(v)\|_2^2) \right], \quad (4.39)$$

where $\varrho(e) = e / \text{sg}((e + c)^p)$ is the adaptive reweighting of Subsection 4.4.3 and the squared errors are summed over the plan before reweighting. Four properties of (4.38)–(4.39) are choices of this implementation:

- **The x -tangent is the analytic velocity $x_1 - x_0$, not a predicted one.** This is the conditional velocity used to construct the training path.
- **(τ, r) are two independent draws** on the transport axis, the larger assigned to τ and the smaller to r .
- **A fraction of samples is drawn with $r = \tau$,** i.e. $h = 0$, at which (4.38) reduces to $u_{\text{tgt}} = v$ and the objective becomes (4.30). These anchor the field.
- **The instantaneous head has a separate regression loss.** Sampling uses the average-velocity head; the auxiliary instantaneous head is not integrated.

Sampling takes K interval jumps of size $h = 1/K$, passing both the position and the interval:

$$x_{\tau_{k+1}} = x_{\tau_k} + \frac{1}{K} u_\theta(x_{\tau_k}, \tau_k, h = \frac{1}{K}, c), \quad \tau_k = \frac{k}{K}, \quad (4.40)$$

which at $K = 1$ has the form of (4.33). Both the current transport time and interval are passed at every step. This is the sampler used without projection and with iterate projection; the field queried by endpoint projection is specified in Subsection 4.5.3.

4.4.4. Consistency-Interpolated Average-Velocity Matching

The third objective [Zha+25b] also learns average velocity, using a consistency step ratio α to interpolate between the conditional instantaneous velocity and a stop-gradient prediction of the network. Its comparison with the other objectives is reported in Subsubsection 6.1.2.1 and Section 6.2.

For $h > 0$ and $0 < \alpha \leq 1$, set $\delta = \alpha h$ and $\mathbf{v} = \mathbf{x}_1 - \mathbf{x}_0$. The start-anchored target is

$$\mathbf{u}_{\text{tgt}} = \alpha \mathbf{v} + (1 - \alpha) \text{sg}(\mathbf{u}_\theta(\mathbf{x}_r + \delta \mathbf{v}, r + \delta, h - \delta)). \quad (4.41)$$

The implementation explicitly uses $\mathbf{u}_{\text{tgt}} = \mathbf{v}$ for $h = 0$ samples. At $\alpha = 0$ it selects the analytic JVP target of (4.38). With $e_u = \|\mathbf{u}_\theta - \mathbf{u}_{\text{tgt}}\|_2^2$ and $e_v = \|\mathbf{v}_\theta - \text{sg}(\mathbf{v})\|_2^2$, summed over the unmasked plan entries, the loss is

$$\mathcal{L}(\theta) = \mathbb{E}[w_{\text{br}} \varrho(e_u) + \varrho(e_v)], \quad w_{\text{br}} = \begin{cases} \alpha & \alpha > 0 \text{ and } h > 0, \\ 1 & \text{otherwise.} \end{cases} \quad (4.42)$$

The branch weight affects the average-velocity loss; the auxiliary instantaneous-head loss has unit weight; here ϱ uses $p = 1$. During training the ratio follows a sigmoid curriculum from one to α_{end} [Zha+25b, Alg. 2].

At $\alpha = 1$, the target is the instantaneous velocity. For a small positive ratio, a first-order expansion gives $\mathbf{u}_{\text{tgt}} = \mathbf{u}_\theta + \alpha(\mathbf{v} - \mathbf{u}_\theta + h D\mathbf{u}_\theta) + O(\alpha^2)$, where D is the derivative in direction $(\mathbf{v}, +1, -1)$. The target itself tends to the current network output; its residual divided by α tends to the analytic residual. The gradient correspondence in [Zha+25b, Thm. 1] concerns the appropriately scaled objective. The exact $\alpha = 0$ implementation is a separate analytic branch, not direct substitution into (4.41). Both constructions depend on the learned network: one uses its shifted output and the other its directional derivative.

4.4.5. A Common Interface

The inherited system is organised around two calls: one that turns an iterate into a proposed mean, and one that loops over K steps and applies the constraint mechanism. Every generative model uses this interface for unguided and iterate-projected sampling, with the task conditioning, normalisation and candidate selection supplied by the

common planning loop. Written as the next iterate μ^k that the projection of Section 4.5 acts on, the substitution is

$$\mu^k = \begin{cases} \frac{\beta_k \sqrt{\bar{\alpha}_{k-1}}}{1 - \bar{\alpha}_k} \hat{x}^0 + \frac{(1 - \bar{\alpha}_{k-1}) \sqrt{\bar{\alpha}_k}}{1 - \bar{\alpha}_k} x^k & \text{diffusion (baseline), (4.24)} \\ x_{\tau_k} + \frac{1}{K} v_\theta(x_{\tau_k}, \tau_k, c) & \text{instantaneous velocity} \\ x_{\tau_k} + \frac{1}{K} u_\theta(x_{\tau_k}, \tau_k, h = \frac{1}{K}, c) & \text{average velocity,} \end{cases} \quad (4.43)$$

with $\tau_k = k/K$ and the reverse index of the first line replaced by the forward index of the other two. These lines specify the proposal means; the diffusion sampler also draws reverse noise. Endpoint sampling of the two-time models uses the zero-interval query described in Subsection 4.5.3.

4.4.6. Backbone and Conditioning

The generative models use matched temporal U-Net backbones, including the diffusion baseline. For the vision-conditioned task, a visual encoder produces a fixed-width latent, and the backbone never sees an image.

The trajectory backbone

The backbone is a one-dimensional temporal U-Net in the lineage of [Jan+22]: the plan is treated as a length- H signal with d channels, and convolution runs along *time*, not along the state dimension. Channel widths follow $(c_0, \dots, c_L) = (d, w, 2w, 4w, 8w)$ for a base width w , with three stride-two down/up stages and skip concatenations. The horizon is zero-padded up to the next multiple of eight before the network and cropped back afterwards; $H = 8$ needs no padding.

Every level is a pair of residual blocks. Writing $h \in \mathbb{R}^{c_{\text{in}} \times H}$ for a block’s input, $\mathcal{B}$ for a (Conv1d $\rightarrow$ GroupNorm $\rightarrow$ Mish) unit and z for the block’s conditioning vector,

$$\text{Block}(h, z) = \mathcal{B}_2\left(\mathcal{B}_1(h) + (W_{\text{emb}} \varphi(z)) \mathbf{1}_H^T\right) + Ph, \quad (4.44)$$

with φ a pointwise nonlinearity, P a 1×1 channel-matching convolution, and $\mathbf{1}_H^T$ broadcasting the projected embedding along the horizon. The conditioning enters as a per-channel additive bias that is constant in time.

The conditioning vector

For the state-based environments z is the transport-time embedding alone, $z = \phi_\tau(\tau)$, a sinusoidal positional encoding followed by a two-layer MLP; conditioning on the current observation is done by inpainting (Section 4.1).

For the vision-conditioned environment, the visual encoder is that of D3IL [Jia+24], which adopts the encoder of Diffusion Policy [Chi+23, Sec. 3.2]. Each of the two RGB views, at 96×96 pixels, has its own ResNet-18 [He+16], trained end-to-end with the generative model. Its global average pooling is replaced by spatial-softmax pooling [Man+21] over 32 keypoints, followed by a linear layer to 64 features, and its batch normalisation by group normalisation [WH18]. The two feature vectors are concatenated to a per-frame descriptor, and the descriptor is averaged over the T_{win} -frame observation window, giving one latent per plan,

$$\mathbf{e} = \frac{1}{T_{\text{win}}} \sum_{i=1}^{T_{\text{win}}} \left[E_{\text{view1}}(\mathbf{I}_i^{(1)}) ; E_{\text{view2}}(\mathbf{I}_i^{(2)}) \right] \in \mathbb{R}^{128}. \quad (4.45)$$

$\mathbf{e}$ is the entire interface between perception and generation: the input channels of the U-Net are the d channels of the plan, and the perception stack is the same for every generative model.

The latent conditions the backbone by *feature-wise conditional biasing* [Per+17, Sec. 3]. It is projected by its own MLP and concatenated with the time embedding,

$$\mathbf{z} = [\phi_{\tau}(\tau) ; \phi_{\text{vis}}(\mathbf{e})] \in \mathbb{R}^{2w}, \quad (4.46)$$

so that it reaches every residual block through the additive bias of (4.44).

Two-time models

The two average-velocity objectives need $u_{\theta}(\cdot, \tau, h)$, so the backbone takes a second time input. It is embedded by its own MLP and added to the transport-time embedding, before the visual latent is concatenated:

$$\mathbf{z} = [\phi_{\tau}(\tau) + \phi_h(h) ; \phi_{\text{vis}}(\mathbf{e})]. \quad (4.47)$$

The interval embedding and the auxiliary instantaneous-velocity head extend the matched temporal U-Net backbone. Reported backbone capacity excludes the image encoders; the model sizes and training settings are specified in Section 5.6.

The analytic target (4.38) differentiates the network with a forward-mode JVP in $(\mathbf{x}_r, r, h)$. The latent $\mathbf{e}$ depends on none of these inputs, so the encoder is evaluated once, outside the differentiated function, and $\mathbf{e}$ enters it as a constant with zero tangent. The two ResNet towers are therefore not differentiated by the JVP, and they are still trained end-to-end through the loss.

4.5. Constraint Projection

Both projection methods solve the same program on the same feasible set with the same solver. Iterate projection applies it to the sampler's current iterate; endpoint projection applies it to the plan the sampler is predicted to reach. Plans generated with neither are unguided.

4.5.1. The Projection Problem

The feasible set and the projection

Let $\mathcal{Z} \subset \mathbb{R}^n$ collect the plans that satisfy the test-time state and action constraints, and let $\mathcal{Z}_f \subseteq \mathcal{Z}$ additionally impose the dynamics model. Following [RRS25, Eq. (9)], the *model-based* projection of a plan is the program

$$\Pi_{\mathcal{Z}_f}(\mathbf{x}) = \arg \min_{\tilde{\mathbf{x}} \in \mathcal{Z}} \|\mathbf{x} - \tilde{\mathbf{x}}\|_2^2 \quad (4.48)$$

$$\text{s.t. } \tilde{\mathbf{s}}_{t'+1|t} = f(\tilde{\mathbf{s}}_{t'|t}, \tilde{\mathbf{a}}_{t'|t}), \quad \forall t' \in \{t, \dots, t+H-2\}, \quad (4.49)$$

with projection cost $c_{\mathcal{Z}_f}(\mathbf{x}) = \|\mathbf{x} - \Pi_{\mathcal{Z}_f}(\mathbf{x})\|_2^2$. The states and actions in the constraint belong to the decision variable $\tilde{\mathbf{x}}$. The projection is *goal-independent*: its objective is proximity to the proposed plan. It may improve task success by removing a violation, but success is not its objective. Both projection methods therefore change the learned plan as little as possible. The projection is also a *nonlinear* program whenever the geometric constraints are non-convex, as circular keep-out regions make them; the solver used throughout this thesis is sequential least-squares quadratic programming (SLSQP) [Kra88; VGO+20], inherited from [RRS25].

The program

With the plan flattened to $\mathbf{x} \in \mathbb{R}^n$, (4.48) is instantiated as a quadratic objective with linear equality rows for the dynamics, linear inequality rows for the bounds, and one quadratic inequality per horizon step per obstacle:

$$\Pi_{\mathcal{Z}_f}(\mathbf{x}) = \arg \min_{\mathbf{z} \in \mathbb{R}^n} \frac{1}{2} \mathbf{z}^T \mathbf{Q} \mathbf{z} - \mathbf{x}^T \mathbf{Q} \mathbf{z} = \frac{1}{2} \|\mathbf{z} - \mathbf{x}\|_{\mathbf{Q}}^2 + \text{const} \quad (4.50)$$

$$\text{s.t. } \mathbf{A} \mathbf{z} = \mathbf{b} \quad \text{(dynamics rows, (4.49))} \quad (4.51)$$

$$\mathbf{C} \mathbf{z} \leq \mathbf{d} \quad \text{(state / action bounds, halfspaces)} \quad (4.52)$$

$$\mathbf{z}_i^T \mathbf{P}_j \mathbf{z}_i + \ell_j^T \mathbf{z}_i \leq \nu_j \quad \text{(obstacle } j \text{ at horizon step } i), \quad (4.53)$$

with Q diagonal — the identity unless a subset of dimensions is weighted — and z_i the i -th horizon block of z ; (P_j, ℓ_j, v_j) are the quadratic, linear and constant coefficients of obstacle j . The rows (4.53) are what makes $\mathcal{Z}_f$ non-convex, so (4.50)–(4.53) is a nonlinear program (NLP) and is solved by SLSQP [Kra88; VGO+20], warm started at x itself, with box bounds inherited from the normalisation. The first dynamics block is overwritten with the measured current state, so the projection cannot move the point the plan is anchored at. The returned projection cost is $c_{\mathcal{Z}_f}(x) = \frac{1}{2} \|\Pi_{\mathcal{Z}_f}(x) - x\|_Q^2$, which is the quantity the cumulative-cost selection rule of Subsection 4.5.5 sums. Each candidate plan is a separate program. Exact projection yields a feasible point when the set is nonempty and the problem is solved. The implementation uses a numerical local solve and can return its last iterate on non-convergence, so feasibility of that output is not guaranteed; constraint satisfaction is evaluated on the executed trajectories.

The dynamics rows, written out

The Euler model (4.5) enters (4.51) as one equality row per integrated channel per consecutive pair of horizon blocks, $H - 1$ of them per channel. Writing $z_{i,d}$ for dimension d of horizon block i , a channel pair $(d_{\text{pos}}, d_{\text{act}})$ contributes

$$z_{i,d_{\text{pos}}} + t_s z_{i,d_{\text{act}}} - z_{i+1,d_{\text{pos}}} = 0, \quad i = 0, \dots, H - 2, \quad (4.54)$$

plus one row pinning $z_{0,d_{\text{pos}}}$ to the measured current state — the only row whose right-hand side changes at every control step. Two details of (4.54) are easy to get wrong and are therefore stated. First, the projection operates in *normalised* coordinates, so each row is rescaled by the per-dimension data range: with $\Delta_d = z_{\text{max},d} - z_{\text{min},d}$ and $\Sigma_d = z_{\text{max},d} + z_{\text{min},d}$, the implemented row is

$$\Delta_{d_{\text{pos}}} z_{i,d_{\text{pos}}} + t_s \Delta_{d_{\text{act}}} z_{i,d_{\text{act}}} - \Delta_{d_{\text{pos}}} z_{i+1,d_{\text{pos}}} = -t_s \Sigma_{d_{\text{act}}}, \quad (4.55)$$

which is (4.54) conjugated by the normaliser; forgetting the affine offset Σ silently biases every dynamics row. Second, the row count is the number of *channel pairs*, and that number is a modelling decision, not a constant: it is four for the two-dimensional manipulator and six for both three-dimensional environments, because each of them anchors a commanded and a measured position in three axes (Table 4.3).

The activation schedule

A threshold $\eta \in [0, 1]$ selects sampling steps near the data end. The inherited diffusion code uses the zero-based reverse index $i = K - 1, \dots, 0$ and activates projection when

$i \leq \eta K$. In the one-based equations of Subsection 4.4.1, $i = k - 1$. Its active count is $\min\{K, \lfloor \eta K \rfloor + 1\}$. The ascending flow samplers, with $k = 0, \dots, K - 1$, use

$$k \geq \lfloor (1 - \eta)K \rfloor \quad \text{or} \quad k = K - 1, \quad (4.56)$$

where the last clause enforces terminal projection. Their active count is

$$n_{\text{act}}(K, \eta) = \max\{K - \lfloor (1 - \eta)K \rfloor, 1\}. \quad (4.57)$$

Thus a shared threshold value need not give the same number of projection solves across the two samplers. At $\eta = 0$, both apply only the terminal projection. Experiment-specific thresholds are reported in Section 5.6.

4.5.2. Per-Step Projection (DPCC)

Per-step projection is the constraint mechanism of DPCC [RRS25].

The constrained denoising step

The inherited algorithm is (4.21) with a projection inserted between its steps. Its justification is a guidance argument: introducing a binary feasibility variable O with likelihood $p(O=1 \mid x, k) \propto \exp(-\frac{1}{2\sigma_k^2}d(x, \mathcal{Z}_f)^2)$, where d is the Euclidean distance to the feasible set, a first-order expansion of $\log p(O \mid x, k)$ about the learned mean — the same expansion that underlies classifier guidance [DN21] — gives a guidance term that, for closed convex $\mathcal{Z}_f$, is exactly the displacement to the projection. The conditioned reverse step is therefore

$$p_\theta(x^{k-1} \mid x^k, k, O=1) \approx \mathcal{N}(\Pi_{\mathcal{Z}_f}(\mu_\theta^k), \sigma_k^2 \mathbf{Id}_n) \quad (4.58)$$

[RRS25, Theorem 1]. Because (4.58) projects the *mean* and then adds noise, the drawn sample can still leave $\mathcal{Z}_f$; the projection procedure therefore acts *after* the reverse noise is drawn. With $\tilde{\sigma}_k$ denoting its configured sampling standard deviation (zero at the terminal step), the update is

$$x^{k-1} = \Pi_{\mathcal{Z}_f}(\mu_\theta(x^k, k, c) + \tilde{\sigma}_k \epsilon^k), \quad \epsilon^k \sim \mathcal{N}(\mathbf{0}, \mathbf{Id}_n), \quad (4.59)$$

which gives constraint satisfaction under an exact feasible projection [RRS25, Eq. 16]. (4.59) is the single equation this thesis replaces: Section 4.4 substitutes the generative model that produces μ_θ , and Subsection 4.5.3 substitutes the operator applied to it.

What is projected

This method projects the *current iterate*. Under (4.56) with a large η that iterate is still far from the data end of the transport, so (4.50)–(4.53) is posed on an object that is not yet a plan: the geometric rows (4.53) are being asked about positions that the sampler has not finished producing. Endpoint projection instead constrains a predicted final plan. For average-velocity models, the implemented samplers also differ in their field query (Subsection 4.5.3).

Constraint tightening

The dynamics f used inside (4.49) is an approximation, with mismatch bounded by $\|w_t\|_2 \leq \gamma$. Feasibility of the *predicted* plan therefore does not imply feasibility of the executed one. The inherited remedy tightens the predicted-state constraints by the mismatch ball,

$$\tilde{\mathcal{S}}_{t+1} = \mathcal{S}_{t+1} \ominus \mathcal{B}_\gamma, \quad (4.60)$$

where $\mathcal{B}_\gamma$ is the Euclidean ball of radius γ and $\ominus$ denotes set erosion (Pontryagin difference): $\mathcal{S} \ominus \mathcal{B}_\gamma = \{z : z + \mathcal{B}_\gamma \subseteq \mathcal{S}\}$. Under the mismatch-bound and feasibility assumptions of [RRS25, Theorem 2], tightening provides the margin needed for the next executed state to satisfy the original set. Here γ is estimated from unconstrained rollouts. Tightening is relative to the original constraints, not relative to the diffusion baseline; the configured margins and their use in the comparisons are given in Section 5.6.

4.5.3. Endpoint Projection

Endpoint projection [LAA25] is re-derived here on this work’s own constraint set.

Sampling as optimal control

A control u_τ is added to the learned velocity field, and the constraints are imposed on the final sample only [LAA25, Problem 1]:

$$\min_{u_{[0,1]}} C(x_1) + \lambda_{\text{oc}} \int_0^1 \frac{1}{2} \|u_\tau\|_2^2 d\tau \quad \text{s.t.} \quad \dot{x}_\tau = v_\theta(x_\tau, \tau, c) + u_\tau, \quad x_1 \in \mathcal{Z}_f, \quad (4.61)$$

starting from the drawn noise x_0 . C is an optional terminal cost, and the effort term keeps the controlled sampler close to the learned one.

Predicting the final sample

For an affine interpolation path the expected final sample given an intermediate state follows in closed form from the state and the marginal velocity [LAA25, Lemma 1]. For the linear path (4.29) the expectations of the final sample and of the initial noise are

$$\mathcal{M}_\tau(\mathbf{x}) = \mathbb{E}[\mathbf{x}_1 \mid \mathbf{x}_\tau = \mathbf{x}] = \mathbf{x} + (1 - \tau) v(\mathbf{x}, \tau), \quad \mathcal{W}_\tau(\mathbf{x}) = \mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}_\tau = \mathbf{x}] = \mathbf{x} - \tau v(\mathbf{x}, \tau), \quad (4.62)$$

with $\mathbf{x} = \tau \mathcal{M}_\tau(\mathbf{x}) + (1 - \tau) \mathcal{W}_\tau(\mathbf{x})$. At $\tau = 1$ the prediction is the state itself.

One problem per sampling step

The method discretises (4.61) on the Euler grid of (4.31), splits it into one problem per step in which the final sample is replaced by its prediction (4.62), and changes the decision variable from the next state to the predicted final sample, so that the constraint set is not distorted by the network [LAA25, Problems 2–5]. Inverting the prediction with one fixed-point step gives the algorithm it runs [LAA25, Algorithm 1]. With $\Delta\tau = 1/K$, a guided step k reads

$$\bar{\mathbf{x}}_{k+1} = \mathbf{x}_{\tau_k} + \Delta\tau v_\theta(\mathbf{x}_{\tau_k}, \tau_k, \mathbf{c}), \quad (4.63)$$

$$\bar{\mathbf{x}}_1 = \bar{\mathbf{x}}_{k+1} + (1 - \tau_{k+1}) v_\theta(\bar{\mathbf{x}}_{k+1}, \tau_{k+1}, \mathbf{c}), \quad (4.64)$$

$$\hat{\mathbf{x}}_1^* = \arg \min_{\hat{\mathbf{x}}_1 \in \mathcal{Z}_f} C(\hat{\mathbf{x}}_1) + \frac{\lambda_{\text{oc}} \tau_{k+1}^2}{2 \Delta\tau} \|\hat{\mathbf{x}}_1 - \bar{\mathbf{x}}_1\|_2^2, \quad (4.65)$$

$$\mathbf{x}_{\tau_{k+1}} = \bar{\mathbf{x}}_{k+1} + \tau_{k+1} (\hat{\mathbf{x}}_1^* - \bar{\mathbf{x}}_1). \quad (4.66)$$

(4.63) is an unguided Euler step, (4.64) predicts its final sample, (4.65) corrects the prediction, and (4.66) — $\tau_{k+1} \hat{\mathbf{x}}_1^* + (1 - \tau_{k+1}) \mathcal{W}_{\tau_{k+1}}(\bar{\mathbf{x}}_{k+1})$ written out — pulls the correction back into the sampler, damped by τ_{k+1} . At the final step $\tau_K = 1$, so the output equals $\hat{\mathbf{x}}_1^* \in \mathcal{Z}_f$ and satisfies the constraints when the terminal projection is feasible [LAA25, Prop. 1]. The optimisation runs only in the later sampling steps, where the prediction (4.62) is accurate [LAA25]. (4.63)–(4.66) are the steps of [LAA25, Alg. 1], written in the notation of this thesis. Figure 4.1 shows the sequence they produce on the obstacle-avoidance task used here, as the source draws it. The state the solver holds is disordered until the last steps, while the prediction it constrains is a trajectory from the first step on; the constraint is imposed where the plan already exists.

The configuration used here

The program (4.65) is built from the constraint list of iterate projection (Subsection 4.5.1), and the terminal cost is zero, $C \equiv 0$. The objective is then a quadratic proximity term

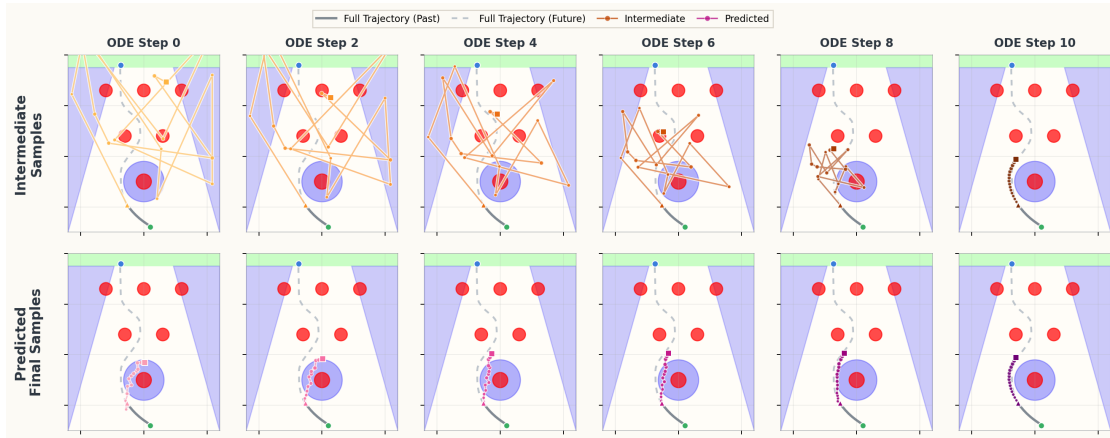


Figure 4.1.: Endpoint projection over the sampling steps, on the obstacle-avoidance task. The upper row draws the intermediate sample x_τ the solver actually holds; the lower row draws the predicted final sample $\mathcal{M}_\tau(x_\tau)$ of (4.62), which is the point the program (4.65) constrains. Red disks are the obstacles and the shaded wedges the excluded halfspaces; the green band is the goal region, the solid grey curve the path already executed and the dashed grey curve its continuation. Columns are every second step of a ten-step solver, and the policy replans, so the panels are one planning instance of an episode. Figure reproduced, unaltered, from [LAA25, p. 22, Fig. 11] under the Creative Commons Attribution 4.0 licence; the task, the constraint set and the run are the source's.

whose minimiser does not depend on its weight, so

$$\hat{x}_1^* = \Pi_{\mathcal{Z}_f}(\tilde{x}_1), \quad (4.67)$$

the projection of Subsection 4.5.1 applied to the predicted final sample. For instantaneous-velocity matching, the field in (4.63) and (4.64) is v_θ . For the two average-velocity models, both steps query $u_\theta(x, \tau, 0)$, the learned zero-interval proxy for instantaneous velocity. This differs from the finite-interval query of (4.40); the comparison therefore concerns the implemented sampling and projection procedures on a fixed checkpoint.

Each active nonterminal step adds the lookahead evaluation in (4.64) to the Euler evaluation. The terminal step skips this lookahead because its coefficient is zero. Hence, for each candidate under this Euler sampler,

$$\text{NFE} = K + n_{\text{guide}}, \quad (4.68)$$

where n_{guide} is defined below. The task-specific budgets and activation thresholds are specified in Section 5.6.

4.5.4. Steps at Which Projection Guides Sampling

The activation convention (4.56) of endpoint projection can leave the terminal step as the only active one. At the terminal time the lookahead vanishes, the pull-back replaces the sample by the projection of its prediction, and no later step exists for the network to react to. What runs is then projection applied to a finished sample.

The guiding step count

Of the n_{act} steps counted by (4.57), one is the terminal step, which does not guide the sampler. The *guiding step count* is

$$n_{\text{guide}}(K, \eta) = n_{\text{act}}(K, \eta) - 1 = \max\{K - \lfloor (1 - \eta)K \rfloor, 1\} - 1. \quad (4.69)$$

It depends on the step budget and the threshold together, so two configurations with the same step budget and different thresholds can differ in it. The cases are:

n_{guide}	what runs	what a result supports
0	projection of the final sample only	terminal-projection behaviour
1	one guided step before the final one	effect of a single guiding step
≥ 2	several nonterminal guiding steps	repeated guidance during sampling

The *first lookahead*, the distance $1 - \tau_{k_0+1}$ remaining at the first guiding step with $k_0 = K - n_{\text{act}}$, is reported alongside it. It measures how far the prediction of the final sample extrapolates at that step.

A configuration with no guiding step evaluates terminal projection. One guiding step can already change the state queried by the next field evaluation; it is a guided configuration, with evidence limited to that single correction. The results identify the step budget and threshold so that these cases remain distinguishable (Subsubsection 6.1.2.6).

4.5.5. Candidate Selection

Candidate plans

Rather than one plan, B plans are drawn per timestep and one is executed. The inherited selection rules are *temporal consistency* — take the candidate closest to the previous timestep’s executed plan on their overlapping segment — and *cumulative projection cost* — take the candidate that (4.48) altered least, summed over the steps at which it ran [RRS25, Sec. 5.4]. The third rule in this work’s implementation is random selection, which is the convention the two rules were proposed against.

The three selection rules

Let $\mathbf{o}_{t:t+H-2|t}^j$ be the unnormalised observation blocks in candidate j . The previous selected plan ends at $t + H - 2$, so temporal consistency compares only this shared segment, excluding the action channels. With one-based candidate indices, the rules are

$$\begin{aligned} i_{\text{rand}}(t) &= 1, \\ i_{\text{tc}}(t) &= \arg \min_j \left\| \mathbf{o}_{t:t+H-2|t}^j - \mathbf{o}_{t:t+H-2|t-1}^{i(t-1)} \right\|_2, \\ i_{\text{cc}}(t) &= \arg \min_j \sum_k c_{\mathcal{Z}_f}(\tilde{\mathbf{x}}^{k,j}). \end{aligned} \tag{4.70}$$

The random rule takes the first unranked stochastic candidate. At the first control step, temporal consistency also falls back to that candidate because no previous plan exists. The quantity $\tilde{\mathbf{x}}$ is the pre-projection proposal used for the projection-cost score. With $B = 1$, all three rules select the sole candidate.

4.6. Environments

Obstacle avoidance is the benchmark of DPCC, taken over with its setup unchanged, and is where the new generative models and projection methods are first established

against the baseline. Vision-conditioned alignment and the quadrotor scenes were built for this thesis and test them further, in a new observation modality and a new embodiment. Where several environments appear together, they are named D3IL-avoiding, D3IL-aligning, and, per scene, UAV-corridor, UAV-pillars and UAV-s-curve.

The three environments share the plan layout of Table 4.3 and differ in their observations, their dynamics rows and their constraint sets. Each constraint below enters the program (4.50)–(4.53) as one family of rows, acts on the planned position $\mathbf{p}_i$ of every horizon step i , and is shrunk by the tightening (4.60) in the tightened geometries. Scene dimensions are configuration values and are listed with the experimental setup in Section 5.2.

Two constraint forms recur. A halfspace, given by a boundary line and the side to be excluded, is the linear row

$$\mathbf{n}^\top \mathbf{p}_i \leq d_{\text{obs}}, \quad (4.71)$$

with unit normal $\mathbf{n}$ and offset d_{obs} , distinct from the plan width d . A circular keep-out region with centre $\mathbf{o}$ and radius ρ is the quadratic row

$$\|\mathbf{p}_i - \mathbf{o}\|_2^2 \geq \rho^2, \quad (4.72)$$

which makes the feasible set non-convex.

A third family bounds the action rather than the position. The planned increment is held inside a box whose per-channel limits are read off the demonstrations at evaluation time, so the bound follows the dataset rather than the scene and is the same family in all three environments. It is referred to below as the action bound.

4.6.1. Obstacle Avoidance

The obstacle-avoidance task of D3IL [Jia+24] is the benchmark of DPCC [RRS25, Sec. 6.1]. It is simulated in MuJoCo [TET12]: a seven-joint Franka Emika Panda carries a cylindrical rod as its end effector and moves in a plane [Jia+24, Sec. 3.3]. The state stacks the commanded and the measured planar end-effector position, $\mathbf{s} = (\mathbf{p}_{\text{des}}, \mathbf{p}) \in \mathbb{R}^4$, the action is a planar increment $\Delta \mathbf{p} \in \mathbb{R}^2$, and the four dynamics rows are (4.5). The test-time constraints are halfspaces (4.71) and circular regions (4.72) on the measured position, none of which the demonstrations respect, together with the action bound [RRS25, Sec. 6.1]. The baseline runs in the configuration of DPCC — the same plan horizon, number of diffusion steps and number of candidate plans — on the same constraint geometries.

4.6.2. Vision-Conditioned Alignment

The alignment task of D3IL [Jia+24] asks the robot to push a box into a target pose. It runs in MuJoCo; its task files, demonstrations, camera placement and vision pipeline are D3IL’s; the constrained-control environment around them was built for this thesis — the plan and its dynamics rows, the constraint set, and the coupling of the visual encoder to the generative network.

It uses the controller stack of the manipulator; what changes is the plan and the observation. The action is a three-dimensional increment $\Delta \mathbf{p} \in \mathbb{R}^3$, the state half of the observation is $(\mathbf{p}_{\text{des}}, \mathbf{p}) \in \mathbb{R}^6$ — commanded and measured end-effector position — and the plan width is therefore $d = 9$ per horizon step, laid out as $(\mathbf{a}, \mathbf{p}_{\text{des}}, \mathbf{p})$. The pose of the box and the pose of its target are drawn afresh for every episode and appear nowhere in that vector, so the two camera views are the only place the policy can read them. The six dynamics rows of (4.54) are the pairs

$$(d_{\text{pos}}, d_{\text{act}}) \in \{(3, 0), (4, 1), (5, 2)\} \cup \{(6, 0), (7, 1), (8, 2)\}, \quad (4.73)$$

i.e. the commanded triple and the measured triple both integrate the same action, which is (4.5) written in three dimensions.

Perception sits above the plan: two RGB views are encoded to a 128-dimensional latent that conditions the backbone through the feature-wise bias of (4.46), and the backbone never sees an image (Subsection 4.4.6). Below the setpoint, the control chain is that of the state-based benchmark.

Its constraint set is built from the same two forms. A halfspace (4.71) runs diagonally across the far corner of the table and excludes it, and a circular keep-out region (4.72) sits in the table plane between the box and its target. A workspace box $\mathbf{p}_{\min} \leq \mathbf{p}_i \leq \mathbf{p}_{\max}$ bounds the planned end-effector position to the reachable part of the table, and the action carries the action bound.

4.6.3. Quadrotor Scenes

The vehicle is the Skydio X2 from MuJoCo Menagerie [ZTM22], copied verbatim and patched only to spawn level and to drop an unused sensor block and keyframe, and simulated in MuJoCo [TET12]. Walls and pillars are passive rigid bodies and do not change the dynamics of the vehicle. Three scenes are flown, and they are two constructions rather than three instances of one. UAV-corridor and UAV-s-curve plan in three dimensions with a planner trained on flights in those scenes. UAV-pillars flies the planner of the obstacle-avoidance benchmark, unchanged, in that benchmark’s obstacle field scaled up around the vehicle.

UAV-corridor and UAV-s-curve: the spatial plan

On these two scenes the plan stacks action, commanded position, measured position and measured velocity (Table 4.3, $d = 12$), a plan is queried every third physics step (4.12), and the six dynamics rows anchor the commanded and the measured position as in (4.73). The corridor is two parallel walls; the s-curve is two offset corridor segments joined by a diagonal passage. The constraint set of each combines four families:

- a workspace box $\mathbf{p}_{\min} \leq \mathbf{p}_i \leq \mathbf{p}_{\max}$ bounding extent and altitude;
- wall halfspaces (4.71), each existing only over the span $[x_a^w, x_b^w]$ of x that its wall occupies. At every replan the projection receives the walls whose span contains the vehicle's current x — the measured position, or the commanded one where the geometry is bound to the setpoint — and applies them to every step of the horizon,

$$\mathcal{W}(x) = \{ w \mid x \in [x_a^w, x_b^w] \}, \quad \mathbf{n}_w^T \mathbf{p}_i \leq d_w \quad \forall w \in \mathcal{W}(x_t), \forall i, \quad (4.74)$$

so a wall the horizon will only reach later enters at the replan that reaches it;

- keep-out disks (4.72) at the free ends of the corridor walls and at the inner corners of the s-curve;
- the action bound, on the s-curve; the corridor leaves the action unconstrained and binds its geometry to the commanded position as well as to the measured one (Section 5.6).

The test-time constraints of the corridor are two further halfspaces, evaluated one at a time as the geometries of obstacle avoidance are, and both leave the horizontal plane. The first is a boundary that crosses the corridor obliquely and is leaned out of the vertical, so that its normal $\mathbf{n}$ has a component along z and the room it leaves changes with altitude. The second is a roof: two halfspaces in the x - z plane that rise to a ridge and fall again, each over its own span of x , in the form of (4.74). The demonstrations were flown in the level corridor before either boundary existed. The dimensions of every scene and boundary are listed in Section 5.2.

Every wall, plane, roof and disk of these two scenes is moved outward by the vehicle's rotor reach r_b before the projection sees it — the distance from the centre to the rim of a rotor disk along a body axis, with the vehicle at zero yaw and every approach in these scenes lateral — so that a plan for the vehicle's centre is a plan for the vehicle under that assumption. The offset is carried in every geometry, and it is not the tightening (4.60): the two are separate quantities, and only the tightening is what the tightened variant of a geometry changes.

UAV-pillars: the obstacle-avoidance planner flown

This scene changes the plant and nothing else. The obstacle field of Subsection 4.6.1 is scaled by a factor s , turned by a quarter turn into a horizontal arena, and its six posts are extruded into pillars; the three test-time constraint geometries of that benchmark are mapped into the arena by the same similarity map. The factor is chosen so that the rod’s radius on the table maps onto the vehicle’s radial rotor reach, and that is how the vehicle’s extent enters: no offset is added to the mapped surfaces. Writing (x_a, y_a) for a point on the table and (X, Y, Z) for a point in the arena,

$$X = s(y_a - y_0), \quad Y = -s(x_a - x_0), \quad Z = z_{\text{fly}}, \quad (4.75)$$

with (x_0, y_0) the centre of the field and z_{fly} a fixed flying altitude.

The planner, its plan, its observation, its projection and its constraint rows are those of D3IL-avoiding: the $d = 6$ layout of Table 4.3, the planar increment, the four dynamics rows, and the halfspaces and keep-out disk of the geometry in the table’s own coordinates. The loop of Subsection 4.3.3 runs unchanged, and only the plant behind (4.7) differs. Each commanded position is mapped into the arena by (4.75) and handed to the cascaded geometric controller of Subsection 4.3.5 as a setpoint; the controller’s vertical loop holds z_{fly} , because the plan carries no altitude, and its reference slides towards the setpoint at a bounded speed for one control period, without velocity feed-forward. The vehicle’s measured position is then mapped back to the table by the inverse of (4.75) and becomes the measured half of the next observation. A collision with a pillar ends the episode as a failure, as a collision of the rod with a post does in D3IL-avoiding. The scene has no demonstrations of its own: its planner is trained on the human demonstrations of D3IL-avoiding, and what the scene measures is the difference between two plants executing the same plans.

Expert demonstrations of the corridor and the s-curve

The demonstrations of these two scenes are generated in two layers: a geometric reference trajectory, and its execution by the cascaded geometric tracking controller of Subsection 4.3.5 in the MuJoCo scene.

The reference of an episode is built from a chain of waypoints. The waypoints fix on which side of each obstacle the path passes; in scenes with several such classes, the classes are cycled across episodes. Consecutive waypoints are joined by straight segments, and every non-collinear interior waypoint is replaced by a circular arc of radius at most r_f tangent to both adjacent segments, with the radius reduced where the arc would otherwise use more than half of either segment. The result is a path $p_{\text{path}} : [0, L_{\text{path}}] \rightarrow \mathbb{R}^3$ parametrised by arc length, travelled with one cosine speed

profile over the episode duration T_{ep} ,

$$\mathbf{p}_{\text{des}}(t) = \mathbf{p}_{\text{path}}(s(t)), \quad s(t) = \frac{L_{\text{path}}}{2} \left(1 - \cos \frac{\pi t}{T_{\text{ep}}} \right), \quad 0 \leq t \leq T_{\text{ep}}, \quad (4.76)$$

so the reference is at rest only at its start and its end. Its velocity and acceleration,

$$\dot{\mathbf{p}}_{\text{des}} = \mathbf{p}'_{\text{path}}(s) \dot{s}, \quad \ddot{\mathbf{p}}_{\text{des}} = \mathbf{p}'_{\text{path}}(s) \ddot{s} + \mathbf{p}''_{\text{path}}(s) \dot{s}^2, \quad (4.77)$$

are the feed-forward terms of (4.13); $\mathbf{p}''_{\text{path}}$ vanishes on the segments and is the centripetal term on the arcs. The altitude and the duration are drawn at random per episode, together with a small lateral offset of the start in the centre lane of the corridor and a small lateral offset of the whole path in the s-curve.

The vehicle starts level and at rest, and the controller tracks (4.76)–(4.77) at the physics step Δt_{phys} . An episode is discarded if the vehicle is in contact with an obstacle for more than a scene-specific fraction of its steps, or if it falls below a minimum altitude. The accepted rollouts are recorded at the planning interval, every n_{dec} -th physics step of (4.12), as

$$\mathbf{o}_j = (\mathbf{p}_{\text{des}}(t_j), \mathbf{p}(t_j), \dot{\mathbf{p}}(t_j)), \quad \mathbf{a}_j = \mathbf{p}_{\text{des}}(t_{j+1}) - \mathbf{p}_{\text{des}}(t_j), \quad t_j = j n_{\text{dec}} \Delta t_{\text{phys}}, \quad (4.78)$$

the form of (4.7)–(4.8). The action is the increment of the reference, not an output of the controller: the generative model learns to plan the reference, and at deployment the same controller tracks it. The corridor’s two test-time boundaries did not exist when its demonstrations were flown, so, as on the obstacle-avoidance benchmark, the constraints the projection enforces are constraints the data does not satisfy.

5. Experimental Setup

The evaluation has two phases. First, the generative models and projection methods are established on the obstacle-avoidance benchmark of DPCC, with its setup taken over unchanged so that the diffusion baseline is met on its own benchmark. Second, they are tested further in two environments built for this thesis: a vision-conditioned alignment task and a quadrotor benchmark with three scenes. Throughout this chapter and the results, these environments are called *D3IL-avoiding*, *D3IL-aligning*, *UAV-corridor*, *UAV-pillars*, and *UAV-s-curve*; the descriptive subsection titles and these prefixed names refer to the same tasks. Section 4.6 describes how each environment is constructed; this chapter gives, for each of the three, the task and constraint values, the data, the metrics it is scored on, the baseline it is compared with, and the training and evaluation protocol.

5.1. Simulated Platforms

Every environment is simulated by MuJoCo, the rigid-body physics engine used throughout this thesis (Section 2.7) [TET12]. The two manipulation tasks run the seven-joint Panda manipulator in MuJoCo, with the model files, task definitions and demonstrations of D3IL [Jia+24]. The quadrotor benchmark runs the Skydio X2 model from MuJoCo Menagerie [ZTM22] with the quadrotor task patch distributed by MuJoCo MPC [How+22]. MuJoCo is the simulator in every case; D3IL and MuJoCo MPC supply the task files and data that run in it. Figure 5.1 shows both models; the X2 dimensions used in projection are drawn in Figure A.1 in the appendix. Table 5.1 gives the properties that enter the experiments.

How a plan reaches each of them differs. On the manipulator the planner commands an end-effector increment; inverse kinematics turns each Cartesian setpoint into joint commands. On the quadrotor the planner updates a commanded position increment once every three 0.01 s MuJoCo physics steps. Between planner updates, the cascaded geometric controller of Subsection 4.3.5 computes rotor thrusts at every physics step.



Figure 5.1.: The two simulated platforms, rendered in MuJoCo from the model files the experiments load, without their environments. Left: the Panda manipulator, from the model files of D3IL, used for D3IL-avoiding and D3IL-aligning. Right: the Skydio X2 model from MuJoCo Menagerie with the MuJoCo MPC task patch, used in the three aerial scenes. Both are simulated by MuJoCo. The two renders are not to scale with each other. The dimensions of both platforms are in Table 5.1; the quadrotor’s rotor layout and the footprint the projection uses for it are drawn in Figure A.1 in the appendix.

Table 5.1.: Properties of the two platforms that enter the experiments. The Panda is taken from the model and task files of D3IL, the X2 from MuJoCo Menagerie with the MuJoCo MPC task patch; the X2 footprint includes the rotor disks of the collision model (Figure A.1 in the appendix).

Platform	Model	Plan commands	Plan interval
Franka Panda	Emika seven joints; rod end effector, radius 0.01 m, length 0.30 m; wrist RGB camera	end-effector increments, planar or spatial	every physics step
Skydio X2	four rotors; mass 1.325 kg; rotor centres 0.228 m out; 0.54×0.62 m footprint; thrust $[0, 13]$ N per rotor	spatial position increments	0.03 s of simulated time; controller at 0.01 s

5.2. Tasks and Environments

The three environments differ in what the planner observes, how many dimensions its plan has, and which constraint forms the projection must satisfy. Each is given below with its task, its constraint set and the values that enter the experiments.

5.2.1. Obstacle Avoidance

The obstacle-avoidance task is the benchmark of DPCC, simulated in MuJoCo from the task files and demonstrations of D3IL: the same environment, Panda manipulator, demonstrations, constraint geometries, projection configuration and evaluation protocol [Jia+24; RRS25] (Subsection 4.6.1). Holding that complete benchmark fixed provides the controlled validation of replacing DPCC’s diffusion model with the three flow-based objectives. A manipulator moves its end effector across a field of six obstacles to a goal line (Figure 5.2).

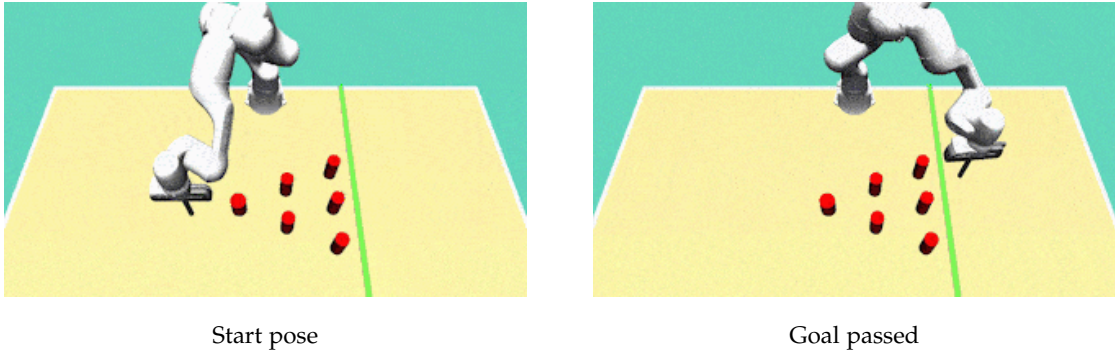


Figure 5.2.: The Panda manipulator in the MuJoCo obstacle-avoidance scene. Left: the starting pose, with six red posts between the arm and the goal line. Right: the end effector beyond the green goal line.

At test time the plan is constrained by halfspaces that the demonstrations never respect, in three geometries — *top-left-hard*, *top-right-hard* and *both-hard*, defined in Table 5.2 and drawn in Figure 5.3 — together with the circular keep-out regions of Subsection 4.6.1. The geometries are those of the DPCC configuration [RRS25, Sec. 6.1], and all three share the workspace $x \in [0.2, 0.8]$, $y \in [-0.3, 0.4]$, the six obstacles of radius 0.025 and the goal line at $y = 0.35$. Every constraint also has a tightened version, shrunk by the margin of 0.025 that DPCC introduces and applies to this benchmark [RRS25, Sec. 5.3] (Subsection 4.5.2). The same holds for every constraint set of this thesis, and one convention is fixed here for every result that follows: **every result of Chapter 6 is on the tightened set**. It is the set DPCC reports on, and it is the set

that gives the better result: on this benchmark, tightening raises the published rate of success with constraint satisfaction from 0.49 to 0.98 and cuts the violating steps from 6.0 to 0.0 per episode [RRS25, Table 1]. Subsection 5.5.1 quotes those numbers in full next to their reproduction; the shrinkage itself, the Minkowski difference with the mismatch ball, is stated in Subsection 4.5.2. Untightened evaluations are therefore not reported. An unprojected plan is the same plan on either set, and its violations are scored against the tightened boundary.

Table 5.2.: The three constraint geometries of the obstacle-avoidance task, in metres. A halfspace keeps the planned end-effector position below the line through its two points and the keep-out disk outside the circle; the tightened version of each moves the line and enlarges the disk by 0.025. The last column counts the D3IL demonstrations that satisfy the geometry at every step.

Geometry	Halfspace: below the line through	Keep-out disk	Satisfied by
top-left-hard	$(0.8, -0.5)$ and $(0.4, 0.5)$	$(0.4, 0.08)$, $r = 0.08$	0 of 96
top-right-hard	$(0.2, -0.5)$ and $(0.6, 0.5)$	$(0.6, 0.08)$, $r = 0.08$	1 of 96
both-hard	$(0.8, -0.3)$ and $(0.575, 0.5)$; $(0.2, -0.3)$ and $(0.425, 0.5)$	$(0.5, -0.09)$, $r = 0.08$	2 of 96

Each geometry is two things (Figure 5.3). The *halfspace* is a straight line across the field with one side declared out of bounds for the planned end-effector position: the plan must stay below the line, so the region above it — the shaded corner — is excluded, and the constraint is a single linear row per plan step. The *keep-out disk* is a circle of radius 0.08 m that the planned position must stay outside of, placed over one of the posts so that it covers the post and a margin of free table around it; it is the quadratic row of the set, and it is what makes the free lane through the field a corridor between the halfspace on one side and the disk on the other. *both-hard* has two halfspaces, one from each side, and its disk on the first post. Neither element is a physical object: the posts are the only obstacles the arm can collide with, and a plan that crosses a halfspace or enters a disk violates the constraint without any collision. The tightened version of each set moves the line and enlarges the disk by 0.025 m, the dashed boundaries of the figure.

5. Experimental Setup

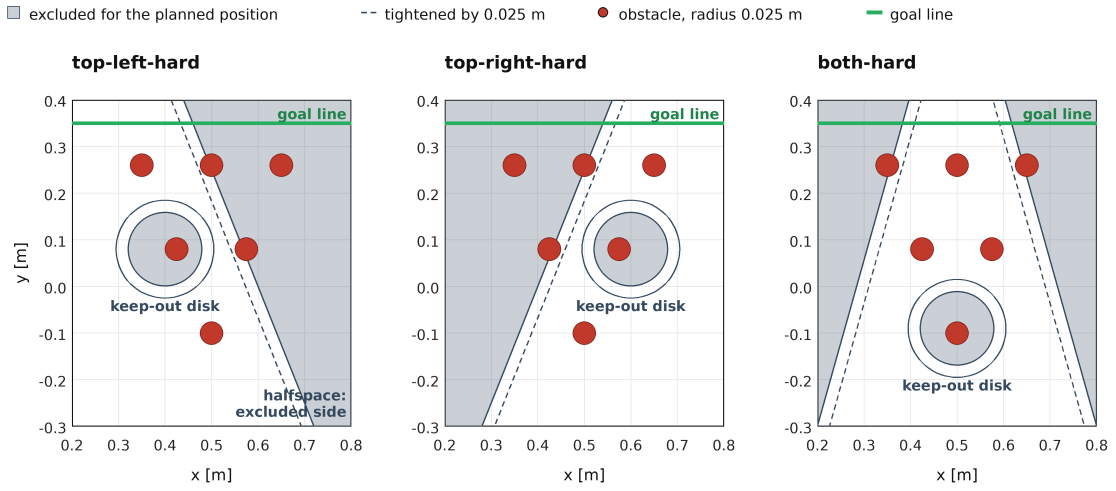
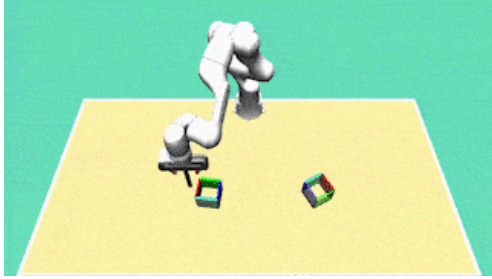


Figure 5.3.: The three constraint geometries of Table 5.2, seen from above on the field of Figure 5.2, with nothing but the constraints drawn: the shaded region is the side of the halfspace the planned position is excluded from, the shaded circle is the keep-out disk it must stay outside of, the dashed boundaries are the tightened versions, the red disks are the six posts and the green line the goal line. The demonstrations are drawn over the same three panels in Figure 5.9.

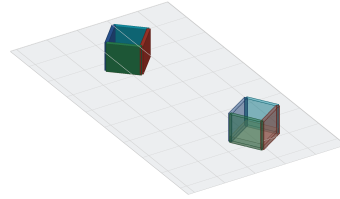
5.2.2. Vision-Conditioned Alignment

The alignment task of D3IL [Jia+24] asks the manipulator to push a box into a target pose on a table. The task runs in MuJoCo; its task files, demonstrations and camera set-up are D3IL’s. Around them this thesis built a constrained-control environment (Subsection 4.6.2): a three-dimensional plan with its dynamics rows, a constraint set on the planned end-effector position, and the coupling of the visual encoder to the generative network. The policy observes two RGB camera streams, a front view and an in-hand view, instead of the state (Figure 5.5). Each episode starts from an initial box pose and a target pose, together called a context. The tightened constraints are shrunk by 0.03 m.

Where the box is can only be seen. The vector half of the observation is $(p_{\text{des}}, p) \in \mathbb{R}^6$, the commanded and the measured end-effector position (Subsection 4.6.2); it carries no box pose, no target pose and no object state of any kind. The two RGB streams are the sole source of both, so the policy must locate the box in the images before it can move toward it — which is why a 0.10 m box in a 0.20×0.15 m draw region is a harder problem than its planar geometry suggests.



MuJoCo simulator view



orthographic task-geometry view

Figure 5.4.: The vision-conditioned alignment task. Left: a frame rendered by MuJoCo with the manipulator, movable box and coloured target. Right: a geometry view of the first held-out context, reconstructed from the scene description and its recorded box and target poses. The transparent footprint marks the target pose. These panels show the task; the policy itself observes the front and in-hand RGB streams.

Obstacle avoidance is a planar task that imitation policies already solve: the best policies reported by D3IL reach a success rate of 0.95 on it [Jia+24, Table 3], so it tests constraint satisfaction on a plan that is easy to produce. This environment raises three things at once. The plan is three-dimensional rather than planar; the policy sees the scene through two cameras and a visual encoder rather than through a state vector; and

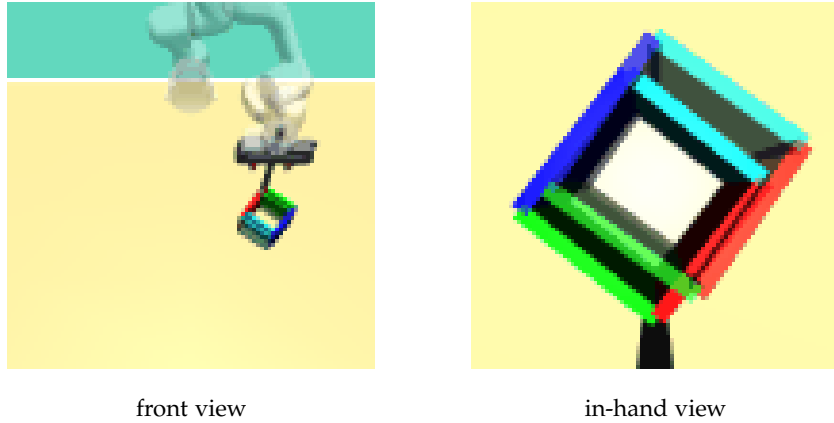


Figure 5.5.: The two 96×96 RGB observations the visual policy consumes, at one instant of a recorded demonstration with the box already at its target, rendered by MuJoCo. Left: the front view of manipulator, box and table. Right: the in-hand view of the box and its coloured sides. Camera placements are those of the D3IL task files [Jia+24].

the task itself is not solved — the best image-based policy reported by D3IL finishes roughly a third of its episodes (Subsection 5.5.2, Table 5.7). It therefore asks whether the DPCC construction still holds where the plan is harder to produce, and it puts the projection where only some episodes finish at all.

The active geometry is *combined-5* (Table 5.3, Figure 5.6): a halfspace that cuts off the far corner of the table, a circular keep-out region between the box and its target — the same two geometric forms D3IL-avoiding uses, on a task whose plan is three-dimensional. Neither is respected by the demonstrations, so satisfying them is left to the projection.

Table 5.3.: The constraint set of the alignment task, geometry *combined-5*, in metres, on the planned end-effector position. The tightened version moves the halfspace 0.03 m inward and grows the keep-out radius by the same margin.

Constraint	Form	Values
Halfspace	below the line through two points	$(0.65, 0.45)$ and $(0.80, -0.45)$
Keep-out region	outside a circle	centre $(0.50, 0.00)$, radius 0.06
Tightening	both, moved inward	0.03

The context is drawn at random, and this is the difference from obstacle avoidance that matters most for what the policy has to do. In obstacle avoidance every episode begins at one fixed end-effector position and the six posts never move, so a policy can learn a route. Here the box is placed anywhere in $x \in [0.4, 0.6]$, $y \in [-0.25, -0.10]$ at any yaw in $\pm 90^\circ$, and its target anywhere in $x \in [0.4, 0.6]$, $y \in [0.20, 0.35]$ at any yaw in $\pm 90^\circ$; the demonstrations were recorded from such draws and the evaluation replays an enumerated set of them (Subsubsection 5.6.3.2). Two episodes therefore ask for two different pushes. Figure 5.6 draws the ten contexts the evaluation replays, which are the ten every result in Section 6.2 is averaged over. They are ten different pushes: their lengths range from 0.41 to 0.52 m and average 0.4530 m, the starting distance Section 6.2 reports, so a policy that had learned one route would satisfy none of them. One context in that figure is drawn with the real box footprints at their recorded yaw, which every draw carries, because the marks alone understate the object — a 0.10 m box has to be pushed around a 0.06 m keep-out region.

5. Experimental Setup

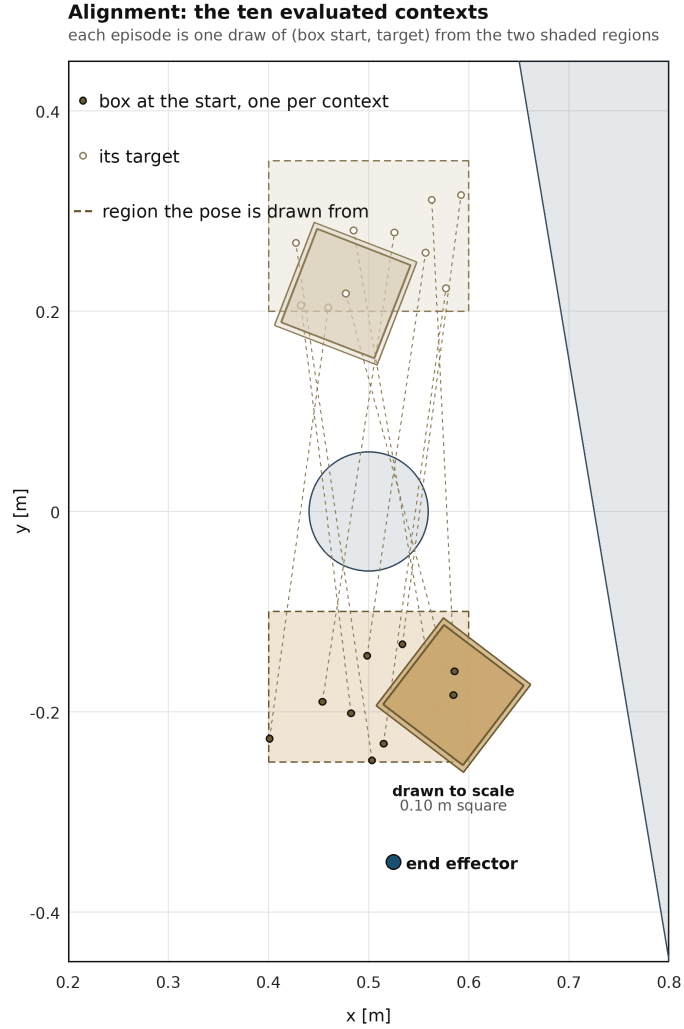


Figure 5.6.: The ten contexts the evaluation replays, on the plane of Table 5.3. Filled marks are the box at the start of an episode, hollow marks its target, and a dotted line joins the two poses of one context; the dashed rectangles are the regions the environment draws each pose from. Both marks are box *centres*, which is what the context and the final-distance metric refer to. One context also carries the real 0.10 m footprints at their recorded yaw.

5.2.3. Quadrotor Benchmark

The quadrotor benchmark was built for this thesis around the Skydio X2 model of MuJoCo Menagerie [ZTM22] (Subsection 4.6.3). The vehicle is underactuated and open-loop unstable (Section 2.7), and its plan is tracked by the controller of Section 4.3. Three scenes are defined (Figure 5.7, Table 5.4):

UAV-pillars — D3IL-avoiding flown by the quadrotor. The obstacle field of Subsection 5.2.1 is scaled by 36, so that the rod’s radius becomes the vehicle’s rotor reach, and its six obstacles are extruded into pillars; the three test-time constraint geometries of that benchmark are mapped into the arena at the same scale. The planner is the one trained on the avoiding demonstrations and is not retrained: the quadrotor flies the plans it produces. The scene takes the configurations that already solve D3IL-avoiding, flies them, and reads the flown outcome beside the table outcome of the same configuration, before and after projection (Subsection 6.3.1). The constraint set of each geometry is the table’s: one or two halfspaces and one keep-out disk, which covers one of the six pillars. The other five pillars are obstacles of the scene and belong to no constraint set, on D3IL-avoiding or on UAV-pillars; a flight in which the vehicle collides with one ends there.

UAV-corridor — the three-dimensional test. Two parallel walls leave a clear width of 1.90 m, and the scene carries two test-time constraints, evaluated separately, as the geometries of D3IL-avoiding are. The first, *tilt*, is a halfspace boundary that slopes by 14° across the corridor in the horizontal plane and is leaned over by 60° from the vertical about the launch altitude, so that one plane pushes the vehicle sideways and downward at once: at the launch altitude a sideways move alone does not clear it on any lane. The second, *hump*, is a roof in the vertical plane that rises from the floor at $x = -1.5$ m to 1.10 m at $x = 0$ and falls back to the floor at $x = 1.5$ m, which the vehicle must climb over; the ceiling of the workspace box is raised for it. The models are trained on the demonstrations of a narrower, level corridor and never see either boundary, so both are test-time constraints in the same sense as the halfspaces of D3IL-avoiding.

UAV-s-curve — the tracking-controller caveat. Two offset corridor segments, joined by a diagonal passage, require two sharp changes of direction, and the scene is reported for what those turns do to the controller (Subsubsection 6.3.3.3), not as a third benchmark.

The three scenes use the same two constraint forms as D3IL-avoiding — halfspaces and circular keep-out regions — so the projection solves the same kinds of rows on UAV-pillars as on D3IL-avoiding (Figure 5.8); on UAV-pillars they are the avoiding

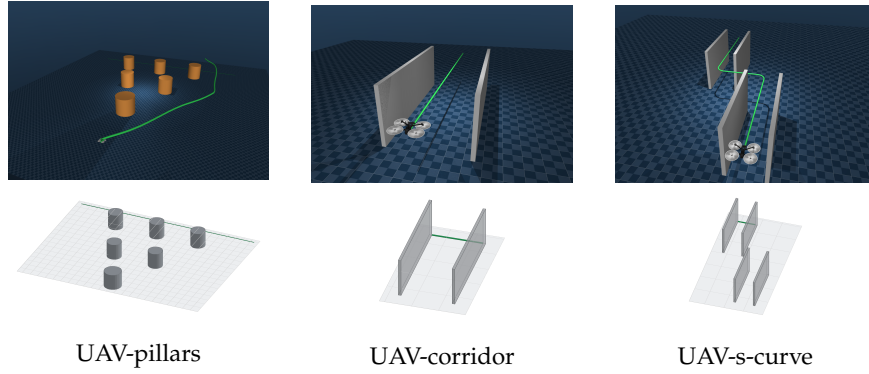


Figure 5.7.: The three quadrotor scenes. Upper row: the MuJoCo scene of each, seen from behind the start, with the Skydio X2 at its start and a reference path in green: on UAV-pillars a mapped demonstration of D3IL-avoiding, on the other two the generator’s reference route (Subsection 5.3.3). Lower row: the same scene files, orthographic, at true proportions; the green line at the far end is the finish line, at the end of the walls on UAV-corridor and UAV-s-curve. The test-time constraints of UAV-corridor are drawn in Figure 5.8.

rows themselves. What differs on the two scenes with quadrotor demonstrations is that a wall acts only over the span of x on which it exists, that the corridor’s two constraints leave the horizontal plane — the tilt is one leaned plane, the hump a roof — and that every wall, plane, roof and corner disk is inflated by the rotor reach of 0.31 m before it reaches the solver, because the vehicle is a body and not a point. The tightened variants move each of these boundaries a further 0.025 m inward, the margin of DPCC.

Figure 5.8 draws the constraint sets, one column per scene and one panel per set. On UAV-pillars the three geometries of D3IL-avoiding are mapped into the arena, halfspaces and keep-out disk at scale 36, and the six pillars are drawn as the physical obstacles they are; the mapped scale already carries the vehicle’s extent, so no rotor-reach band is added there and the tightening is the mapped 0.90 m. On UAV-corridor the tilt is drawn twice — from above (x - y) at the launch altitude, where the leaned plane cuts the corridor, and from the side (x - z) along the corridor, where the lean itself is — and the hump from the side, where the roof is; the walls hold under both. UAV-s-curve has one set. Every panel names the plane it is drawn in. A wall constrains only the span of x over which it exists, which is why the halfspace it excludes stops short of the panel edge.

Table 5.4.: The constraint set of each quadrotor scene: halfspaces and circular keep-out regions (Subsection 4.6.3). On UAV-corridor and UAV-s-curve each boundary is inflated by the vehicle’s rotor reach of 0.31 m before it reaches the solver, and the tightened variants move it a further 0.025 m inward. On UAV-pillars the boundaries are those of D3IL-avoiding mapped into the arena, and the vehicle’s extent enters through the scale, which sends the rod’s radius onto the rotor reach.

Scene	Constraint	Values
UAV-pillars	three avoiding geometries, mapped	the halfspaces and the keep-out disk of <i>top-left-hard</i> , <i>top-right-hard</i> and <i>both-hard</i> (Table 5.2) at scale 36: keep-out radius 2.88 m; physical pillars of radius 0.90 and 1.08 m in an arena of 21.6×25.2 m
UAV-corridor	two wall halfspaces	inner faces $y = \pm 0.95$ m, active over $x \in [-2, 2]$; clear width 1.90 m
	<i>tilt</i> : the leaned slide	the sloping line from $(-2.0, 0.95)$ to $(2.0, -0.05)$, 14° in the horizontal plane, leaned by -60° from the vertical about $z = 1.11$ m: the feasible distance grows by $\tan(-60^\circ)(z - 1.11)$, so descending gains room; ceiling 1.80 m
	<i>hump</i> : two roof halfspaces (x - z)	from $(-1.5, 0)$ to $(0, 1.10)$ and from $(0, 1.10)$ to $(1.5, 0)$, the plan above both, each active over its own span of x ; ceiling raised to 2.80 m
	wall ends	four disks of radius 0.05 m at $x = \pm 2$, $y = \pm 1$
UAV-s-curve	four switched wall halfspaces	feasible $y \in [-1.25, -0.35]$ over $x \in [-3, -0.5]$, and $y \in [0.35, 1.25]$ over $x \in [0.5, 3]$
	inner corners	two disks of radius 0.05 m at $(\mp 0.5, \mp 0.3)$

5. Experimental Setup

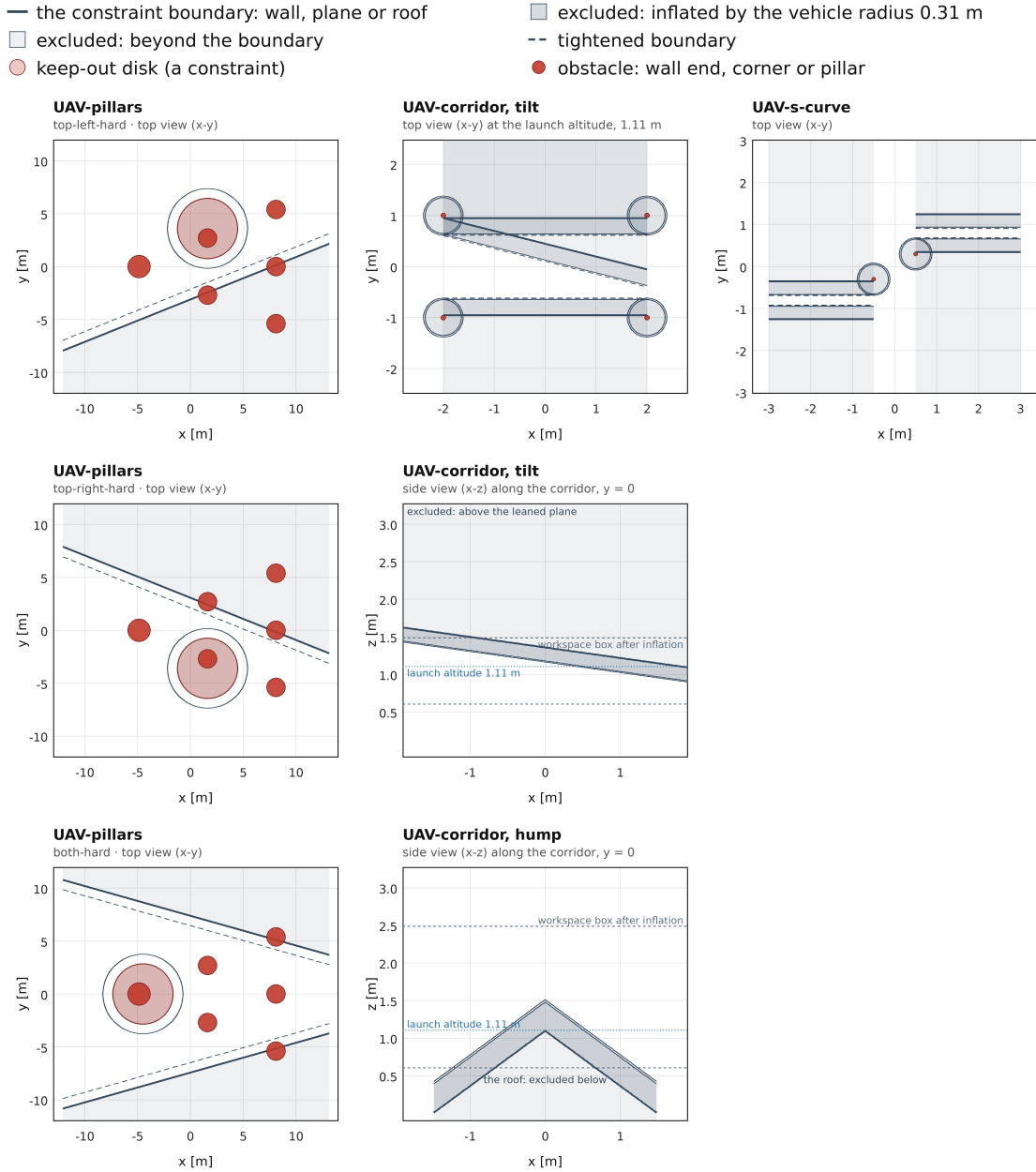


Figure 5.8.: The constraint sets of the quadrotor scenes, one column per scene and one panel per set. Solid line: the constraint boundary; grey band: the 0.31 m rotor reach added to it; dashed: the tightened boundary; faint wash: the rest of the excluded region; white: where the plan may go. Left: UAV-pillars, the three geometries of Table 5.2 at scale 36, the keep-out disk translucent over the pillar it covers, the six pillars solid. Middle: UAV-corridor, the tilt from above at the launch altitude and from the side, and the hump from the side. Right: UAV-s-curve from above.

5.3. Datasets

5.3.1. Obstacle Avoidance

The demonstrations are those of the D3IL obstacle-avoidance task [Jia+24], as used by DPCC. Each step records the commanded and the measured planar end-effector position and the planar increment between consecutive commanded positions (Subsection 4.6.1). The dataset holds 96 demonstrations, drawn in Figure 5.9. None of them satisfies the constraints of top-left-hard, one those of top-right-hard and two those of both-hard (Table 5.2), so satisfying the constraints is left to the projection.

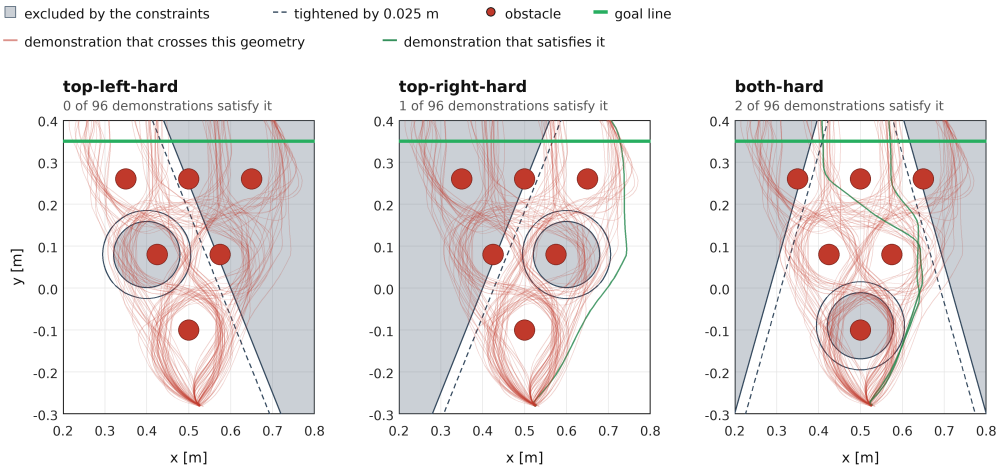


Figure 5.9.: The 96 demonstrations against the three constraint geometries of Table 5.2, the companion of Figure 5.10 and Figure 5.11. Shaded: the region excluded by the halfspaces and the keep-out disk; dashed: the tightened boundary. A demonstration is drawn red where it crosses the geometry of its panel and green where it satisfies it at every recorded step. The number of green lines in each panel is the last column of Table 5.2.

5.3.2. Vision-Conditioned Alignment

The demonstrations are those of the D3IL alignment task [Jia+24]: its 900 training demonstrations, with the two camera streams rendered by MuJoCo. For this thesis each step is laid out as the nine-dimensional plan of Subsection 4.6.2 — the three-dimensional increment and the commanded and measured end-effector positions — and paired with the two images. Cut into plans of eight steps, they give 168 274 training windows.

The demonstrations stand in the same relation to the constraints as the obstacle-avoidance ones. Figure 5.10 draws every recorded context — the 60 used for training and the 60 held out — as the straight push from the box to its target, which is the shortest route a demonstration could take. The figure is the top view of the table plane, x - y , the plane the push happens in; it draws the contexts and not the demonstrated end-effector paths themselves. That segment crosses the keep-out region in 106 of the 120 contexts, and in 119 of them once the region is tightened. The demonstrations were recorded without the region existing, so the task they teach is precisely the one the constraint forbids; going around it is the projection’s work, not something the policy could have imitated.

In all three environments, a random tenth of the training windows is held out and used only to select the checkpoint (Table 5.8).

5.3.3. Quadrotor Benchmark

Two of the three scenes have quadrotor demonstrations, and one has none. On UAV-pillars the planner is the one trained on the demonstrations of D3IL-avoiding (Subsection 5.3.1) — human teleoperation of the manipulator — and it is not retrained: the scene is that obstacle field scaled into an arena, and the quadrotor executes the plans the planner produces there (Subsection 5.2.3). The scene therefore compares the execution of the same plans by two plants, the manipulator of D3IL-avoiding and the quadrotor with its controller.

The demonstrations of UAV-corridor and UAV-s-curve were generated for this thesis as described in Subsection 4.6.3. That section gives the waypoint-and-arc construction, cosine speed profile, tracking equations, rejection rule and recorded observation–action pair; Table 5.5 records only their numerical and scene-specific parameters. The corridor with the sloping halfspace and the roof reuses the demonstrations and trained models of the level corridor, because both boundaries are test-time constraints.

These demonstrations are not human. There is no public teleoperated quadrotor dataset for these scenes, so the data this thesis trains on was *generated* for it: a reference path is laid out analytically per passage class (Table 5.5), and every episode is then *flown in MuJoCo* by the same cascaded geometric controller the evaluation uses (Section 4.3), under the same physics and at the same time step. What is recorded is the measured state of that flight, not the reference. An episode is kept only if the flight it produced stays airborne and within its collision budget (Table 5.5). Each demonstration is therefore a trajectory the plant has been shown to track, and *expert* means that in this thesis and nothing more: a competent generated flight, not a human one.

Where these demonstrations differ from the two manipulation datasets is that they were generated against the obstacles they fly among, and Figure 5.11 shows what

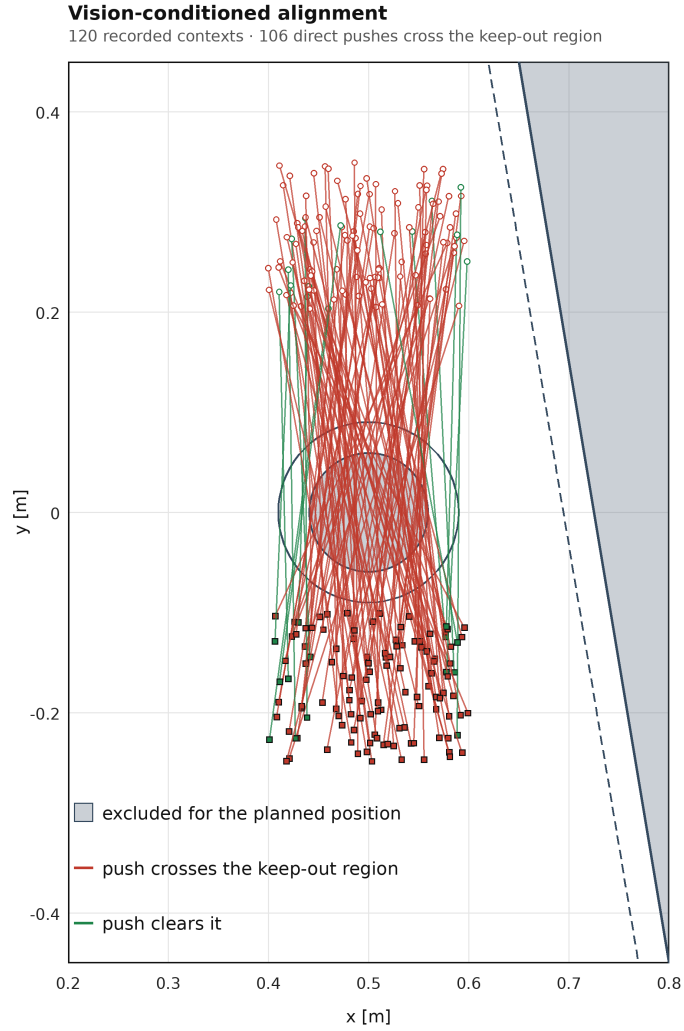


Figure 5.10.: The 120 recorded contexts of the alignment task over the constraint set of Figure 5.6. Each line joins the initial box centre (square) to its target centre (circle); red where that straight push crosses the circular keep-out region, green where it clears it. The draw ranges of Subsection 5.2.2 are visible as the two bands the squares and circles fall in.

Table 5.5.: Generation of the quadrotor demonstrations of UAV-corridor and UAV-s-curve; UAV-pillars has none and uses the avoiding planner. Shared by the two scenes: altitude drawn uniformly from $[0.90, 1.30]$ m per episode, physics step 0.01 s, recording every third physics step (0.03 s), the nominal gains of the tracking controller, rejection of an episode that falls below 0.50 m or touches an obstacle on more than the stated fraction of its steps. Durations and offsets are drawn uniformly from the stated ranges.

	UAV-pillars	UAV-corridor	UAV-s-curve
Passage classes	— (the D3IL-avoiding demonstrations)	three lanes at $y \in \{-0.12, 0, 0.12\}$ m	one
Reference path	—	straight, x from -2.8 to 2.8 m	waypoints joined by arcs of radius ≤ 0.45 m
Duration	—	$[6, 10]$ s	$[16, 22]$ s
Lateral offset	—	start, $[-0.03, 0.03]$ m, centre lane	whole path, $[-0.04, 0.04]$ m
Maximum collision fraction	—	2 %	8 %
Episodes generated	—	500	500
Episodes accepted	—	500	500

follows from it. The figure draws the demonstrations of the two scenes that have their own over their constraint sets, the tightened boundary dashed, and the grey silhouette on a route is the vehicle to scale at its tightest moment (Figure 5.1), for size only. On UAV-s-curve the single route satisfies the constraint set the projection later enforces, tightening included: the generator threads a passage the scene already contains, so nothing in the data has to be corrected. The projection is then asked to repair a plan that was never broken, which is why that scene can expose a limit of the tracking controller but cannot order the projection methods.

UAV-corridor is built the other way, and it is the way the obstacle-avoidance benchmark is built. The three lanes clear both walls, but every one of them crosses the tilt — at the launch altitude, where the leaned plane cuts the corridor along the same line as before it was leaned, by 0.24, 0.36 and 0.47 m for the left, centre and right lane — and every one of them passes under the hump, whose roof at its peak stands 1.48 m above the floor once inflated while the demonstrations fly between 0.90 and 1.30 m. Both boundaries are introduced at test time and did not exist when the demonstrations were flown. The scene in which the constraint is new to the data is exactly the scene in which the projection has something to do.

UAV-pillars is the second kind by construction, and more sharply than the corridor: its demonstrations are the 96 of D3IL-avoiding (Figure 5.9), mapped into the arena at scale 36 by the same similarity map as its constraint set, so they are not drawn again

in Figure 5.11, and its constraint sets are the three test-time geometries of that task, which 0, 1 and 2 of the 96 demonstrations satisfy (Subsection 5.2.1); so on that scene the projector has the same repair to do as on D3IL-avoiding, against the same constraint sets, which leave five of the six pillars out (Subsection 5.2.3). Subsection 6.3.1 reads it on those terms, on the configurations that win on D3IL-avoiding.

On the two quadrotor-demonstrated scenes that figure draws the references the generator laid out rather than the flights that tracked them, which is the conservative choice: a constraint crossed by the reference is crossed by construction rather than by tracking error.

One thing has to be read correctly in that figure and in every quadrotor result that follows. A violation is a property of the *path*, not of the aircraft: what is tested is the line of vehicle centres against the constraint surfaces, and those surfaces already carry the 0.31 m rotor reach (Table 5.9), so the vehicle’s own extent is inside the boundary rather than measured against it. A route turns red when its centre line crosses an inflated surface, and nothing is ever scored by sweeping the rotor disks through an obstacle. The scenes are built to be flyable on those terms — the free channel that remains after inflation is a real one — so a red route means a plan that had somewhere else to go, not a vehicle too large for the gap.

5.4. Metrics

5.4.1. Obstacle Avoidance

The metrics are those of DPCC [RRS25, Sec. 6.1]: the number of control steps to reach the goal (steps per episode), the success rate of reaching the goal (fraction of episodes), the success rate of reaching the goal while satisfying the constraints at every step (fraction of episodes), and the number of control steps on which a constraint is violated (steps per episode).

The first of those is a headline number in both works this chapter measures against, and not a quantity introduced here: DPCC reports the control steps an episode takes beside its success rates [RRS25, Sec. 6.1], and the endpoint-projection work, which follows the setup of DPCC on this same D3IL task and counts projection after every sampling step, the projection of DPCC, among its baselines, reports the average number of steps the manipulator takes to reach the target on its constraint-satisfying episodes as one of its three columns, beside the safety rate and the computation time [LAA25, Sec. VII-A, Table II]. This thesis compares the two projection methods in the same way: endpoint projection against the per-step projection of DPCC as the baseline, with the control steps among the measures. A plan that arrives in fewer control steps is a plan that completes the task sooner, which is why it is reported next to success rather than

5. Experimental Setup

- demonstration / reference path satisfies the constraints
- demonstration / reference path crosses them
- the vehicle to scale, for size only (the 0.31 m reach is in the surface)

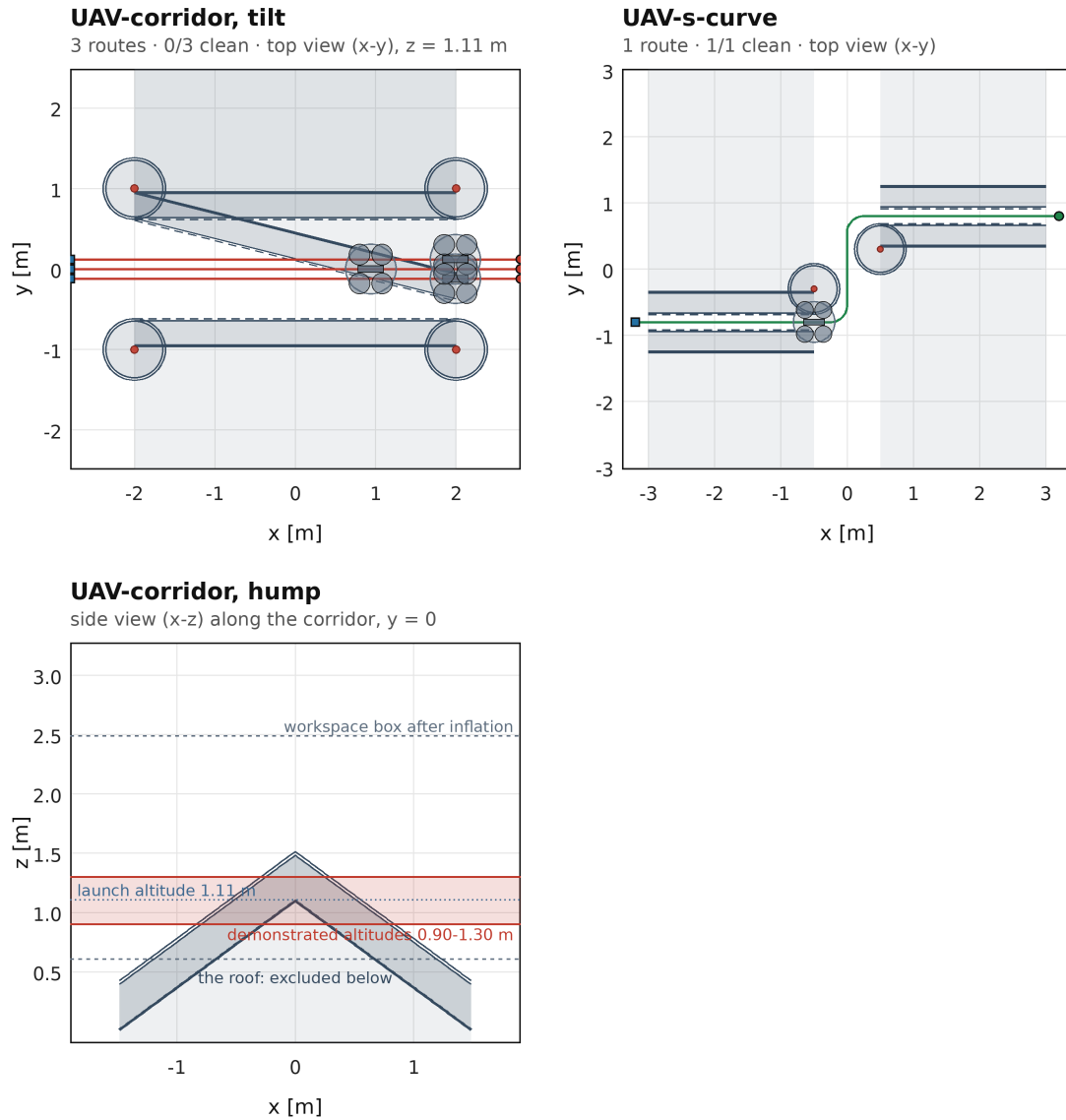


Figure 5.11.: The demonstrations of the two quadrotor-demonstrated scenes over their constraint sets, drawn as in Figure 5.8: UAV-corridor, the three reference lanes against the tilt from above and the flown altitude band against the hump from the side; UAV-s-curve, its single route. UAV-pillars flies the demonstrations of D3IL-avoiding (Figure 5.9) and is not drawn. Green satisfies the constraints, red crosses them; the tightened boundary is dashed and the grey silhouette is the vehicle to scale.

folded into it.

To these this thesis adds the time to compute one action (milliseconds, or seconds where a configuration is slow enough to warrant it), the quantity DPCC states for its own method [RRS25, Sec. 6]. It is measured around the planner at every control step and covers everything the planner does to produce the action that is executed: generating the B candidate plans, projecting them onto the constraint set, and selecting one. The same measurement is used in all three environments, and Subsection 5.6.1 gives the machine every configuration was measured on.

5.4.2. Vision-Conditioned Alignment

Each episode is scored by

- the final distance between the box and its target in the table plane (metres), summarised over the contexts by its median and its minimum, and the share of the initial distance that was closed, $(d_{\text{init}} - d_{\text{final}})/d_{\text{init}}$ in per cent, against an initial distance averaging 0.4530 m: 100 % is a box brought onto its target and 0 % one that was not moved towards it at all;
- whether the box was moved at all (a count of contexts);
- whether the box ends in position: within 0.018 m of its target in the table plane, the position tolerance of the task's own success test (a count of contexts); the orientation half of that test is not used, for the reason given below;
- the number of control steps on which a constraint is violated (steps per episode);
- the time to compute one action (milliseconds), as in Subsection 5.4.1.

When generative models are compared, the distance is measured on plans executed without projection, so that it reflects the generative model alone; when projection methods are compared, it is measured with projection.

5.4.3. Quadrotor Benchmark

Each flight is scored by

- success: the vehicle crosses the finish line of its scene. This is the success criterion of D3IL-avoiding transferred to the quadrotor — there an episode succeeds when the end effector crosses the goal line (Subsection 5.4.1), here when the vehicle crosses the line that closes its route. On UAV-corridor and UAV-s-curve a crossing counts only in controlled flight: the flight was not aborted (below), its lowest

altitude stayed above 0.2 m, and the vehicle collided with the scene's walls on no more than 2 % of its physics steps on UAV-corridor and 8 % on UAV-s-curve, the tolerances with which the demonstrations of each scene were accepted. On UAV-corridor and UAV-s-curve the finish line is the end of the walls, $x = 2.0$ m and $x = 3.0$ m, where the walls and every test-time constraint end; a flight is scored the moment it passes it and flies on as before. On UAV-corridor the line through the route's goal point, 0.8 m further on, is not used: the episode limit was set from the demonstrations of the level corridor, and a projected flight that has descended under the tilt or climbed over the hump can reach that limit after the constraints have ended while still flying towards the goal. The counts at the goal point are printed beside the others where they differ. Every quadrotor table and figure of Chapter 6 calls a flight that does so a *success* and prints the count of them;

- whether it is violation-free: no control step on which the flown path violates a constraint. The test is on the path, not on the airframe: the surfaces it is tested against already carry the 0.31 m rotor reach (Table 5.9);
- success with constraint satisfaction: success on a violation-free flight, as on D3IL-avoiding;
- on UAV-corridor and UAV-s-curve, where almost no flight is free of violations, the constraint is read as the number of *violating control steps* per flight, which the tables of those scenes print in place of success with constraint satisfaction. Fewer is better, and the cost frontier of UAV-corridor is drawn on violating steps against the time to compute one action, among the configurations that reach the finish line on every flight; within groups of configurations that violate about equally it is drawn as on D3IL-avoiding, on control steps to the finish line against time;
- on UAV-pillars, the metrics of D3IL-avoiding (Subsection 5.4.1) scored on the flown path — success at the mapped goal line, success with constraint satisfaction, violating control steps, control steps — so that a plan's outcome on D3IL-avoiding and on UAV-pillars is read in the same units. A flight in which the vehicle collides with a pillar ends there and is not a success; the collision is not a constraint violation, since the pillars it concerns are in no constraint set, and the tables count it through the success column;
- whether the flight was *aborted*: the evaluation ends a flight the moment the vehicle inverts — its body vertical axis points below the horizon — because a vehicle that has lost attitude control does not recover and would otherwise be flown on to the episode limit. The guard tests the attitude only, never a collision. An aborted

flight is not a success, its outcome columns are read as they stood when the guard fired, and the control step at which it fired is recorded. The quadrotor tables of Chapter 6 print the count of aborted flights per cell;

- the number of control steps (steps per flight), and the final distance to the goal point of the route (metres), which describes where a flight that does not succeed ends;
- the time to compute one action (milliseconds, Subsection 5.4.1), split into generation and projection.

5.4.4. Aggregation and Variability

Every metric above is measured once per episode and reported as an average over many of them, so a bare average would hide how much it moves. Chapter 6 therefore prints *mean $\pm$ spread* wherever a spread exists, and what the spread is over depends on what the result is a claim about.

- **Multi-seed results** — every D3IL-avoiding table — are reported as the mean over the five training seeds and the *sample standard deviation across those seeds*. A claim there is a claim about a model and not about one training run, so the variation that matters is how much the result moves when the model is trained again. It is also the larger of the two spreads available and therefore the conservative one. Each seed is first averaged over the constraint geometries it covers, then the mean and the standard deviation are taken over the seeds (Subsubsection 5.6.3.1).
- **Single-seed results** — the one-seed D3IL-avoiding tables, D3IL-aligning and the quadrotor scenes — carry no seed dimension, so the spread is the sample standard deviation *across the episodes, contexts or flights* of the cell, which is stated with each table. Where a cell averages several constraint geometries, the within-geometry standard deviations are pooled.
- **Counts are printed as counts.** A result of the form “10/10 flights” or “20/20 episodes” is not an average and gets no $\pm$; neither does a median, which is reported with the number of contexts it is taken over.

5.5. Baselines

Three generative models are compared with the diffusion model of DPCC (*Diffusion* in tables): *instantaneous-velocity matching* (FM) [Lip+23], which learns the instantaneous velocity of the probability path; *analytic average-velocity matching* (MeanFM)

[Gen+25a], which learns the average velocity over an interval through an analytic Jacobian–vector-product target; and *consistency-interpolated average-velocity matching* (*CI-MeanFM*) [Zha+25b], whose consistency step ratio α interpolates the target between the instantaneous velocity at $\alpha = 1$ and a stop-gradient prediction towards the analytic average-velocity objective as $\alpha \rightarrow 0$. In this thesis α is annealed from 1 and held at a final value α_{end} , which each result states. Two projection methods are compared: the per-step projection of DPCC and endpoint projection [LAA25]. All generative models share the temporal U-Net backbone of the baseline, with 4.0 M parameters, unless a table states otherwise. Throughout all figures and tables, the four models are labelled *Diffusion*, *FM*, *MeanFM*, and *CI-MeanFM*, respectively; running prose retains the full mechanism names.

5.5.1. Obstacle Avoidance

The baseline is DPCC in the configuration its authors report: the diffusion model at a step budget of $K = 20$, per-step projection on tightened constraints, and selection by cumulative projection cost over $B = 4$ candidate plans [RRS25, Sec. 6.1]. Table 5.6 places the published result next to its reproduction in this thesis. The baseline is compared on the tightened constraint set, and so is every other configuration of this thesis (Subsection 5.2.1): tightening is the margin DPCC itself introduces so that a plan projected onto the shrunk set stays feasible under the tracking and discretisation error of execution [RRS25, Sec. 5.3], it is the setting under which DPCC reports its results, and it is the setting that gives the better result. DPCC reports the same configuration with and without tightening [RRS25, Table 1 and Fig. 2]: without it, the rate of success with constraint satisfaction is 0.49 at 76.1 ± 21.5 steps with 6.0 ± 11.5 violating steps per episode, and 0.41, 0.51 and 0.49 across the three selection criteria; with it, 0.98 at 69.0 ± 12.9 steps with 0.0 ± 0.3 violating steps, and 0.96 to 0.99 across the criteria. The first two rows of Table 5.6 are those numbers, the third the reproduction of the tightened one. The reason is the margin: a plan that clears the tightened boundary clears the real one under the tracking and discretisation error of execution; a plan projected onto the plain boundary has no such margin.

The reproduction uses the five training seeds and two episodes per seed and geometry that DPCC itself reports on (Subsubsection 5.6.3.1). A twenty-episode evaluation of two of the flow-based models at the budgets they are operated at exists and agrees with this protocol (Section B.5); the baseline’s own twenty-episode run did not complete and is not used.

Table 5.6.: The DPCC baseline on the obstacle-avoidance task: the published result without and with constraint tightening, and the reproduction of the tightened configuration in this thesis, on the metrics of Subsection 5.4.1. S&C denotes success with constraint satisfaction; time is seconds per control step. The spread on the reproduced row is the standard deviation over episodes, pooled over the five training seeds within a geometry and then averaged over the three geometries, so it carries both the seed-to-seed and the episode-to-episode variation.

Source and protocol	Steps	Success	S&C	Violating steps	s/step
Published, no tightening [RRS25, Table 1]	76.1 ± 21.5	0.96	0.49	6.0 ± 11.5	—
Published, tightened [RRS25, Table 1]	69.0 ± 12.9	1.00	0.98	0.0 ± 0.3	—
Reproduced, tightened, protocol of DPCC	70.1 ± 9.0	1.000	1.000	0.0 ± 0.0	0.553

5.5.2. Vision-Conditioned Alignment

No published result exists for the configuration evaluated here, so there is no external baseline to reproduce, as there is on D3IL-avoiding. The task itself is D3IL’s [Jia+24], which evaluates imitation-learning policies on it from images: two 96×96 RGB views, front and in-hand, each passed through its own ResNet-18 convolutional trunk with spatial-softmax pooling.

What D3IL scores as *success* on this task is a threshold on the final pose of the box, and both halves of it must hold at once: the box centre within 0.018m of the target centre *and* the box orientation within $0.048\pi \approx 8.6^\circ$ of the target orientation, the second of which is what makes the coloured sides line up. An episode that meets both counts as 1 and every other episode as 0; the published figure is the share of episodes counted 1.

That metric is not used here. The planner of this thesis is a different network with a different training target and output — a plan of positions that is projected and tracked, not an action applied directly (Table 5.7) — so the published success rate does not transfer to it. The alignment task is scored by the final distance of the box to its target (Subsection 5.4.2), and the D3IL numbers that follow describe how hard the task is.

From images the strongest policy is BESO, a score-based policy with a history of observations, at 0.359 ± 0.031 ; next are DDPM-GPT at 0.316 ± 0.026 and DDPM-ACT, which adds action chunking, at 0.278 ± 0.071 . From the state the same task is solved far more often: VAE-ACT reaches 0.891 ± 0.022 and BESO 0.861 ± 0.016 . On obstacle avoidance from the state BESO reaches 0.950 ± 0.015 [Jia+24, Table 3]. Seen from images, this is a task that the policies of D3IL finish in roughly a third of episodes, where

obstacle avoidance from the state (D3IL-avoiding) is close to solved.

Those policies are small, and most of what they carry is perception. The *head* — everything behind the image encoder — is a four-layer transformer at an embedding width of 72 for BESO and DDPM-GPT. For DDPM-ACT and VAE-ACT it is an encoder-decoder pair at a width of 64, two layers in the encoder and four in the decoder; VAE-ACT carries a third two-layer stack at the same width, the variational branch that produces its latent [Jia+24, Table 9].

Counted out at those shapes, with the four-fold feed-forward expansion the implementation uses, a width-72 block is 63.0 k parameters and the four-layer stack 0.25 M. A width-64 encoder block is 49.9 k and a decoder block 62.3 k — the difference is the three cross-attention projections — so DDPM-ACT’s two stacks come to 0.35 M and VAE-ACT’s three stacks, with its embeddings and output heads, to 0.47 M. None of the four heads reaches half a million parameters.

What carries the weight of an image-conditioned policy there is the encoder, and it is not close: each of the two camera streams has its own ResNet-18 trunk of roughly 11 M parameters, so the perception is about 22 M against a policy of under half a million — some fifty times the head it feeds. This thesis takes that encoder over unchanged (Subsection 4.4.6) and places the 4.0 M temporal U-Net of the state-based benchmark behind it, so the two set-ups differ in the planner and not in how the images are seen.

The two set-ups also differ in what the network produces and in what happens to it afterwards (Table 5.7), which is the second reason those numbers are a reference and not a baseline. The comparison for the flow-based models is therefore made inside the thesis: the diffusion model of DPCC, trained on the same demonstrations with the same visual encoder, backbone, horizon and projection, at $K = 20$.

Table 5.7.: The alignment task as evaluated in D3IL and in this thesis. The simulator, the demonstrations and the visual encoder are shared; the policy and the closed loop are not.

	D3IL [Jia+24]	This thesis
Network output	actions, executed directly	a plan of $H = 8$ steps of action and state
Network	MLP, GPT and action-chunking transformer heads, under 1 M parameters, behind two ResNet-18 encoders	the temporal U-Net of the state-based benchmark, 4.0 M parameters, conditioned on a 128-dimensional visual latent from the same two encoders (Subsection 4.4.6)
Closed loop	the policy’s action is applied	receding horizon: the plan is projected onto the constraint set, its first action is applied, and a new plan is generated at the next control step
Constraints	none	the constraint set of Table 5.3, enforced by the projection
Scoring	success rate and entropy of the behaviours	final distance of the box to its target, violations, time per control step (Subsection 5.4.2)
Reported result	a success rate: at best 0.359 ± 0.031 from images, 0.891 ± 0.022 from the state	no success rate is defined; an episode is scored by how close the box ends to its target

5.5.3. Quadrotor Benchmark

The quadrotor benchmark is new, and no learned planner has been evaluated on it before. Its closest precedent is the quadrotor task distributed with MuJoCo MPC [How+22], which flies the same Skydio X2 model [ZTM22]. That task is described here as it is distributed, because the benchmark of this thesis starts from its vehicle model and departs from it in everything else.

The original task. The quadrotor task distributed with MuJoCo MPC [How+22] is the following. The scene is a racing course of eight square gates, each a frame of four capsules with a 1.5 m opening, placed at different headings and heights around a closed loop. The vehicle starts on the floor. A goal marker leads it around the course: it sits at one of twelve fixed positions, between 0.75 and 2.25 m above the floor, placed so that flying from one to the next passes through a gate, and it advances to the next position whenever the vehicle comes within 0.5 m of it, returning to the first after the last. The gates do not appear in the objective; passing through them is a consequence of where the goal positions are placed.

The controller is Predictive Sampling, the sampling-based predictive controller of MuJoCo MPC (Subsection 4.3.6), and the simulator itself is its model. At each planning instant it perturbs its current control sequence into 32 candidate sequences of the four rotor thrusts, each parameterised by five spline points, rolls every candidate forward in MuJoCo over a horizon of 0.5 s at a time step of 0.01 s, and keeps the cheapest. The cost of a rollout sums four quadratic terms over the horizon: the distance of the vehicle from the current goal position, weighted 25; its linear velocity and its angular velocity, each weighted 1.25; and the deviation of each rotor thrust from the hover thrust, weighted 10^{-3} . The first control of the kept sequence is applied to the rotors, and planning repeats. Nothing is learned, no demonstrations are used, and no success rate is defined: the task is an interactive demonstration of the controller.

This benchmark. The benchmark of this thesis is built on the principle D3IL-avoiding and DPCC are built on [Jia+24; RRS25]: a policy is trained on demonstrations of a task, constraints the demonstrations do not satisfy are introduced at test time, and each run is scored both for finishing the task and for satisfying those constraints at every step. It keeps the vehicle model and the simulator of the original task and changes the three things that make it a test of constrained generative planning. The plan comes from a network trained on demonstrations instead of from sampling against the true dynamics; the environment is a set of walls, pillars and a test-time sloping halfspace that the plan must not violate, instead of gates that a goal sequence leads through; and the plan is a sequence of positions tracked by the cascaded geometric controller

(Section 4.3), instead of rotor thrusts applied directly. Each flight is scored for success and for constraint satisfaction (Subsection 5.4.3).

5.6. Training and Evaluation Protocol

Every model is trained once per training seed, and the same checkpoint is then evaluated under every step budget, projection method and selection rule; only the evaluation parameters vary between results. This section gives the machine everything ran on, then the training parameters, then the evaluation ones.

5.6.1. Compute Environment

All training and evaluation reported in this thesis ran on a single shared GPU node of the institute’s cluster, scheduled with SLURM. The node carries two 16-core AMD EPYC 7282 processors (32 physical cores, 64 hardware threads) and eight NVIDIA RTX A5000 accelerators with 24 GB of device memory each. **Every job was allocated exactly one GPU**: no result in this thesis comes from a distributed or multi-GPU run. The full hardware and software inventory, including package versions, is given in Appendix C, Table C.1.

The measurements of Section 5.4 were taken on this machine; what the measured times contain, and what they leave out, is discussed in Chapter 8.

5.6.2. Training

Table 5.8 lists the training parameters and what each controls. The same table covers all three environments; where a value differs between models it is given per model.

The generated plan has tensor shape (N, H, d) , with training batch size N and $H = 8$; its model output has the same shape. In D3IL-avoiding, the state anchor is $(N, 4)$ and the plan is $(N, 8, 6)$: a planar position increment and the commanded and measured planar positions. In D3IL-aligning, the state anchor is $(N, 6)$ and the plan is $(N, 8, 9)$: a spatial position increment and the commanded and measured spatial positions. Each of its two RGB cameras supplies one $(N, 1, 3, 96, 96)$ image tensor; the concatenated visual feature has shape $(N, 128)$. In the UAV scenes, the state anchor is $(N, 9)$ and the plan is $(N, 8, 12)$: a spatial position increment, commanded position, measured position and measured velocity. The embodiment layouts are introduced in Section 4.3. For the **B** entry the actual transport time is $1 - \text{Beta}(1.5, 1)$; for **LN** the recorded parameters are $P_{\text{mean}} = -0.4$ and $P_{\text{std}} = 1.0$. Their exact distributions are given in Section A.1.

The optimiser settings in that table are not uniform across the models, and the reason is the training target. The four objectives fit different quantities — a noise residual, an

5. Experimental Setup

Table 5.8.: Training parameters. Values are those recorded by the training runs of the reported models. The effective batch is the batch size times the number of gradient accumulation steps. “Diffusion (DPCC)” is the baseline of Table 5.6: on D3IL-avoiding it is the training configuration of DPCC, taken over unchanged; on the other two environments the same model and backbone are trained on that environment’s data, with the settings this table lists. The visual input is two RGB tensors, each $(N, 1, 3, 96, 96)$, encoded to a combined $(N, 128)$ feature; N is the training batch size.

Parameter	Meaning	D3IL-avoiding			D3IL-aligning	UAV, all scenes
		Diffusion (DPCC)	FM	MeanFM, CI-MeanFM	all models	all models
Plan horizon H	steps in one generated plan	8	8	8	8	8
Plan width d	action and state channels per step	6	6	6	9	12
Plan input/output	model tensor (N, H, d)	$(N, 8, 6)$	$(N, 8, 6)$	$(N, 8, 6)$	$(N, 8, 9)$	$(N, 8, 12)$
State input	current-state tensor	$(N, 4)$	$(N, 4)$	$(N, 4)$	$(N, 6)$	$(N, 9)$
Visual input	image streams; exact shape in caption	—	—	—	two RGB views	—
Training steps	optimiser updates	10^5	10^5	10^5	10^5	10^5
Batch size $\times$ accumulation	plans per update	8×2	8×2	32×2	64×2	8×2
Learning rate	Adam step size	10^{-4}	10^{-4}	$5 \cdot 10^{-4}$	$2 \cdot 10^{-4}$	10^{-4}
Weights evaluated	raw weights, or their exponential moving average (decay 0.995)	raw	raw	moving average	raw	raw
Training-time sampling	U: discrete uniform; B: reflected Beta; LN: logit-normal pair. See Section A.1	U	B	LN	by family: U, B, LN	by family: U, B, LN
Checkpoint	which saved weights are evaluated	lowest loss on the held-out 10 % of windows				

instantaneous velocity, and an average velocity over an interval — and a gradient of one is not a gradient of another: the scale of the loss, and how much it varies between two draws of the transport time, differ by construction. Each model was therefore given the batch size, step size and weight averaging under which its own loss curve converged, rather than a single setting imposed on all four. On D3IL-avoiding this makes the two average-velocity models the ones trained with four times the batch and five times the learning rate of instantaneous-velocity matching and the diffusion baseline, and the only ones evaluated with the exponential moving average of their weights; the logit-normal transport-time sampler in their column is the same kind of choice, made for the same reason.

What is held fixed is what the comparison rests on. All four models share the backbone, its parameter count, the dataset, the plan horizon and the projection they are read through, and each is evaluated at the step budgets stated with its results. What varies is how each objective was brought to convergence, which is a property of the training run and not of the comparison.

5.6.3. Evaluation

Table 5.9 lists the evaluation parameters, which are the quantities Chapter 6 varies or holds fixed. The subsections after it state, per environment, how many training seeds and how many episodes a result rests on. D3IL-avoiding is evaluated over five training seeds, so a result there carries the variation across seeds, and so is UAV-pillars, which flies its models; D3IL-aligning, UAV-corridor and UAV-s-curve are evaluated with one training seed, within the storage and compute available to this work.

Several of those parameters enter how a result should be read. The projector acts on planar position in D3IL-avoiding and UAV-pillars, and on spatial position in D3IL-aligning, UAV-corridor and UAV-s-curve; individual geometric predicates may use only some of those spatial coordinates.

The *step budget* counts the sampling steps of a plan, and it is not the same quantity in the two families: for the flow-based models it is the number of ODE solver steps, chosen at inference, and for the diffusion baseline the number of denoising steps, fixed at 20 when the model is trained. Without endpoint projection it is also the number of network evaluations per plan. Under endpoint projection each guiding step adds one further evaluation, the lookahead from the projected state, so a plan there costs $K + n_{\text{guide}}$ evaluations (Subsection 5.6.4); the tables name the sampling steps K throughout. The *selection rule* is one of the three of Subsection 4.5.5 — random, cumulative projection cost, or temporal consistency — and the *projection method* one of the three of Subsection 4.5.1; both vary per result and neither is ever averaged over. *Low-level control* means inverse kinematics followed by a joint-space PD controller on

5. Experimental Setup

Table 5.9.: Evaluation parameters, and the value each takes in each environment. A result that departs from the default states so. UAV-pillars flies the planner of D3IL-avoiding and takes that column's values, in the table's coordinates; only its low-level control is the quadrotor's.

Parameter	Meaning	D3IL-avoiding	D3IL-aligning	UAV-corridor, UAV-s-curve
Step budget K	sampling steps per plan	per result	per result	per result
Candidate plans B	plans generated in parallel per control step	4	4	4
Selection rule	which of the B plans is executed	per result	per result	per result
Projection method	none, per-step or endpoint	per result	per result	per result
Projected position	coordinates constrained in the plan	x, y (2D)	x, y, z (3D)	x, y, z (3D)
Activation threshold η	share of the sampling steps the projection runs on	0.5; 1.0 at the budget floor	lowered with the budget	0.5
Tightening δ	safety margin, in metres, that every constraint surface is shrunk by; the value DPCC introduces, and the only quantity the tightened variants change	0.025	0.025	0.025
Body inflation	separate offset, in metres, moving every surface outward so the <i>vehicle</i> clears it and not only its centre point; the manipulator's end effector is constrained as a point and carries none	none	none	0.31
Replanning	how often a new plan is generated	every step	every step	every step
Episode limit	control steps before a run is stopped	200	400	396 / 871
Low-level control	how a setpoint reaches the actuators	IK, then joint-space PD		cascaded geometric tracking control

the manipulator, and on the quadrotor the cascaded geometric tracker of Section 4.3, which MuJoCo MPC replaces in Subsubsection 6.3.3.3 and nowhere else.

An *episode* is one run of the task from its initial state to its end: the environment is reset, the planner replans at every control step and executes the first action of its plan, and the run ends when the goal is reached — the goal line crossed, the box delivered, the finish line of the scene flown through — or when the *episode limit* is reached. That limit is a hard cap on how long a single run may go on, and nothing else. A planner that has lost the task would otherwise replan forever, and every control step of the quadrotor scenes costs a projection solve, so a run that cannot finish is stopped and scored unsuccessful. The three limits differ because the tasks are of different lengths: the quadrotor limits are set at 1.2 times the longest demonstration of each scene, which is why the three scenes have three different values.

The *activation threshold* is not held at one value either. It selects the last ηK sampling steps, so the cost of a projected plan is the cost of one solve times that count, and at the budgets D3IL-aligning is run at the default of 0.5 would mean solving ten programs for every one of the four candidate plans at every one of up to 400 control steps. That is not affordable at the scale of this evaluation, and it is also not necessary: what endpoint projection needs is not a fixed *share* of the sampling steps but a fixed *number* of steps left to guide. The threshold is therefore lowered as the budget rises, so that the two alignment budgets are compared at the same number of guiding steps rather than at the same share. The one movement in the other direction — $\eta = 1.0$ on D3IL-avoiding — is the same argument at the opposite end: at the budget floor half of the sampling steps is one step, and one step is the terminal one, so the threshold is raised until the method has anything to act on at all. The two families select the same steps except when ηK is a whole number: the flow-based samplers project on the last $\lceil \eta K \rceil$ steps, while the diffusion baseline’s gate includes the step at the threshold itself and projects on one more — eleven rather than ten at $K = 20$ and $\eta = 0.5$. Subsection 5.6.4 works that arithmetic through.

The *tightening* margin is 0.025 m in all three environments, the value DPCC introduces. It shrinks the feasible set, so that a plan which satisfies a constraint numerically still has room left to be tracked, and it is the one quantity the tightened variants of a constraint set change.

Body inflation is a second offset and a different quantity, and only the quadrotor carries it. A constraint is written for a point, and the manipulator’s end effector is treated as one; a quadrotor is not a point, so every surface of the quadrotor scenes is moved outward before the projection ever sees it — by the vehicle’s rotor reach of 0.31 m. The configuration also defines a planning pad of 0.02 m on top of that reach, but every scene evaluated here switches the pad off for the planner, so the planner and the violation scorer are offset by the same 0.31 m. The 0.31 m is the largest per-axis

distance from the centre of mass to the rim of a rotor disk at zero yaw: the four rotors sit at $(\pm 0.14, \pm 0.18)$ m and each disk has a radius of 0.13 m, so the reach is $0.18 + 0.13 = 0.31$ m across the vehicle and $0.14 + 0.13 = 0.27$ m along it, and the larger of the two is taken. The two offsets are kept apart in the configuration and are kept apart here, because they answer different questions: 0.025 is margin, 0.31 is geometry.

Table 5.10.: What one reported result rests on, per environment. D3IL-avoiding follows the protocol of DPCC. Results are averaged per geometry first and then across geometries, so a geometry covered by fewer seeds does not carry more than its share.

Environment	Training seeds	Episodes per configuration	Averaged over
D3IL-avoiding	5	$5 \times 2 = 10$ per geometry	the three constraint geometries
D3IL-aligning	1	10 contexts $\times$ 1 episode	the ten contexts, compared context by context
UAV-pillars	5	$5 \times 2 = 10$ per geometry	the three constraint geometries, as on D3IL-avoiding
UAV-corridor	1	10 flights, spread over the routes L, C and R	the ten flights
UAV-s-curve	1	10 flights on its single route	the ten flights

5.6.3.1. Obstacle Avoidance

DPCC averages over “five training seeds and ten test seeds for each constraint set formulation” [RRS25, Sec. 6.1]; its released evaluation runs two episodes for each of five training seeds. This thesis uses those five seeds and two episodes per seed for its main comparison (Table 5.10). Every unmarked result in Section 6.1 carries all five seeds and all three geometries. The separate twenty-episode evaluation is reported in Section B.5.

The instantaneous-velocity-matching results were produced by the sampler inherited from the baseline, so that the initial noise, the candidate fan and the projection call are the baseline’s own (Section 4.4). That initial noise has standard deviation 0.5, as in the baseline, while the model was trained from noise of standard deviation 1; the two average-velocity models are sampled from noise of standard deviation 1, the scale they were trained at. The same holds on D3IL-aligning and on the quadrotor scenes.

5.6.3.2. Vision-Conditioned Alignment

The evaluation resets the environment to a fixed, enumerated set of contexts — the initial box and target poses — so every model and projection method starts from the same ten situations (Table 5.10). Because the contexts are shared, two configurations are compared context by context: for each context, which of the two leaves the box closer to its target. A result is the number of contexts in which one configuration is ahead, together with the median final distance over the ten.

5.6.3.3. Quadrotor Benchmark

Each scene is reported on its own. UAV-pillars is evaluated at the protocol of D3IL-avoiding — five training seeds, two episodes per constraint geometry, averaged per geometry and then over the three (Subsubsection 5.6.3.1) — for the configurations Chapter 6 selects there. The vehicle flies there at a fixed altitude of 1.0 m; each commanded position is tracked for one control step of 1 s, 100 physics steps of 0.01 s, with the reference moving towards it at no more than 1 m/s and without velocity feed-forward, and a flight that touches a pillar ends there. UAV-corridor is evaluated with ten flights per configuration, spread over its left, centre and right routes, on each of its two constraints separately; its projection constrains the commanded position in addition to the measured one and leaves the action unconstrained, a configuration the other two scenes do not use. UAV-s-curve is evaluated with ten flights per configuration on its single route: the flow-based models at one, two and twenty network evaluations and the diffusion baseline at its own twenty, without projection; the configuration that crosses the finish line most often, under the per-step projector of DPCC with each of its three selection rules, since at the one evaluation that configuration uses endpoint projection has no guiding step (Subsubsection 6.3.3.2); and its plans under the two tracking controllers. Every flight is tracked by the cascaded geometric controller of Subsection 4.3.5; the controller comparison of Subsubsection 6.3.3.3 replaces it by MuJoCo MPC and says so.

5.6.4. Guiding Steps of Endpoint Projection

Two of the evaluation parameters above interact, and their product decides whether a configuration can carry a statement about endpoint projection at all. One worked example is enough to read every projected result in Chapter 6.

Endpoint projection runs on the sampling steps the activation threshold selects, which are the last $\lceil \eta K \rceil$ of the K steps. Of those, the terminal step is not a guiding one: there the predicted endpoint *is* the finished sample, no later step is left for the network to react to, and what runs is a projection applied to a finished plan — which is what

per-step projection does at that step too. The steps that remain are the guiding ones:

$$n_{\text{guide}}(K, \eta) = \lceil \eta K \rceil - 1,$$

as described in Subsection 4.5.4.

Read it on the two settings that matter most in Chapter 6. On D3IL-aligning at $K = 20$ with $\eta = 0.2$, the projection runs on $\lceil 4 \rceil = 4$ sampling steps and three of them are guiding ones, so the reported endpoint-projection rows there are the method doing its own work. On D3IL-avoiding at the budget the benchmark is actually run at, $K = 2$ with the default $\eta = 0.5$, the projection runs on $\lceil 1 \rceil = 1$ sampling step and that step is the terminal one — so $n_{\text{guide}} = 0$, endpoint and per-step projection compute the same plan, and a difference between them there would be noise rather than a property of either method. The same arithmetic gives zero at $K = 1$ for every threshold below 1.0.

Where endpoint projection has guiding steps, its sampler is not the plain one on the two average-velocity models: at every step it queries the network at a zero interval, where the plain sampler queries it at the step width $1/K$. On those models a difference between endpoint and per-step projection therefore belongs to the sampler and the point of projection together.

Three rules follow from that, and they hold for every result in Chapter 6.

- A configuration with $n_{\text{guide}} = 0$ is reported separately and never averaged with configurations that have guiding steps.
- The count is not assumed. The evaluation records how many programs the projection solved per plan, and that number is compared against $\lceil \eta K \rceil$ before a result is reported.
- The threshold is stated with every projected result that departs from the default of Table 5.9.

Which configurations the first rule excludes, and how closely the recorded program counts follow the prediction, is reported with the results (Subsubsection 6.1.2.6).

6. Results

The results follow the order of the study: D3IL-avoiding, the benchmark of DPCC, first, then the two environments built for this thesis, D3IL-aligning and the UAV scenes. Section 6.4 brings the three together and answers, per environment, how the generative models and the projection methods compare. Each environment is reported in those two parts and closes with the supported model–projection combination. Each table states its backbone and its evaluation protocol.

6.1. Obstacle Avoidance

Every comparison in this section is made at the protocol of DPCC — five training seeds and two episodes per seed and geometry, ten episodes per geometry (Subsubsection 5.6.3.1). Results are averaged per geometry and then across the three geometries. At every control step the planner generates $B = 4$ candidate plans and executes one of them, and every model is shown under each of the three rules that make that choice (Subsection 4.5.5): random selection (r), cumulative projection cost (c) and temporal consistency (t). The models are then compared at their best rule.

The generative-model comparison first reads the raw outputs with projection switched off. All results with a projector in the loop are then reported together under projection methods, including the comparison of the four models through the fixed per-step projector of DPCC and the comparison between per-step and endpoint projection. The section closes with the supported model–projection combinations.

6.1.1. Generative Models

The four models of Section 5.5 are compared, on the shared temporal U-Net: FM, MeanFM, CI-MeanFM and the diffusion model of DPCC. One property of the step budget decides how the results below are read. K counts network evaluations in both families but is not the same quantity in them (Table 5.9): for the flow-based models it is the number of ODE solver steps, chosen at inference, and for the baseline the number of denoising steps, fixed when its noise schedule is discretised during training — so every diffusion budget is a separately trained checkpoint and every flow-based budget

is the same checkpoint read again. This subsection reads the plans executed without projection; projected results begin in Subsection 6.1.2.

The demonstrations teach the route to the goal. With projection switched off, each of the three flow-based models reaches it in all thirty episodes at both reported budgets; the diffusion model reaches it in 28 of thirty at one denoising step and 29 of thirty at twenty (Table 6.1). Reaching the goal is therefore distinct from satisfying the test-time constraints: no unprojected row exceeds 0.100 success with constraint satisfaction.

6.1.1.1. Raw Model Outputs

With the projector switched off, Table 6.1 measures goal arrival, constraint satisfaction and generation time on the same cells as Table 6.2.

Table 6.1.: D3IL-avoiding with the projection switched off: the cells of Table 6.2 at the same protocol, five training seeds $\times$ two episodes per geometry, averaged per geometry and then over the three; tightened constraints, four candidate plans, temporal U-Net of 4.0 M parameters. Success, S&C and violating steps per episode as in Subsection 5.4.1; each cell mean $\pm$ standard deviation over the seeds. No rule column: the evaluation records one unprojected arm per cell. The budgets $K = 5$ and 10 were evaluated as well and are not printed.
 • the baseline of record at its own budget.

Model	K	Success	S&C	Violating steps	Steps	ms/step
MeanFM	1	1.000 $\pm$ 0.000	0.033 $\pm$ 0.075	15.5 $\pm$ 2.1	63.4 $\pm$ 5.6	10.3 $\pm$ 0.4
MeanFM	2	1.000 $\pm$ 0.000	0.033 $\pm$ 0.075	15.5 $\pm$ 0.4	62.7 $\pm$ 1.8	19.3 $\pm$ 0.2
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	1	1.000 $\pm$ 0.000	0.033 $\pm$ 0.075	15.6 $\pm$ 0.7	60.5 $\pm$ 1.5	9.8 $\pm$ 0.2
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	2	1.000 $\pm$ 0.000	0.033 $\pm$ 0.075	15.7 $\pm$ 0.7	62.9 $\pm$ 1.2	18.7 $\pm$ 0.3
FM	1	1.000 $\pm$ 0.000	0.100 $\pm$ 0.091	16.5 $\pm$ 3.4	68.6 $\pm$ 5.9	9.0 $\pm$ 0.0
FM	2	1.000 $\pm$ 0.000	0.100 $\pm$ 0.091	15.6 $\pm$ 2.6	61.7 $\pm$ 2.2	17.8 $\pm$ 0.0
FM	20	1.000 $\pm$ 0.000	0.133 $\pm$ 0.075	16.5 $\pm$ 2.1	70.2 $\pm$ 6.3	183.5 $\pm$ 0.7
Diffusion	1	0.933 $\pm$ 0.149	0.000 $\pm$ 0.000	19.0 $\pm$ 2.9	68.3 $\pm$ 16.1	9.4 $\pm$ 0.1
Diffusion	2	1.000 $\pm$ 0.000	0.000 $\pm$ 0.000	20.4 $\pm$ 2.4	61.7 $\pm$ 8.3	18.3 $\pm$ 0.1
• Diffusion	20	0.967 $\pm$ 0.075	0.100 $\pm$ 0.091	17.9 $\pm$ 2.5	67.8 $\pm$ 16.8	179.3 $\pm$ 1.2

Read down the success column first. Every flow-based model reaches the goal in every one of its thirty episodes at every budget in the table, and the baseline in 0.933 of them at one denoising step and 0.967 at twenty. Producing a plan that arrives is not what separates these models on this benchmark.

Read the next column and none of them arrives legally. The highest success with

constraint satisfaction anywhere in the table is 0.133, instantaneous-velocity matching at twenty evaluations; the two average-velocity models read 0.033 at the budgets they are operated at, instantaneous-velocity matching 0.100, the baseline 0.100 at its own budget and 0.000 at one denoising step. Between 15.5 and 19.0 control steps of an average episode violate a constraint, and raising the budget does not work that number down: the intermediate budgets of five and ten, which the table does not print, hold it between 14.2 and 19.1, and the best and the worst of those counts both come at a budget of ten. The demonstrations all four models learn from cross the same constraint sets (Table 5.2), so a plan that respects them is not something any of them could have imitated. On this benchmark constraint satisfaction is the projector’s work, for the flow-based objectives exactly as for the diffusion baseline.

The last column prices generation alone. One network evaluation of a flow-based model costs 9.0 to 10.3 ms per control step and twenty denoising steps of the baseline 179.3 ms. Against the projected table the remainder is the projector: about 8 of the 18.3 ms analytic average-velocity matching takes at one network evaluation, and about 374 of the baseline’s 553.4 ms at its own twenty denoising steps. The projector runs on $\lceil \eta K \rceil$ sampling steps (Subsection 5.6.4), which is one at the flow-based operating budget and ten at the baseline’s, so the budget is paid twice — once in the network and again in the solver.

Those three columns score what the models produce. Figure 6.1 shows it. The raw plan is to a planning model what a generated image is to an image model: the fidelity of the sample itself, before any use is made of it [HJA20; Lip+23; Gen+25a], and the columns above are that fidelity scored on the task. The one thing to be clear about before reading the figure is that it is not the executed path — the executed path is not drawn in it at all. At every control step the policy returns a plan tensor of shape $(N, 8, 6)$ (Table 5.8): $N = 4$ candidate plans, each $H = 8$ waypoints long, each waypoint carrying a planar position increment together with the commanded and the measured planar position. What the figure draws, per candidate, is the *measured* planar position channels of those eight waypoints — one eight-point curve — and the green dot marks the first of the eight, the state the plan is anchored at. Drawing the fan at every control step would fill the panel with a solid block, so it is drawn at every fourth; the stored tensor keeps all of them.

Of the eight waypoints exactly one is ever carried out. The selection rule picks one of the four candidates (Subsection 4.5.5), its first action is applied as a Cartesian increment, and the other seven waypoints are thrown away and regenerated at the next control step, because the planner replans every step (Table 5.9). So the eight-step plan is never executed as drawn — and that is what makes it worth drawing. It is the network’s output before the projector repairs it and before the controller smooths it, the quantity that for an image generative model would be the fidelity of a sample. Every other

figure and table in this section reads the models through something downstream; this one reads them directly.

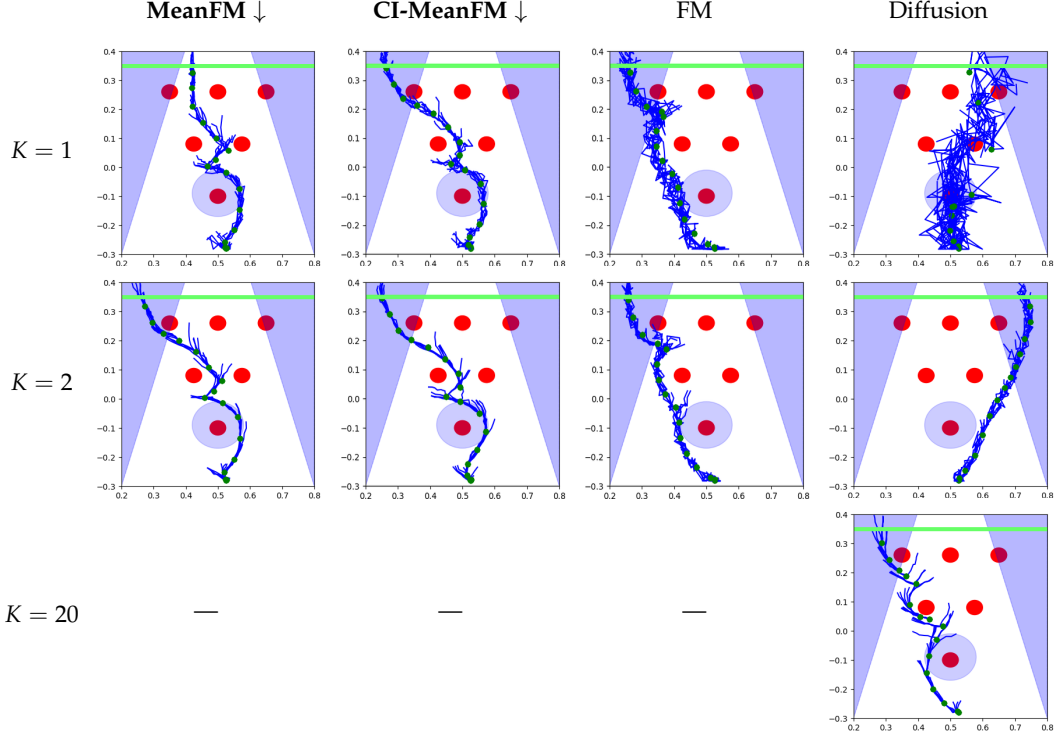


Figure 6.1.: Plans without projection, one episode per panel: the plans the model emitted at every fourth control step, all four candidates, each drawn as the measured planar position of its eight waypoints, green at the waypoint the plan is anchored at; the executed path is not drawn. Columns: MeanFM, CI-MeanFM, FM and the diffusion model of DPCC; rows: the step budget; *both-hard*, one training seed, temporal U-Net of 4.0M parameters. The arrows mark the two average-velocity columns; the last row is the baseline at its trained budget, where the flow-based models are not run (dashes). Table 6.1 aggregates the outcomes.

The four panels added to this figure were rendered by their evaluations at a different dashboard size from the three older ones, over the same data range. They are cropped so that the plot frame lands on the pixels the older panels already use, which is why the obstacles read as circles of one size across the whole matrix rather than as ellipses beside circles. The crop is declared in `DA_in_Paper/plotting/sources.py` and recorded per panel in `figures/MANIFEST.md`. The $K = 20$ panel comes from a second

evaluation campaign of the same checkpoint, scene, seed and geometry, run with a different projection threshold; this is the unprojected arm, so no projector ran in either campaign and the plans drawn are the model’s own.

Read that way, the models separate (Figure 6.1). At one network evaluation the successive plans of analytic average-velocity matching form a narrow ribbon: consecutive fans overlap, and each of the eight-point curves is a bounded arc towards the goal. The diffusion baseline at the same budget emits plans that jump across the workspace from one control step to the next.

That difference does not appear in the motion, and it is worth saying why before the panels are used for anything. Two things sit between a plan and the arm (Section 5.1): only the *first* action of the plan is executed, and that action is a Cartesian increment reaching the joints through inverse kinematics and a joint-space PD controller. A path assembled one first-action at a time, through an integrator, is smooth whether or not the plans behind it were. The panels are therefore the only place the incoherence is visible at all. The analytic average-velocity matching panel at two evaluations shows the ribbon surviving a second evaluation.

The two remaining objectives, in the middle two columns, fall between those extremes and in the order the projected results below give. Instantaneous-velocity matching throws a wide, jagged fan at both budgets, and much of its length lies inside the forbidden halfspace on the left of the panel rather than in the free corridor. Consistency-interpolated average-velocity matching is noisy at one evaluation and markedly tighter at two, where its plans thread the obstacle field in a band comparable to the analytic objective’s. Neither is as tight as analytic average-velocity matching at the same budget, and all three are far from the near-scribble of the baseline at one evaluation.

The baseline’s small-budget panels are two models, one trained at each budget. At $K = 2$ its plans form a single coherent bundle from start to goal, and that bundle runs along the wrong side of the constraint. At its own twenty steps, in the last row, the baseline draws a single tight bundle from the start state to the goal line, the upper half of which lies inside the left halfspace.

In the cell these panels are drawn from — one training seed, *both-hard*, two episodes, no projection — the violating control steps per episode are 6.0 for analytic average-velocity matching at one evaluation and 12.0 at two, and for the diffusion baseline 28.0 at one, 21.5 at two and 18.5 at twenty.

Each panel is one episode. The panels share the geometry and the training seed and come from separate evaluations. A flow-based column is one checkpoint read at both budgets; the three diffusion panels are three separately trained models, one per budget, since a diffusion model’s step count is fixed at training. The last row is the baseline at twenty denoising steps; the flow-based models are operated at one and two evaluations and have no panel there.

6.1.2. Projection Methods

The remaining results all include projection. The first comparison holds the per-step projection of DPCC fixed and varies the generative model. The later comparison holds the generative model fixed and varies the projection method.

6.1.2.1. At the Protocol of DPCC

This pass puts the projector back and holds it fixed at the per-step projection of DPCC, so that the generative model is the only thing that varies; the two projection methods are compared against each other in Subsubsection 6.1.2.5. Every number from here to the end of the subsection is an executed-episode result of that pipeline.

Two marks recur in this table and in the ones after it. The **operating point**, ►, is the configuration this thesis carries forward on the benchmark — the cheapest one that holds its success, not the highest number in a column. The **baseline of record**, ●, is the diffusion model of DPCC at its own budget of twenty under its own best selection rule, and it is what every claim of this chapter is measured against. Every row of the table carries all five training seeds and all three geometries: the budgets are $K \in \{1, 2, 5, 10\}$ for analytic average-velocity matching, $\{1, 2\}$ for the consistency-interpolated model, $\{1, 2, 20\}$ for instantaneous-velocity matching and $\{1, 2, 10, 20\}$ for the diffusion baseline, whose step count is fixed at training.

The baseline reproduces its published result (Table 6.2, Table 5.6): under cumulative projection cost it reaches a success rate with constraint satisfaction of 1.000 in 70.1 control steps, at 553.4 ms per control step. Temporal consistency reaches the same rate in more steps and random selection falls short of it.

At two denoising steps under cumulative projection cost the baseline also reaches 1.000, in 61.2 control steps at 211.4 ms, on five seeds and three geometries: the same constraint satisfaction in fewer control steps at 38 % of the time. At ten steps under temporal consistency it reaches 1.000 in 68.7 control steps at 321.7 ms. The baseline of record is the configuration DPCC reports, at twenty steps, and the cost factors below are stated against it; against the two-step diffusion configuration the $30\times$ below is $12\times$.

That two-step time is six times the one-step time for twice the denoising steps, and where the difference comes from is set out in Subsubsection 6.1.2.4.

Two flow-based models clear that row at a single network evaluation, on all three of its axes at once. Instantaneous-velocity matching reaches 1.000 in 67.0 control steps and consistency-interpolated average-velocity matching 1.000 in 59.2, the fewest in the table, each at about 18 ms per control step — a thirtieth of the baseline’s, on the same temporal U-Net of 4.0 M parameters. Neither gives up success with constraint satisfaction, and both finish in fewer control steps. This is the one place in the thesis

where a flow-based model is ahead of the baseline on every reported axis at once rather than trading one against another.

Analytic average-velocity matching, the model this benchmark is operated with, reaches 0.967 in 58.6 control steps at 18.3 ms — against the baseline’s best rule, one episode of thirty fewer reaching the goal within the constraints, 11.5 fewer control steps, and a thirtieth of the time. What moves that result is the selection rule: under cumulative projection cost the same model drops to 0.933 and stalls at 72.4 or 97.2 control steps, the behaviour described in Subsubsection 6.1.2.2. The three models differ in how exposed they are to that choice. Instantaneous-velocity matching is the only one that holds 1.000 under all three rules at both budgets; consistency-interpolated average-velocity matching holds it under temporal consistency at both, but falls to 0.900 under random selection at two evaluations and stalls under cumulative projection cost in the same way.

6.1.2.2. Candidate Plans and Selection Rules

With a single candidate plan the three selection rules return the same plan: success and control steps are identical in all fifteen paired seed–geometry combinations, for the diffusion baseline, analytic average-velocity matching, instantaneous-velocity matching and consistency-interpolated average-velocity matching. With four candidates, generation time changes by at most six per cent, because the four plans are one batched network evaluation, while projection time grows by a factor of 3.2 to 4.9, because the programs are solved one after another on the CPU. Solving them in parallel would bring analytic average-velocity matching at $K = 1$ from 18.1 to about 11.7 ms per control step with all four candidates kept; this is a prediction from two measured points.

Cumulative projection cost slows analytic average-velocity matching down on this task. In Table 6.2 it takes 72.4 control steps at $K = 1$ and 97.2 at $K = 2$, against 58.6 and 59.4 under temporal consistency, and reaches 0.933 success with constraint satisfaction against 0.967: the rule occasionally selects a plan that stalls, and a stalled plan costs steps without buying constraint satisfaction. The same happens to consistency-interpolated average-velocity matching, which goes from 60.1 steps under temporal consistency to 89.7 under cumulative projection cost at $K = 2$, and it does not happen to the diffusion baseline, for which cumulative projection cost is the best of the three. Temporal consistency is therefore the selection rule the flow-based models are operated with on this task.

6. Results

Table 6.2.: D3IL-avoiding at the protocol of DPCC: five training seeds $\times$ two episodes per geometry, averaged per geometry and then over the three; tightened constraints, four candidate plans, temporal U-Net of 4.0 M parameters. Rules: r random, c cumulative projection cost, t temporal consistency. Columns as in Subsection 5.4.1, each mean $\pm$ standard deviation over the seeds. The budgets $K = 5$ and 10 were evaluated as well and are not printed. $\blacktriangleright$ the operating point, $\bullet$ the baseline of record, bold the cells that clear it on all three columns.

Model	K	Rule	S&C	Steps	ms/step
MeanFM	1	r	0.900 ± 0.149	64.3 ± 2.6	19.2 ± 1.6
		c	0.933 ± 0.091	72.4 ± 8.4	18.3 ± 0.5
		$\blacktriangleright t$	0.967 ± 0.075	58.6 ± 2.2	18.3 ± 0.2
MeanFM	2	r	0.967 ± 0.075	65.1 ± 2.6	27.6 ± 0.5
		c	0.933 ± 0.091	97.2 ± 4.8	27.1 ± 0.2
		t	0.967 ± 0.075	59.4 ± 2.0	27.7 ± 0.5
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	1	r	1.000 ± 0.000	61.5 ± 0.8	17.9 ± 0.2
		c	0.967 ± 0.075	68.4 ± 2.3	18.0 ± 0.6
		t	1.000 ± 0.000	59.2 ± 1.2	18.1 ± 0.2
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	2	r	0.900 ± 0.149	63.3 ± 1.3	27.0 ± 0.3
		c	0.933 ± 0.091	89.7 ± 8.2	26.6 ± 0.2
		t	1.000 ± 0.000	60.1 ± 3.4	27.0 ± 0.2
FM	1	r	1.000 ± 0.000	67.9 ± 1.2	17.4 ± 0.1
		c	1.000 ± 0.000	67.0 ± 0.8	17.3 ± 0.1
		t	1.000 ± 0.000	68.2 ± 1.7	17.7 ± 0.6
FM	2	r	1.000 ± 0.000	65.7 ± 0.8	26.2 ± 0.9
		c	1.000 ± 0.000	67.0 ± 3.8	25.5 ± 0.2
		t	1.000 ± 0.000	63.8 ± 1.9	25.9 ± 0.2
FM	20	r	0.867 ± 0.075	73.3 ± 7.9	852.6 ± 624.9
		c	1.000 ± 0.000	63.2 ± 1.5	476.7 ± 26.0
		t	1.000 ± 0.000	63.7 ± 2.2	486.5 ± 41.4
Diffusion	1	r	0.600 ± 0.303	67.7 ± 10.1	33.9 ± 8.6
		c	0.667 ± 0.391	72.1 ± 17.6	33.3 ± 14.5
		t	0.600 ± 0.190	73.5 ± 14.1	33.3 ± 13.2
Diffusion	2	r	1.000 ± 0.000	65.1 ± 1.4	191.1 ± 26.5
		c	1.000 ± 0.000	61.2 ± 1.2	211.4 ± 18.8
		t	0.933 ± 0.091	60.9 ± 1.5	225.9 ± 48.5
Diffusion	20	r	0.967 ± 0.075	74.7 ± 5.8	573.2 ± 22.2
		$\bullet c$	1.000 ± 0.000	70.1 ± 8.2	553.4 ± 28.0
		t	1.000 ± 0.000	76.1 ± 15.7	563.0 ± 25.4

6.1.2.3. The Cost Frontier

At equal success, two costs remain: how many control steps an episode takes, and how long one action takes to compute. They are not exchangeable — a plan that arrives in fewer steps but costs half a second per step is not obviously better than the reverse — so this section does not add them into one score. It places every configuration on both axes at once and asks which of them no other configuration improves on in both (Figure 6.2).

A configuration is *non-dominated* when no other configuration in the comparison is at the same time faster per action and shorter in control steps. Only configurations that keep the task solved take part: a point enters the comparison when its success with constraint satisfaction is within 0.05 of the best value in its panel, so that a fast configuration that reaches the goal less reliably cannot appear as a favourable trade. Reading the figure, colour identifies the model, marker shape the selection rule, the number beside a point its step budget, and a ring marks each non-dominated point; down and to the left is better on both axes, which the boxed arrow in the legend repeats. The step axis is broken at 80: every configuration that solves the task lies below that line, and the few above it — the hollow $K = 2$ points and one diffusion cell — are drawn on a compressed scale so that the solved band is not squeezed into a third of the panel. The figure and Table 6.2 draw on the same cells: every budget plotted here is a row of that table, on five training seeds and three geometries, under both selection rules, and the frontier is taken over all of them, so the ringed point of a model is that model at its better rule.

The horizontal axis is planner time, not the full executed-step time. Full executed-step timing is available for the separate UAV controller comparison in Table 6.15; it was not recorded for the matched avoiding configurations plotted here.

The frontier is occupied by the flow-based models at their smallest budgets, and the three of them at the lower left reach the baseline’s control-step count at one twentieth to one thirtieth of its time per action. Consistency-interpolated average-velocity matching at one network evaluation sits at the lower left, 59.2 control steps at 18.1 ms, with analytic average-velocity matching beside it at 58.6 and 18.3; no configuration of any model is both faster and shorter than either. Instantaneous-velocity matching at one and two evaluations lies a little longer in steps at a similar time. Every diffusion configuration lies to the right by more than an order of magnitude in time, and the baseline’s own operating point, twenty evaluations with cumulative projection cost, is neither the fastest nor the shortest point in the figure. What the frontier shows is therefore not a single best configuration but a region — one or two network evaluations of a flow-based model — inside which the remaining choices trade a control step against a millisecond.

6. Results

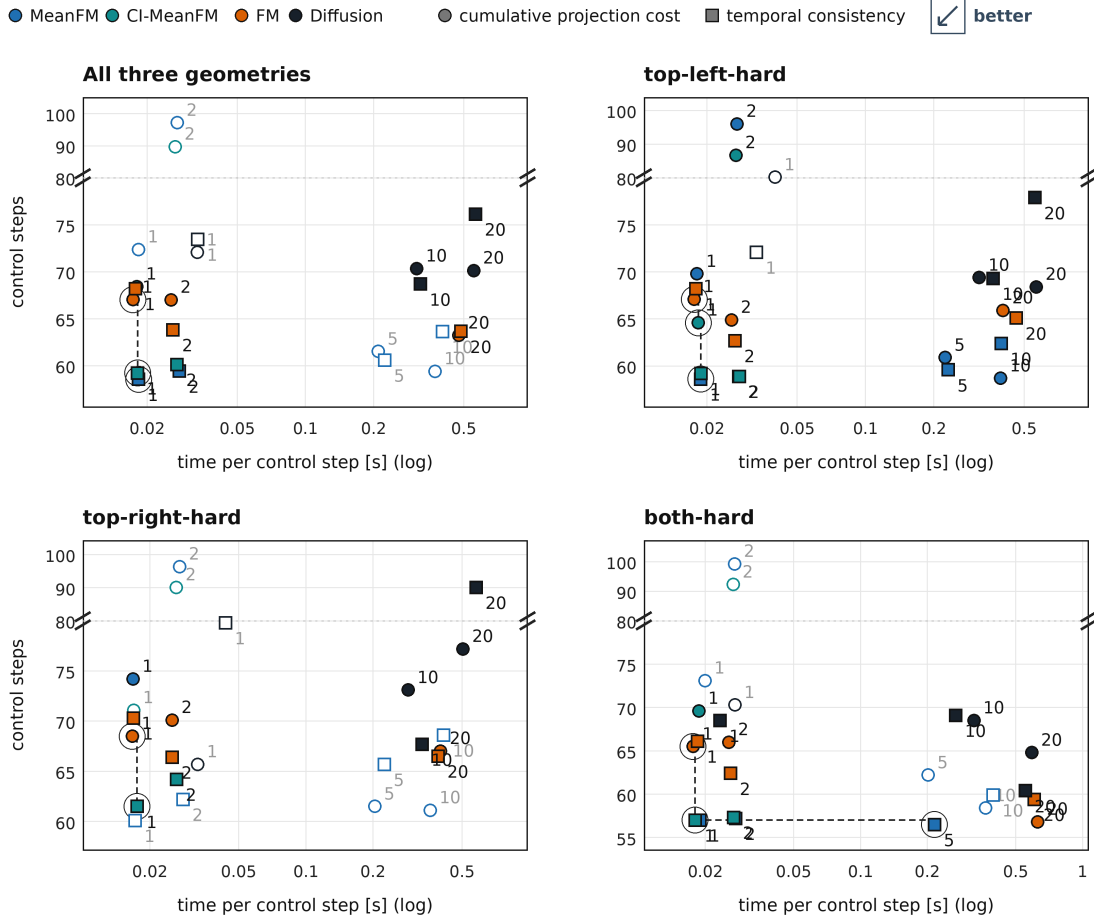


Figure 6.2.: Control steps against the time to compute one action (generation, projection and selection), every step budget of Table 6.2 under both selection rules: circles cumulative projection cost, triangles temporal consistency; colour the model, the number beside a point its step budget, a ring a non-dominated point, the boxed arrow the direction in which both axes improve. Three panels are the constraint geometries; the fourth averages within a geometry and then across the three. The step axis is broken at 80. Five training seeds $\times$ two episodes per geometry, tightened constraints, temporal U-Net of 4.0 M parameters.

6.1.2.4. Caveat: What the Projector Is Handed

One number in Table 6.2 does not scale the way the others do, and it says something about the two sampler families that the table itself cannot. The diffusion baseline’s own cost doubles from one denoising step to two (9.4 to 18.3 ms, Table 6.1), as it should; its projected cost rises six-fold, from 33 to 191–226 ms. The projection is the same program, on the same four candidate plans, against the same tightened constraints.

What changes is the state the projector is handed. The baseline projects at every denoising step with $t \leq 0.5 K$, which at two steps is both of them: the final step, as at every budget, and the first — a plan one denoising step away from Gaussian noise. No larger budget hands the projector such a plan, because there the gated steps begin in the middle of the schedule; and the flow-based models never do, because their projection window is the last $\lceil 0.5 K \rceil$ solver steps, which at two steps is the terminal one alone. Projecting the final step costs what it costs everywhere: the post-processing variant of the same cell, which projects nothing but the final step, runs at 28 ms per control step, 10 ms above the unprojected plan. The remaining 180–190 ms is the first-step projection, about 45 ms per candidate against 2–6 ms for a plan that is nearly denoised. The baseline’s projector is a sequential quadratic program solved iteratively, and its cost is set by how far from feasible and how rough the plan it starts from is: a plan that is still mostly noise violates every constraint at once and the dynamics rows between every pair of steps, and the solver spends its iterations there.

Two evaluations on different days give the same time on all five seeds. At equal noise level the two families cost about the same per solve: at $K = 20$ the diffusion baseline projects eleven steps from the middle of the schedule at about 8.5 ms per solve, and instantaneous-velocity matching ten at about 7 ms. Whether a flow-based model’s first-step plan — a point on the straight path from noise to data — projects more cheaply than the diffusion model’s at the same step is not measured here: at two steps the flow-based window excludes that step. The measurement that would show it is the same cell with the projection window opened to both steps on the flow-based side. The ordering of this benchmark does not rest on this cell: every flow-based row is cheaper than every diffusion row at equal success with constraint satisfaction, and the operating points are at one evaluation, where both families project a single, final step.

6.1.2.5. Endpoint and Per-Step Projection

The models above were read twice, unprojected and under the per-step projector. This subsection holds the model fixed and varies the projector, and it is made on matched work: every comparison gives both methods four candidate plans and reads them on the same training seed, the same three geometries and the same episodes, so that

the two projectors are priced on the same thing. It is read at two budgets. $K = 3$ is the smallest budget at which endpoint projection has a guiding step (Subsubsection 6.1.2.6), and therefore the smallest at which the two methods differ at all; at $K = 10$ both have several. On the two average-velocity models the endpoint sampler also queries the network at a zero interval where the plain sampler queries it at the step width (Subsection 5.6.4), so on those models a difference between the rows belongs to the sampler and the point of projection together. Table 6.3 prints every model at both budgets. No endpoint-projection evaluation of the diffusion baseline exists on D3IL-avoiding, so no statement in this subsection extends to it. Recorded in `data_status/PENDING_20260922_all_lacking_runs.md`.

Table 6.3.: Endpoint projection [LAA25] against per-step projection on D3IL-avoiding, both with **four candidate plans**, tightened constraints, the same seed, geometries and episodes on both sides of every pair. Both under temporal consistency (t) unless marked. At $K = 3$ endpoint projection ran at activation threshold 1.0 (two guiding steps), at $K = 10$ at 0.5 (four). S&C, control steps and the time to compute one action are means over three geometries $\times$ two episodes and over the training seeds listed, except CI-MeanFM at $K = 10$, whose cell holds twenty episodes per geometry. Bold: the better of the pair. Temporal U-Net of 4.0 M parameters.

Model	K	Seeds	Projection	S&C	Steps	ms/step
MeanFM	3	4	per-step, t endpoint	0.917 1.000	59.3 58.9	148 74.5
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	4	per-step, t endpoint	0.958 1.000	60.0 60.0	146 76.1
FM	3	4	per-step, t endpoint	1.000 1.000	61.2 60.8	64.1 71.5
MeanFM	10	1	per-step, t endpoint	1.000 1.000	64.2 64.8	362 176
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	10	1	per-step, t endpoint	0.983 1.000	60.5 60.7	386 176
FM	10	1	per-step, c endpoint	1.000 1.000	63.2 64.0	199 167

At $K = 3$, on the same four training seeds for every model, endpoint projection reaches 1.000 success with constraint satisfaction on all three. On the two average-velocity models it is ahead on the constraint — 1.000 against 0.917 for analytic average-velocity matching and against 0.958 for the consistency-interpolated model — at level control steps and about half the time per step, 74.5 against 148 ms and 76.1 against 146 ms, so there it Pareto-dominates per-step projection. The factor of two is the

projector: the predicted endpoint is close to the feasible set at every step, so the single solve needs few iterations, where the per-step projector solves one program per candidate at each active step. On instantaneous-velocity matching both projectors reach 1.000 and per-step projection is the cheaper, 64.1 against 71.5 ms, with endpoint projection 0.4 control steps shorter: a trade-off. The per-step time differs between the models' evaluations (64 against 146–148 ms), so the table is read within a model.

That result does not move the operating point of this task, because $K = 3$ is not on its cost frontier. Figure 6.2 and Table 6.2 place the frontier at one and two network evaluations: per-step projection at $K = 1$ already reaches 0.967 and 1.000 for the two average-velocity models at about 18 ms per control step, on five seeds. Every $K = 3$ point of Table 6.3 sits three to eight times to the right of that at the same success, so each is dominated by a cheaper configuration that solves the task as well. The budget of three is therefore dropped from this benchmark's operating point, and with it the one budget at which endpoint projection would be the better projector at a price that matters.

At $K = 10$, where the comparison is at the same budget on one training seed, endpoint projection is level or ahead on the constraint and cheaper on all three models. On analytic average-velocity matching both reach 1.000, at 176 against 362 ms; on the consistency-interpolated model endpoint projection reaches 1.000 at 176 ms against 0.983 at 386 ms; on instantaneous-velocity matching both reach 1.000, at 167 against 199 ms. Under the other two rules the picture is mixed. Under random selection analytic average-velocity matching reaches 0.500 with per-step projection and 1.000 with endpoint projection, the largest gap in the table, while the consistency-interpolated model scores 0.833 and 0.883 with endpoint projection against 0.850 and 0.900 with per-step projection, level within one episode. The cumulative-cost rule lengthens the endpoint paths, to 88 to 111 control steps, because with four candidates it can select a stalled plan. The $K = 10$ rows are one training seed: six episodes per rule for analytic average-velocity and instantaneous-velocity matching, sixty for the consistency-interpolated model, whose cell was evaluated with twenty episodes per geometry. The $K = 3$ rows are four training seeds. Nothing here is read at the protocol of DPCC, and nothing here changes a selection made in Subsubsection 6.1.2.1.

The consequence for this task is that endpoint projection is not needed here, and the reason is the budget rather than the method. At one network evaluation the two average-velocity models already reach 0.967 and 1.000 at about 18 ms per control step with the per-step projector, and instantaneous-velocity matching reaches 1.000 at one and at two (Table 6.2); there is no shortfall left for a second projection method to close. Endpoint projection needs sampling steps to guide, and at one and two evaluations it has none or one (Subsection 4.5.4, Subsubsection 6.1.2.6); a run at $K = 2$ with both methods given the same candidate plans produced bit-identical rollouts at 2.45 times

the cost. Where it does act, at three and at ten evaluations, it is level or ahead on the constraint at the same budget, and cheaper on every model except instantaneous-velocity matching at three evaluations — and that is the result carried to the tasks that are run at those budgets, D3IL-aligning and the quadrotor scenes.

On D3IL-avoiding endpoint projection therefore has no task to win: the per-step projector at one or two evaluations already reaches 0.967–1.000 at about 18 ms, and the budget of three, the first at which endpoint projection can act, buys no fewer control steps at three to eight times the time, so it is dominated in the sense of Subsubsection 6.1.2.3. On D3IL-aligning, which needs twenty evaluations, endpoint projection keeps more contexts violation-free than per-step projection (Subsection 6.2.2).

6.1.2.6. Steps at Which Projection Guides Sampling

The guiding step count is defined in Subsection 4.5.4 and the convention for reporting a configuration that has none in Subsection 5.6.4. This section reports what the evaluations measured against it.

The count holds. With the activation threshold at 1.0 the evaluation ran with one guiding step at $K = 2$, two at $K = 3$ and four at $K = 5$, and the number of programs it actually solved grew as 1 : 1.51 : 2.52 against the predicted 1 : 1.5 : 2.5, so the configurations reported below are the ones the prediction says they are.

Two settings in this chapter have no guiding step, and neither supports a statement about endpoint projection. A budget of $K = 1$ has none at any threshold, which is why D3IL-avoiding at its operating budget is reported with per-step projection only and why UAV-corridor has no endpoint row at that budget; at the default threshold of 0.5, $K = 2$ has none either, which is where the bit-identical rollouts of Subsubsection 6.1.2.5 come from and why Table 6.3 starts at $K = 3$.

6.1.3. Conclusion

D3IL-avoiding carries the central claim of this thesis, and the results support it: the average-velocity models Pareto-dominate the diffusion model of DPCC on its own benchmark, at its protocol and with its own temporal U-Net of 4.0 M parameters (Table 6.4). CI-MeanFM at one network evaluation, with temporal consistency and per-step projection, reaches 1.000 success with constraint satisfaction as the baseline of record does, in 59.2 control steps against 70.1 and at 18.1 ms per step against 553.4, a thirtieth of the time. Analytic average-velocity matching at the same budget and projector reaches 0.967 in 58.6 steps at 18.3 ms, one episode in thirty for the same saving.

Instantaneous-velocity matching at one evaluation dominates the baseline as well, at 1.000 in 67.0 control steps and 17.3 ms, so much of the saving belongs to a flow-based

sampler at a small budget; the average-velocity models add to it the fewest control steps of the benchmark, about eight fewer than instantaneous-velocity matching for about a millisecond more per step. The saving does not rest on the baseline’s twenty steps. Retrained at two denoising steps, the baseline also reaches 1.000, in 61.2 control steps at 211.4ms, and against that configuration the flow-based models are twelve times faster rather than thirty. Its two-step time is six times its one-step time for twice the denoising steps, because of what its projector is handed: at two steps it projects a plan one denoising step away from noise, which costs about 45 ms per candidate to repair against 2 to 6 ms for a nearly denoised plan, while the flow-based models at one or two evaluations project only their terminal step. What the frontier shows is therefore a region, not a single point: one or two evaluations of a flow-based model under temporal consistency, inside which the remaining choices trade a control step against a millisecond. Cumulative projection cost, the baseline’s best rule, is not theirs: it occasionally selects a stalled plan and costs them control steps.

Table 6.4.: The supported model–projection combinations on D3IL-avoiding and the baseline of record, at the protocol of DPCC. All use tightened constraints and the temporal U-Net of 4.0M parameters. The episodes column states the number per geometry; every row uses five training seeds and all three geometries.

Model	K	Projection	Rule	Episodes	S&C	Steps	ms/step
► MeanFM	1	per-step	t	10	0.967	58.6	18.3
CI-MeanFM	1	per-step	t	10	1.000	59.2	18.1
• Diffusion	20	per-step	c	10	1.000	70.1	553.4

Endpoint projection is not needed at either low-budget operating point. At one evaluation it has no guiding step, and at two it has only one; raising the budget so that it can guide sampling gives up the low-step advantage that decides this benchmark. Where it can act, at three evaluations, it halves the time per control step of the two average-velocity models at a success with constraint satisfaction of 1.000, because the endpoint it projects is already close to the feasible set: the other face of the two-step baseline’s cost, since the price of a projection is set by how far from feasible the plan it is handed is. Those budgets lie off the cost frontier, so no endpoint-projection configuration improves on the operating point at one evaluation with per-step projection: on its own benchmark the operating point leads both the diffusion model of DPCC and endpoint projection. D3IL-aligning and the UAV scenes below test endpoint projection at larger budgets.

The appendix carries more of this benchmark: the executed end-effector paths of the

two average-velocity models at their operated budgets (Section B.1) and the evaluation at twenty episodes per seed (Section B.5).

6.2. Vision-Conditioned Alignment

All results in this section use one training seed and the same ten contexts, and every comparison is made context by context (Subsubsection 5.6.3.2). The box starts on average 0.4530 m from its target. The generative models are compared on the final box-to-target distance of plans executed without projection, on which the constraint set has no bearing — the plan is the same plan — so all four models are compared at once; constraint violations and the time one action costs are compared in Subsection 6.2.2. The ten contexts are drawn from the training contexts of the D3IL task, not from its held-out set, so these results measure how well each model plans for initial poses of the kind it was trained on.

6.2.1. Generative Models

One model moves the box to its target on this task and the other three barely move it at all (Table 6.5). The median is the column to read there: the spread over the ten contexts is of the order of the mean itself, because a context in which the box is not touched contributes the whole starting distance and one in which it is delivered contributes almost nothing. Analytic average-velocity matching ends a median 0.0741 m from the target, from a starting distance of 0.4530 m — it closes 84 % of the distance — and it is the only model that moves the box in every context. The other three end between 0.39 and 0.46 m. The two flow-based models close 10–14 % of the distance, while the diffusion median is 2 % farther away than the box started; each leaves it untouched in some contexts.

The margins behind that ordering are one-sided. Analytic average-velocity matching ends closer than the diffusion baseline in all ten contexts, by 0.3744 m on average, at 0.64 of the baseline’s time per control step, and closer than instantaneous-velocity matching in nine of ten, by 0.2333 m. Between instantaneous-velocity matching and the baseline there is no ordering: their medians are 0.4085 and 0.4619 m, and each ends closer in about half the contexts. Instantaneous-velocity matching leaves the box unmoved in four of ten contexts and the diffusion baseline in five of ten.

The ordering rests on analytic average-velocity matching’s margins, which hold in nine and ten of ten contexts; the level result for instantaneous-velocity matching and diffusion is the absence of a difference at ten contexts, not evidence that they are equal.

Table 6.5.: The generative models on D3IL-aligning without projection: the three flow-based models at $K \in \{2, 10, 20, 100\}$ and the diffusion baseline at its trained budget, printed last. Final box-to-target distance over the ten contexts from an initial 0.4530 m, in brackets the share of that distance closed (Subsection 5.4.2); *unmoved*: contexts in which the box does not move; mean and time as mean $\pm$ sample standard deviation over the contexts. Temporal U-Net of 4.0M parameters, FiLM conditioning, one training seed. ► the selected configuration; • the diffusion model of DPCC.

Model	K	Median [m]	Mean [m]	Unmoved	ms/step
MeanFM	2	0.2779 (39 %)	0.368 ± 0.301	0/10	28.4 ± 2.0
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	2	0.3738 (17 %)	0.357 ± 0.201	2/10	28.7 ± 5.7
CI-MeanFM, $\alpha_{\text{end}} = 0.05$	2	0.3978 (12 %)	0.336 ± 0.166	0/10	27.6 ± 2.0
FM	2	0.4257 (6 %)	0.375 ± 0.133	3/10	32.5 ± 2.5
MeanFM	10	0.1194 (74 %)	0.173 ± 0.153	1/10	99.2 ± 1.9
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	10	0.4247 (6 %)	0.399 ± 0.094	2/10	98.5 ± 4.0
FM	10	0.4224 (7 %)	0.415 ± 0.085	4/10	145.1 ± 2.0
► MeanFM	20	0.0741 (84 %)	0.093 ± 0.073	0/10	190.5 ± 2.0
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	20	0.3897 (14 %)	0.325 ± 0.224	2/10	189.1 ± 5.6
CI-MeanFM, $\alpha_{\text{end}} = 0.05$	20	0.4488 (1 %)	0.436 ± 0.283	0/10	187.3 ± 2.0
FM	20	0.4085 (10 %)	0.327 ± 0.160	4/10	295.8 ± 1.9
MeanFM	100	0.0673 (85 %)	0.136 ± 0.148	0/10	892.4 ± 1.8
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	100	0.1791 (60 %)	0.327 ± 0.323	1/10	902.2 ± 1.5
FM	100	0.4247 (6 %)	0.352 ± 0.180	0/10	1425.0 ± 1.8
• Diffusion	20	0.4619 (−2 %)	0.468 ± 0.399	5/10	298.3 ± 2.0

6.2.1.1. Step Budget and the Cost Frontier

Analytic average-velocity matching improves with the budget: its median final distance falls from 0.2779 m at $K = 2$ to 0.1194 m at $K = 10$, 0.0741 m at $K = 20$, and 0.0673 m at $K = 100$ (Table 6.5). The last fivefold increase in evaluations reduces the median distance by 0.0068 m while raising the time per control step from 190.5 to 892.4 ms. Unlike D3IL-avoiding, where one network evaluation already saturates the task (Section 6.1), alignment rewards additional evaluations.

The three flow-based models are measured at $K = 100$ as well, and the ordering among them is the same as at twenty: analytic average-velocity matching first (0.0741 and 0.0673 m), then the consistency-interpolated model (0.3897 and 0.1791 m), with instantaneous-velocity matching last (0.4085 and 0.4247 m). A hundred evaluations more than halve the consistency-interpolated model's median and leave instantaneous-velocity matching where it was. On the complete ladder instantaneous-velocity matching closes 6 to 10 % of the distance at every budget from two to a hundred, and the consistency-interpolated model 17, 6, 14 and 60 %, so at ten evaluations it is at the level of instantaneous-velocity matching. Analytic average-velocity matching leads at every budget: at ten evaluations it ends closer than instantaneous-velocity matching at less time per control step (99.2 against 145.1 ms), and closer than the consistency-interpolated model at the same time (98.5 ms). The diffusion baseline is compared at the budget it was trained with. The ladder starts at two evaluations: the analytic model closes 39 % of the distance there against 84 % at twenty, so a smaller budget is not an operating point on this task.

Putting the outcome and its price on the same axes gives this task its cost frontier (Figure 6.3). It is not the frontier of Figure 6.2 and cannot be: on D3IL-avoiding every eligible configuration solves the task and only the two costs differ, whereas here nothing is solved or failed — what a model delivers is a distance, and the control-step count is set by the episode limit rather than by arrival (Table 5.9). The frontier is therefore the outcome itself against the time one action costs, and a configuration is on it when nothing else both ends closer to the target and computes faster.

Three marks of the drawing carry a reading rule. The outcome axis is the share of the starting distance the box was moved towards its target, the bracketed number of Table 6.5, so the dashed line at 0 % is a box that was not moved and 100 % at the top is a box delivered; two points that would print on top of each other are displaced by a few pixels, which moves the drawing and not the data; and the boxed arrow is the direction in which both axes improve.

Four of the ten configurations are on it, and all four are analytic average-velocity matching, at $K = 2, 10, 20$ and 100. Every other model at every budget is dominated, and the way they are dominated is the result: at $K = 20$ the consistency-interpolated

objective, instantaneous-velocity matching and the diffusion baseline all sit close to the line at which the box was not moved at all, at between 189 and 298 ms per control step, while the analytic objective at $K = 2$ — a seventh to a tenth of that cost — has already closed two fifths of the distance. MeanFM’s own $K = 100$ point reduces its median distance by only 0.0068 m at 4.7 times its $K = 20$ cost.

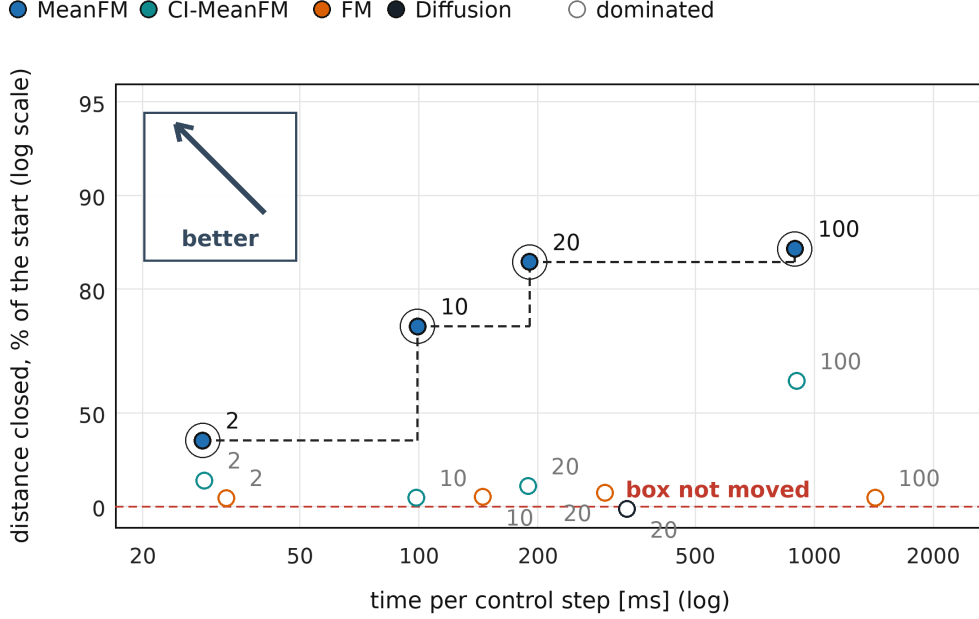


Figure 6.3.: D3IL-aligning without projection: the share of the starting distance the box was moved towards its target, median over the ten contexts, against the time to compute one action on a log axis; every cell of Table 6.5 except the two $\alpha_{\text{end}} = 0.05$ rows, one training seed. The dashed red line at 0% is a box not moved. Rings mark the non-dominated configurations, joined by the dashed staircase; beside each point is its step budget; the boxed arrow points where both axes improve. The outcome axis is logarithmic in the distance left to the target; the tick labels are the share closed.

6.2.1.2. Consistency Interpolation across the Step Budget

The consistency-interpolated objective behaves differently from the analytic one on this task: it does not improve when it is given a larger budget. At two network evaluations the two are level — each ends closer in five of the ten contexts. At twenty, where the analytic objective has moved the box most of the way to its target, the consistency-

interpolated one has not moved at all from where it was at two evaluations, and it ends further away in seven of ten contexts. Raising the budget spends time and buys nothing, and the same holds for the more aggressive setting $\alpha_{\text{end}} = 0.05$ (Table 6.5).

The numbers behind that are flat where the analytic objective's fall. With $\alpha_{\text{end}} = 0.2$ the median final distance moves from 0.3738 to 0.3897 m between two and twenty evaluations, and with $\alpha_{\text{end}} = 0.05$ from 0.3978 to 0.4488 m, against the analytic objective's 0.2779 to 0.0741 m; the time per control step is the same for the three at each budget.

The mechanism is in the training target. It is built from a stop-gradient copy of the network itself, so part of the network's own error enters the field it learns, and that part does not shrink when the solver takes smaller steps (Subsection 4.4.4). A task that rewards more evaluations therefore exposes it, as this one does; a task already saturated at one evaluation, as D3IL-avoiding is, does not. The level result at $K = 2$ is in distance only. Counting the contexts in which any configuration reaches the target, analytic average-velocity matching succeeds in seven and consistency-interpolated average-velocity matching in two (CLOSURE_20260907 R.4).

6.2.2. Projection Methods

D3IL-aligning is the task on which the generative model needs many network evaluations: analytic average-velocity matching comes closest to the target at $K = 20$ (Subsubsection 6.2.1.1), so that is the model and the budget the projection is put on. The projected results below are all read on it.

The activation threshold is compressed, because the budget is high. Everything about the threshold on this task follows from one fact: D3IL-aligning is the task that needs a large step budget, and the default of DPCC, $\eta = 0.5$, was set for small ones. The projector runs on the last $\lceil \eta K \rceil$ sampling steps, so at a fixed share η its cost grows with the budget: every extra sampling step in the active window is another program solved, for each of the four candidate plans, at each of up to four hundred control steps. On D3IL-avoiding at one or two evaluations the default costs one solve; here, at $K = 20$, it would be ten solves per candidate per control step, and at $K = 100$ fifty.

What a solve costs is measured on the two budgets the projector has been run at (Table 6.6). At $K = 20$ with $\eta = 0.2$ the projector runs on four sampling steps, sixteen solves per control step, and the projected control step costs 93 ms more than the unprojected one: about 6 ms per solve. At $K = 10$ with $\eta = 0.4$, the same sixteen solves cost 428 ms, 27 ms per solve, the plans handed to the projector there being further from their final shape. With those two figures the default threshold at $K = 100$ prices itself: fifty active steps are two hundred solves per control step, 1.2–5.4 s on top of the 0.9 s the hundred evaluations take, and an alignment cell is four hundred control steps on ten

contexts — between two and seven hours of computation for one cell as a lower bound, since the early sampling steps of a hundred-step schedule are noisier than any step of a twenty-step one. That pair is not run. The compressed threshold at the same budget, $\eta = 0.1$, is forty solves per control step, which the same figures price at 0.2–1.1 s on top of the generation; measured, the forty solves add 187 ms to the 925 ms of the hundred evaluations, under 5 ms per solve (Table 6.6).

The threshold is therefore compressed as the budget grows, so that the projector keeps a roughly constant *number* of guiding steps instead of a constant share: $\eta = 0.2$ at $K = 20$ and $\eta = 0.4$ at $K = 10$, which is three guiding steps at both (Subsection 5.6.4), where on D3IL-avoiding at one evaluation there were none. Every projected alignment result of this chapter is under that compressed threshold, and Table 6.6 is the ladder it was chosen on. What a lower threshold gives up is the early part of sampling, where the sample is still far from its final shape and the projection has least to hold on to; what it buys is a projected configuration whose time per action is of the same order as the generation it is attached to.

Each row of Table 6.6 is the analytic average-velocity model under random selection on the tightened set, the configuration of the $K = 100$ comparison, under *both* projectors: the per-step projector of DPCC, which is what the threshold governs, and endpoint projection at the same threshold, whose guiding steps are the same count minus the terminal one. The number of guiding steps is the quantity held roughly constant. The $K = 2, 10$ and 20 rows are the random-rule cells of Table 6.7, the compressed $K = 100$ row is its own evaluation, and the uncompressed $K = 100$ row is not run, for the cost set out above.

Along the ladder the compressed threshold holds the projected control step at 1.9, 5.8, 1.5 and 1.2 times the unprojected one at $K = 2, 10, 20$ and 100 under per-step projection, and at 2.1, 1.6 and 1.2 times under endpoint projection where it has guiding steps: at a budget of a hundred, nine guiding steps add a fifth to the time of the generation. Endpoint projection keeps ten of ten contexts violation-free at budgets of ten and twenty and eight at a hundred; per-step projection keeps seven to nine at every budget. Under either projector the median ends further from the target than the unprojected plan at budgets of ten and twenty: 0.179 and 0.197 m against 0.090 m at $K = 20$, 0.434 and 0.242 m against 0.142 m at $K = 10$. At a budget of two the per-step median, 0.275 m, is closer than the unprojected 0.320 m, and at a hundred endpoint projection ends at 0.153 m, the smallest median of any projected row of the ladder.

Table 6.7 compares the two projection methods on the analytic average-velocity model, on tightened constraints, under all three selection rules.

Every cell of that table, and of every other alignment table in this chapter, is the same ten contexts. Some of the evaluations behind them were run over thirty contexts rather than ten; those ten are a subset of those thirty in every case, so each cell is restricted to

Table 6.6.: The activation threshold η on D3IL-aligning: MeanFM under random selection, tightened constraints, ten contexts, one training seed, under both projectors. *Guiding*: sampling steps the projection acts on minus the terminal one, $\lceil \eta K \rceil - 1$ (Subsection 5.6.4). *ms/step*: mean $\pm$ sample standard deviation over the contexts, unprojected and under each projector; *median*: median final box-to-target distance from 0.4530 m; *violation-free*: contexts free of violations. The $K = 2, 10$ and 20 rows are the random-rule cells of Table 6.7.

K	η	Guiding	per-step projection				endpoint projection		
			ms/step	ms/step	Median [m]	Violation-free	ms/step	Median [m]	Violation-free
2	0.5	0	24.6	46.2	0.275	8/10	— no guiding step		
10	0.4	3	89.7	517.5	0.434	7/10	192.5	0.242	10/10
20	0.2	3	172.5	265.9	0.179	9/10	275.3	0.197	10/10
100	0.1	9	924.6	1112.0	0.248	8/10	1107.0	0.153	8/10
100	0.5	49	<i>not run</i> : two hundred solves per control step (text)						

them. The section is then paired on one context set throughout, as Subsubsection 5.6.3.2 says it is, rather than setting a ten-context result beside a thirty-context one.

The $K = 2$ block has per-step rows only. At its activation threshold of 0.5 a two-step budget has no guiding step — $\lceil \eta K \rceil - 1 = 0$ (Subsection 5.6.4, Subsubsection 6.1.2.6) — so endpoint projection does not exist there as a method: what the evaluation ran under that name at $K = 2$ is projection after sampling, and it is not printed as an endpoint row. The smallest budget at which the two methods can be compared is $K = 3$.

At $K = 20$, endpoint projection keeps all ten contexts free of violations with random selection and with cumulative projection cost, and nine with temporal consistency; per-step projection keeps nine, nine and eight. Together with endpoint projection and random selection at $K = 10$, these are the only configurations at ten or more evaluations that keep every context free of violations while the box is still moved. The time per control step is the same for both methods at this budget, 275 against 266 ms with random selection: what endpoint projection adds at $K = 20$ is the removal of the last violations.

At $K = 10$ the difference is larger. Endpoint projection with random selection again keeps all ten contexts free of violations, where per-step projection keeps seven, and it takes 193 against 518 ms per control step, less in every context whose time differs. With the other two rules it is faster by 320 and 211 ms per control step and keeps at least as many contexts free of violations.

Of the two budgets, $K = 20$ is the operating point, because the generative model itself is closest to the target there; $K = 10$ reaches the same constraint satisfaction with

6. Results

Table 6.7.: D3IL-aligning, MeanFM with each projection method, tightened constraints, ten contexts, one training seed. *Violation-free*: contexts with no violating control step; *violating steps*: per context; *moved*: contexts in which the box moves; *median* and *mean*: final box-to-target distance over the ten contexts, the median with the share of the initial 0.4530 m closed in brackets, the mean $\pm$ sample standard deviation; ms/step likewise. Rules r, c, t as in Table 6.2. $\blacktriangleright$ the selected configuration.

K	Projection	Rule	Violation-free	Violating steps	Moved	Median [m]	Mean [m]	ms/step
20	none per-step	—	2/10	31.6	9/10	0.090 (80 %)	0.143 ± 0.134	172.5 ± 60.6
		r	9/10	2.7	8/10	0.179 (61 %)	0.200 ± 0.150	265.9 ± 101.9
		c	9/10	4.3	6/10	0.333 (27 %)	0.306 ± 0.164	325.3 ± 165.8
		t	8/10	1.5	7/10	0.228 (50 %)	0.240 ± 0.165	323.2 ± 175.0
	endpoint	$\blacktriangleright r$	10/10	0.0	8/10	0.197 (57 %)	0.211 ± 0.144	275.3 ± 102.4
		c	10/10	0.0	6/10	0.323 (29 %)	0.286 ± 0.190	282.3 ± 104.1
		t	9/10	0.6	7/10	0.230 (49 %)	0.255 ± 0.154	301.5 ± 128.6
10	none per-step	—	1/10	35.5	8/10	0.142 (69 %)	0.199 ± 0.179	89.7 ± 31.5
		r	7/10	12.3	8/10	0.434 (4 %)	0.420 ± 0.352	517.5 ± 344.3
		c	6/10	21.3	6/10	0.411 (9 %)	0.352 ± 0.174	524.3 ± 417.2
		t	7/10	5.0	4/10	0.408 (10 %)	0.271 ± 0.217	433.5 ± 224.6
	endpoint	r	10/10	0.0	8/10	0.242 (47 %)	0.236 ± 0.126	192.5 ± 72.3
		c	8/10	0.6	4/10	0.425 (6 %)	0.307 ± 0.198	204.2 ± 79.4
		t	8/10	2.3	6/10	0.255 (44 %)	0.284 ± 0.161	222.1 ± 99.0
2	none per-step	—	5/10	46.1	8/10	0.320 (29 %)	0.323 ± 0.279	24.6 ± 8.6
		r	8/10	11.6	9/10	0.275 (39 %)	0.286 ± 0.210	46.2 ± 18.4
		c	8/10	6.5	5/10	0.434 (4 %)	0.329 ± 0.186	47.3 ± 19.1
		t	10/10	0.0	7/10	0.221 (51 %)	0.250 ± 0.177	43.0 ± 16.0
	endpoint	— none: no guiding step at $K = 2$ (Subsection 5.6.4)						

endpoint projection in 70 % of the time. Repeated evaluations of configurations with projection vary by about 0.4 m in final distance, so the median distances of the projected rows are not ranked against each other.

The two-evaluation block says one thing: the budget at which this task can be run cheaply is not the budget at which the generative model comes closest to solving it. Per-step projection under temporal consistency keeps all ten contexts free of violations there at 43 ms per control step, which nothing at the two larger budgets does at that price, but the median ends 0.22–0.43 m from the target, against 0.09 m unprojected at $K = 20$.

6.2.2.1. Endpoint Projection across the Models

The comparison above is one generative model. Table 6.8 runs it across the models at the operating point this section has settled on — $K = 20$ and the compressed threshold $\eta = 0.2$ — under every selection rule, and it holds in the same direction. The budget ladder below twenty was the subject of Table 6.7 and Table 6.6; it is not repeated here, because the model that solves the task does so at twenty and the threshold study has already priced the other budgets.

All three flow-based models carry both projectors at $K = 20$ on the tightened set, and the diffusion baseline carries per-step projection; endpoint projection is not defined for it. The comparison before projection in Table 6.5 and the one after projection here therefore rest on the same four models. Nine comparisons run the two methods on the same model, the same budget and the same rule. Endpoint projection keeps more contexts free of violations in seven of them and per-step projection in two, instantaneous-velocity matching and the consistency-interpolated model under temporal consistency, each by one context. A second evaluation of one projected cell — analytic average-velocity matching at $K = 2$, per-step, random selection — moved it by one context and 3 cm in median, so a one-context difference is within what a re-run changes.

That is the reverse of D3IL-avoiding, and the reason is the budget rather than the task. This task is run at ten and twenty network evaluations because the generative model needs them (Subsubsection 6.2.1.1), so the endpoint solve has several guiding steps to act on, where on D3IL-avoiding at one evaluation it had none (Subsection 6.1.2). Given steps to act on, the method does what it is for: it removes violations from the plan before the plan is finished, rather than correcting each step after the fact.

What it does not do is dominate. Its advantage is on the constraint and not on the task: across the same nine comparisons it moves the box in more contexts than per-step projection in one, in as many in four, and in fewer in four, and its median final distances are level. Nor is it cheaper at this budget as a rule: on the analytic model it costs 0.87 to 1.04 of per-step projection’s time per control step, on the consistency-interpolated

model 0.71 to 1.08, on instantaneous-velocity matching 1.13 to 1.24. Endpoint projection is therefore the better projector on the constraint on this task, at the budget the task is run at, by a margin that is consistent rather than large, and at a comparable price.

On the tightened set one of the ten contexts is never attempted: the tightening enlarges the obstacle until it overlaps the box of context 8, the evaluation’s box–obstacle guard holds the box, and the context scores clean and unmoved in every tightened row. Every $x/10$ of the tightened alignment tables (Table 6.6, Table 6.7, Table 6.8) is therefore $x - 1$ of nine attempted contexts, and the diffusion baseline’s single clean context under c and t is that one. Comparisons between rows are unaffected. Whether the chapter states it is the author’s call (DA_20260923_R2fix_R37_aligning.md §6.2). Consistency-interpolated average-velocity matching and instantaneous-velocity matching do not move the box on this task at $K = 20$ (Table 6.5), so their rows compare the two projectors on a plan that does not solve the task. That is what makes them a clean read on the projector — the constraint columns are not confounded by how far the box travelled — and it is also why the median-distance column of those rows carries no information: every cell sits within a few centimetres of the starting 0.4530 m. The analytic average-velocity rows are the opposite case and are the ones to read for the median.

The diffusion baseline under per-step projection keeps four contexts free of violations with random selection and one with each of the other two rules, moves the box in at most three contexts, and ends with its median slightly farther from the target than the box started. Analytic average-velocity matching under the same rule Pareto-dominates its best row: nine violation-free contexts against four, 0.179 against 0.462 m, 266 against 809 ms per control step. The baseline’s violations change family under projection: halfspace and obstacle violations fall to almost none, while executed steps beyond the action bound rise from 64 without projection to between 145 and 408 over the ten contexts.

The medians of Table 6.7 also hide which contexts changed. The per-context records show three differences that the medians cannot show.

The constraint column turns over almost completely: eight of the ten contexts carry a violating step without a projector, one still does under per-step projection, and none does under endpoint projection. The distances do not turn over with it — they move around, and mostly in the wrong direction.

The cost of that turnover falls on the contexts the unprojected arm had already delivered. Context 1 ends 0.065 m from the target unprojected and 0.21 or 0.25 m with a projector; context 8 goes from 0.099 to 0.28 m; context 9 is pushed to within 0.029 m unprojected and is then not moved at all. What the projector pays for constraint satisfaction, it pays in the contexts that were nearly solved.

The exchange is not one-sided. Context 10 improves from 0.143 to 0.022 m and

6. Results

Table 6.8.: D3IL-aligning, both projection methods on the three flow-based models and the diffusion baseline at $K = 20$, $\eta = 0.2$, tightened constraints, the same ten contexts in every row, one training seed. Columns as in Table 6.7; the MeanFM rows repeat the $K = 20$ block of that table, and a negative share is a box that ends farther from the target than it started. Endpoint projection adds its control to the velocity field of a flow-based sampler (Subsection 4.5.3); the diffusion model has none, so its endpoint row is empty. ► the selected configuration; • the diffusion model of DPCC.

Model	K	Rule	Violation-free	Violating steps	Moved	Median [m]	Mean [m]	ms/step
<i>per-step projection</i>								
MeanFM	20	<i>r</i>	9/10	2.7	8/10	0.179 (61 %)	0.200 ± 0.150	265.9 ± 101.9
		<i>c</i>	9/10	4.3	6/10	0.333 (27 %)	0.306 ± 0.164	325.3 ± 165.8
		<i>t</i>	8/10	1.5	7/10	0.228 (50 %)	0.240 ± 0.165	323.2 ± 175.0
CI-MeanFM	20	<i>r</i>	4/10	44.6	8/10	0.429 (5 %)	0.432 ± 0.182	415.8 ± 268.0
		<i>c</i>	5/10	10.7	5/10	0.443 (2 %)	0.388 ± 0.131	311.4 ± 130.1
		<i>t</i>	6/10	23.2	7/10	0.422 (7 %)	0.353 ± 0.153	320.5 ± 153.3
FM	20	<i>r</i>	5/10	14.3	3/10	0.425 (6 %)	0.409 ± 0.112	409.7 ± 147.9
		<i>c</i>	4/10	14.9	2/10	0.425 (6 %)	0.409 ± 0.111	406.8 ± 149.5
		<i>t</i>	6/10	12.8	3/10	0.425 (6 %)	0.411 ± 0.111	402.0 ± 146.4
• Diffusion	20	<i>r</i>	4/10	14.8	3/10	0.462 (−2 %)	0.397 ± 0.160	809.2 ± 756.0
		<i>c</i>	1/10	35.0	0/10	0.456 (−1 %)	0.453 ± 0.037	1069.7 ± 848.6
		<i>t</i>	1/10	41.1	1/10	0.462 (−2 %)	0.456 ± 0.039	766.0 ± 284.5
<i>endpoint projection</i>								
► MeanFM	20	<i>r</i>	10/10	0.0	8/10	0.197 (57 %)	0.211 ± 0.144	275.3 ± 102.4
		<i>c</i>	10/10	0.0	6/10	0.323 (29 %)	0.286 ± 0.190	282.3 ± 104.1
		<i>t</i>	9/10	0.6	7/10	0.230 (49 %)	0.255 ± 0.154	301.5 ± 128.6
CI-MeanFM	20	<i>r</i>	8/10	2.7	7/10	0.425 (6 %)	0.333 ± 0.183	294.2 ± 113.3
		<i>c</i>	6/10	10.1	4/10	0.456 (−1 %)	0.440 ± 0.076	314.8 ± 119.6
		<i>t</i>	5/10	8.6	6/10	0.434 (4 %)	0.368 ± 0.124	344.8 ± 164.6
FM	20	<i>r</i>	6/10	12.0	3/10	0.443 (2 %)	0.398 ± 0.132	498.0 ± 210.4
		<i>c</i>	5/10	10.4	3/10	0.425 (6 %)	0.407 ± 0.109	502.4 ± 212.0
		<i>t</i>	5/10	7.5	2/10	0.425 (6 %)	0.410 ± 0.110	452.4 ± 193.4
• Diffusion	20	— (not defined: no velocity field)						

context 4 from 0.262 to 0.147 m under the same projectors, so what the median reports as a rise from 0.090 to 0.197 m is not a uniform degradation but a redistribution — three contexts much worse, two clearly better, and one, context 7, that no configuration on this task moves at all.

The projected configurations of Table 6.8 are compared on the two axes of Figure 6.3 in Figure 6.4, and what separates the two drawings is said on the page. Every point of the second is a plan *after* projection on the tightened set, so it is dearer per control step and, for the same model and budget, further from the target than its unprojected twin — which is drawn as a faint grey ring for analytic average-velocity matching at the same budget, so the price can be read directly: from 84 % of the distance closed at 190 ms to 61 % at 266 ms. What that price buys is the constraint: a filled point keeps at least nine of ten contexts free of violations, which is this task’s counterpart of success with constraint satisfaction, where the unprojected plan keeps one or two. That is the trade this task poses: pursuing no violation costs completion — the projected plan ends further from the target — and allowing some violation buys completion, the unprojected plan ending closest of all while crossing the constraint in eight contexts of ten. The figure places the two ends of that trade on the same axes. Only filled points enter the cost frontier; the hollow ones remain visible, and on this task they are every point of the two models that do not move the box. The plotted runs use random selection at $K = 20$, the rows of Table 6.8. The diffusion baseline under per-step projection and the consistency-interpolated model under endpoint projection keep at most eight contexts free of violations (Table 6.8), below the nine a filled point needs, so both are hollow and the frontier does not include them.

6.2.3. Conclusion

D3IL-aligning repeats the result of D3IL-avoiding on a harder task and adds endpoint projection to it. On camera observations and an alignment through contact, analytic average-velocity matching again dominates the diffusion baseline: under per-step projection with random selection it keeps nine contexts free of violations against the four of the baseline’s best projected row, its median box ends 0.179 against 0.462 m from the target, and it takes 266 against 809 ms per control step. Without projection every non-dominated configuration of the task is analytic average-velocity matching: at twenty evaluations it closes 84 % of the distance, against 14, 10 and -2% for the consistency-interpolated model, instantaneous-velocity matching and the baseline, and at two evaluations, at a seventh to a tenth of their cost, it has already closed two fifths. The consistency-interpolated model does not improve with the budget and reaches 60 % only at a hundred evaluations: its training target is built from a stop-gradient copy of the network, so part of its own error enters the field it learns, and smaller steps do not

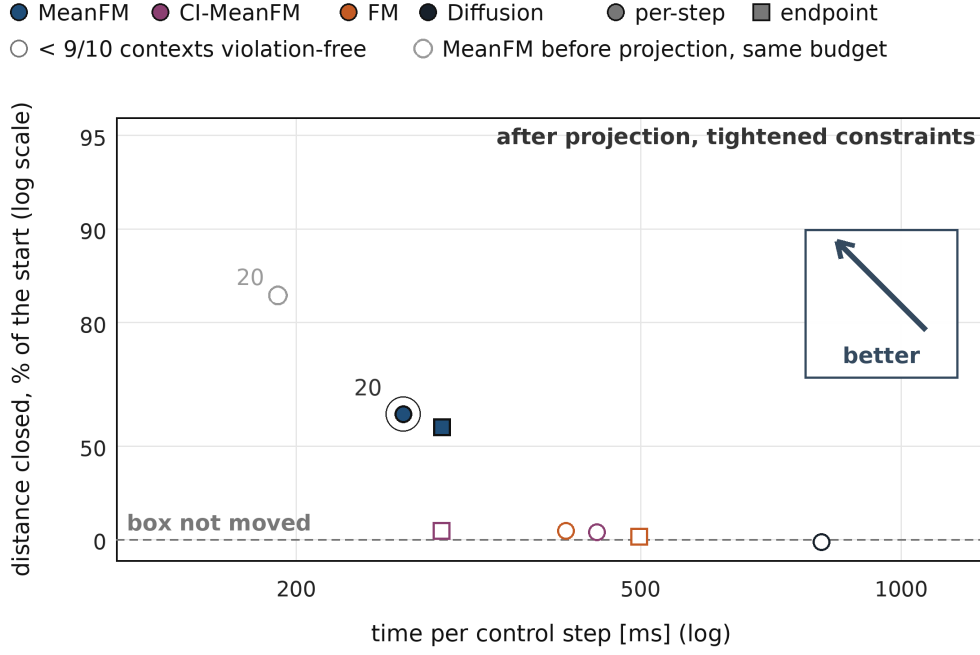


Figure 6.4.: D3IL-aligning after projection, on the axes of Figure 6.3: the cells of Table 6.8 under random selection, $K = 20$, $\eta = 0.2$, tightened constraints, one training seed. Colour: MeanFM, CI-MeanFM, FM, diffusion; circles per-step projection, squares endpoint projection; the faint dashed grey ring is the unprojected MeanFM cell at the same budget. Filled points keep at least nine contexts violation-free, hollow ones do not; rings and the dashed staircase mark the non-dominated filled points; the dashed line at 0 % is a box not moved.

remove it. A task that rewards more evaluations exposes this; D3IL-avoiding, saturated at one evaluation, does not.

At the large budget this task needs, endpoint projection is the better projector on the constraint. With random selection it keeps all ten contexts free of violations and still moves the box towards its target in eight, at the time per control step of per-step projection at $K = 20$ and at 37% of it at $K = 10$, and it keeps more contexts free of violations than per-step projection in seven of nine comparisons. That is the reverse of D3IL-avoiding, and the reason is the budget rather than the task: at ten and twenty evaluations the endpoint solve has steps to guide, and it removes violations before the plan is finished. Its advantage is on the constraint and not on the task, at a comparable price, and the task poses its own trade: pursuing no violation costs completion, the projected plan ending further from the target than the unprojected one, and the budget at which the task runs cheaply, two evaluations at 43 ms, is not the budget at which the model comes closest to it. No context ends in position, within the 1.8 cm of the task’s own test: with endpoint projection the box stops 2.2 and 4.7 cm from its target in the best two contexts and 13 to 27 cm away in six more, and the unprojected plan’s best two end 2.8 and 2.9 cm away. The conclusion is limited to training-set contexts. The executed end-effector paths of the three projection settings, context by context, are drawn in Section B.2.

6.3. Quadrotor Benchmark

The section is one part per scene, and each scene is read like the two manipulation benchmarks: the generative models first on the plans they generate, executed without projection; the same cells then under the projector of DPCC, where the cost frontier and the flown paths are; and the two projection methods against each other last. The scenes come in the order of what they add. UAV-pillars flies the plans of the avoiding planner itself — the obstacle field of D3IL-avoiding scaled into an arena of pillars, the same test-time constraints, the quadrotor as the plant — so it asks whether a plan the projector repairs on D3IL-avoiding survives execution by a vehicle. UAV-corridor is the scene whose test-time constraints leave the horizontal plane — a slide leaned into the vertical, and a roof — read one at a time, as the geometries of D3IL-avoiding are, and neither of which the demonstrations satisfy. UAV-s-curve is the scene of the two sharp turns. It is read in the same order and adds one step: the model whose plans cross its finish line most often is the one the projector and then the controller comparison of Subsubsection 6.3.3.3 are applied to, the one place in this thesis where MuJoCo MPC replaces the tracking controller (Subsection 6.3.3). Each scene closes on its own conclusion.

UAV-corridor and UAV-s-curve use one training seed, and every generative model there is a temporal U-Net with 3.96–3.97 M parameters, trained with the same optimiser settings (Table 5.8); UAV-pillars flies the models of D3IL-avoiding at its five training seeds (Subsubsection 5.6.3.3). Every flight is tracked by the cascaded geometric controller of Subsection 4.3.5 unless stated otherwise. The diffusion model is trained at $K = 20$ and is compared at the budget it was trained with.

6.3.1. UAV-pillars

This scene is D3IL-avoiding flown. The arena is the obstacle field scaled by 36, the constraint sets are the three geometries of the table mapped into it, and the planner is the one of Section 6.1, not retrained (Subsection 5.2.3). What is read here is therefore not a new benchmark but a retest of the result of D3IL-avoiding on another plant. The configurations flown are the Pareto-optimal ones of that benchmark, the two average-velocity models at one and two network evaluations on its cost frontier (Subsubsection 6.1.2.3, Table 6.4), and they are flown again, before and after per-step projection, to test what else happens when a quadrotor and its tracking controller execute the same plans instead of the manipulator: whether the result holds, and where the flown outcome departs from the table’s. Each flown outcome is set beside its table outcome at the same protocol, five training seeds $\times$ two episodes per geometry, averaged over the three geometries. Every flight of the scene ended by crossing the goal line or by touching a pillar; none reached the episode limit and none was aborted.

6.3.1.1. From D3IL-avoiding to UAV-pillars

Before projection the vehicle changes nothing (Table 6.9, upper block). The four unprojected cells read 0.033 success with constraint satisfaction on D3IL-avoiding and 0.033 to 0.100 on UAV-pillars, with the same 15 to 20 violating control steps per episode and the same sixty control steps to the goal line; the planner’s time per action is the same to within a millisecond, since nothing about the plant enters it. Nine of the 120 unprojected flights end in a collision with a pillar, and the rest cross the goal line.

After projection the declared constraints hold on UAV-pillars as they hold on D3IL-avoiding, and a third of the flights still fail. The projected rows read 0.0 violating control steps on every flight that reached the goal line — 78 flights, none of which crosses a halfspace or enters the keep-out disk — so what the per-step projector guarantees on D3IL-avoiding it guarantees on UAV-pillars. Yet success falls from 0.967 and 1.000 to 0.600 to 0.700: 42 of the 120 projected flights end in a collision with a pillar, with no violation of the declared constraint set up to that moment. Since no projected flight violates a constraint, success with constraint satisfaction falls with success. The loss

Table 6.9.: The two average-velocity models (CI-MeanFM at $\alpha_{\text{end}} = 0.2$) at $K = 1$ and 2, before projection and after per-step projection under temporal consistency, on tightened constraints: executed by the manipulator on D3IL-avoiding and flown by the quadrotor on UAV-pillars, one row each. Columns as in Table 6.1. The protocol of DPCC on both: five training seeds $\times$ two episodes per geometry, averaged over the three geometries; mean $\pm$ the spread over the seeds. Every UAV-pillars flight without success ended in a collision with a pillar. $\blacktriangleright$ the selected configuration of D3IL-avoiding.

Model	K	Environment	Success	S&C	Violating steps	Steps
<i>before projection</i>						
MeanFM	1	D3IL-avoiding	1.000 ± 0.000	0.033 ± 0.075	15.5 ± 2.1	63.4 ± 5.6
		UAV-pillars	1.000 ± 0.000	0.100 ± 0.091	19.9 ± 0.6	62.7 ± 3.3
MeanFM	2	D3IL-avoiding	1.000 ± 0.000	0.033 ± 0.075	15.5 ± 0.4	62.7 ± 1.8
		UAV-pillars	0.800 ± 0.274	0.100 ± 0.091	19.1 ± 1.8	60.9 ± 3.2
CI-MeanFM	1	D3IL-avoiding	1.000 ± 0.000	0.033 ± 0.075	15.6 ± 0.7	60.5 ± 1.5
		UAV-pillars	1.000 ± 0.000	0.100 ± 0.149	18.1 ± 3.2	59.7 ± 0.7
CI-MeanFM	2	D3IL-avoiding	1.000 ± 0.000	0.033 ± 0.075	15.7 ± 0.7	62.9 ± 1.2
		UAV-pillars	0.900 ± 0.224	0.033 ± 0.075	19.8 ± 1.7	62.3 ± 4.2
<i>after per-step projection, temporal consistency</i>						
$\blacktriangleright$ MeanFM	1	D3IL-avoiding	0.967 ± 0.075	0.967 ± 0.075	0.0	58.6 ± 2.2
		UAV-pillars	0.600 ± 0.190	0.600 ± 0.190	0.0	61.4 ± 2.3
MeanFM	2	D3IL-avoiding	0.967 ± 0.075	0.967 ± 0.075	0.0	59.4 ± 2.0
		UAV-pillars	0.667 ± 0.236	0.667 ± 0.236	0.0	60.2 ± 1.6
CI-MeanFM	1	D3IL-avoiding	1.000 ± 0.000	1.000 ± 0.000	0.0	59.2 ± 1.2
		UAV-pillars	0.633 ± 0.217	0.633 ± 0.217	0.0	60.7 ± 1.6
CI-MeanFM	2	D3IL-avoiding	1.000 ± 0.000	1.000 ± 0.000	0.0	60.1 ± 3.4
		UAV-pillars	0.700 ± 0.139	0.700 ± 0.139	0.0	60.6 ± 0.6

sits on one geometry. On *both-hard* two to four of ten projected flights succeed and the other six to eight end in a collision, while on the two single-halfspace geometries seven to nine of ten succeed (Table 6.10). The two models lose the same amount on the same geometry at both budgets, so nothing here re-orders them.

Table 6.10.: The projected rows of Table 6.9 per constraint geometry: success with constraint satisfaction on D3IL-avoiding and on UAV-pillars. Every UAV-pillars flight counted as a failure here ended in a collision with a pillar.

Model	K	<i>top-right-hard</i>		<i>top-left-hard</i>		<i>both-hard</i>	
		D3IL-avoiding	UAV-pillars	D3IL-avoiding	UAV-pillars	D3IL-avoiding	UAV-pillars
MeanFM	1	0.90	0.90	1.00	0.70	1.00	0.20
MeanFM	2	0.90	0.80	1.00	0.80	1.00	0.40
CI-MeanFM	1	1.00	0.80	1.00	0.90	1.00	0.20
CI-MeanFM	2	1.00	0.90	1.00	0.90	1.00	0.30

Two things about these collisions are the quadrotor’s own: why they happen, and what follows from them. They happen because of the plans and the scene, not the tracking. Five of the six pillars are in no constraint set a geometry declares — one or two halfspaces and one keep-out disk, which covers the sixth (Subsection 5.2.1) — so the projector keeps the plan away from none of them, and the demonstrations of D3IL-avoiding pass them at the rod’s own radius, which that benchmark never scores. The map of the scene sends that radius onto the rotor reach (Subsection 5.2.3), so a commanded path that skims a post on D3IL-avoiding brings the rotors into the pillar on UAV-pillars: on 50 of the 51 flights that ended in a collision the commanded path itself passes within the reach of that pillar, and only one collision is decided by the tracking lag. On *both-hard* the two halfspaces funnel the plan through the centre of the field and past the outer pillars of the third row, which is where 29 of the 42 projected collisions are; on the single-halfspace geometries the collisions are on the second row, where the plan rounds the pillar next to the keep-out disk.

What follows from a collision is where the quadrotor differs from the manipulator in kind. The arm stands on a fixed base and holds its end effector under position control, so an end effector that meets an obstacle is stopped and stays under control. The quadrotor is held in the air by its rotors alone: a rotor that strikes a pillar no longer gives the thrust the attitude loop relies on, and in most cases the vehicle loses control of its attitude and falls. A collision is therefore the end of a flight, not an event to be counted within it. The simulation does not model the strike itself; the scene ends the flight at its first contact with a pillar, and the tables read it through the success column

(Subsection 5.4.3).

6.3.1.2. Executed Paths: the Manipulator and the Quadrotor

Figure 6.5 draws the flights of the selected configuration, MeanFM at $K = 1$, on each geometry, before and after projection. Before projection every flight crosses the goal line and almost every one crosses the declared boundary on its way: on *top-right-hard* the flights that take the upper passage cut through the halfspace and those that take the lower one pass through the keep-out disk, and on *both-hard* both passages cross a halfspace. After projection no path enters the shaded region or the disk; the paths hug the dashed tightened boundary instead, and where a hugging path meets a pillar that stands at the boundary the flight ends in a collision — one flight on *top-right-hard*, three on *top-left-hard*, eight on *both-hard*, at the third-row pillars on either side of the finish line's centre. The paths that do reach the line are the paths of D3IL-avoiding (Figure B.1) at scale 36.

6. Results

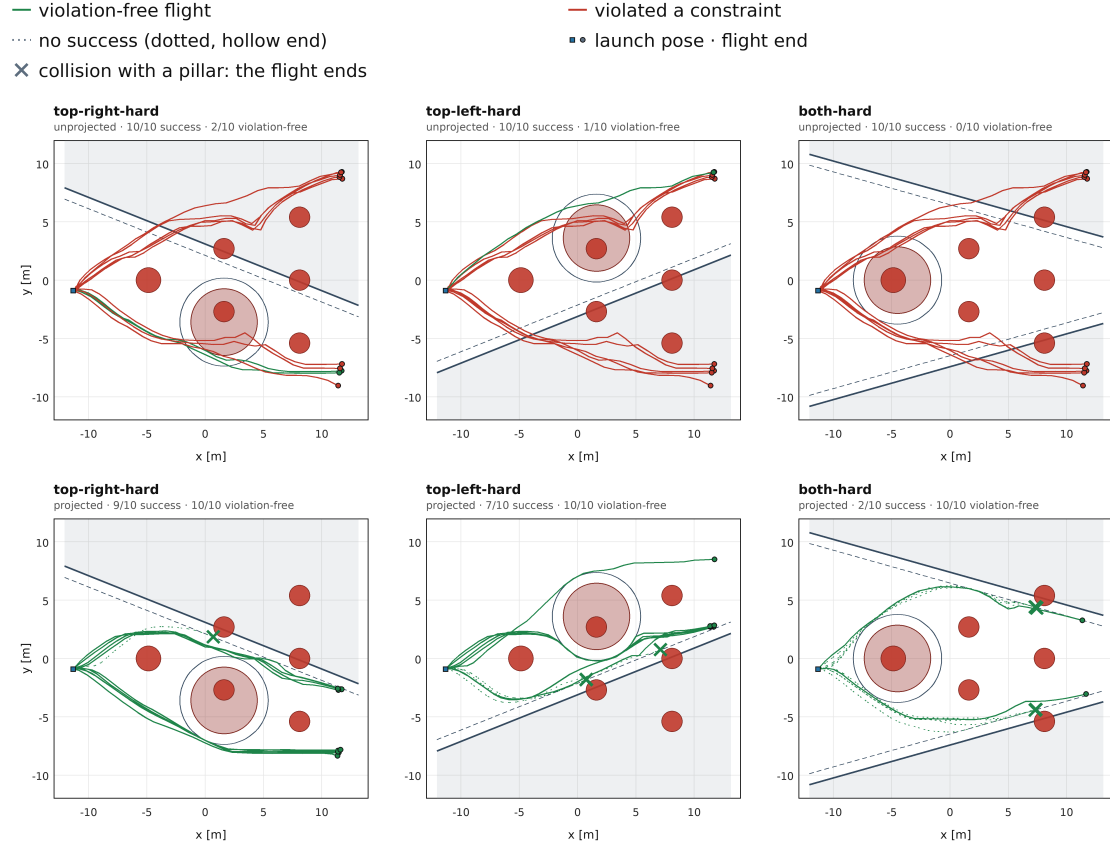


Figure 6.5.: UAV-pillars from above, MeanFM at $K = 1$: the ten flights of each constraint geometry (columns) before projection (upper row) and under per-step projection with temporal consistency (lower row), one seed per pair of flights. Shaded regions and the dashed tightened boundary as in Figure 5.8; the six pillars in red. Green: violation-free; red: violated a constraint; dotted: no success; a cross marks a collision with a pillar, which ends the flight.

6.3.1.3. Conclusion

UAV-pillars flies the Pareto-optimal operating points of D3IL-avoiding again, on the quadrotor, to see what else happens when the plant changes, and it reaffirms D3IL-avoiding. Its demonstrations, planner, projection and constraint rows are those of D3IL-avoiding, and only the plant that executes each commanded position is replaced (Subsection 4.6.3). As far as the declared constraints go, the result of D3IL-avoiding holds when flown: before projection the flown plans violate them as the executed ones do, after projection neither violates them at all, at the same control steps to within three and the same time per action to within a millisecond. The two average-velocity models keep their order and their gap, CI-MeanFM succeeding within the constraints on one flight in thirty more than analytic average-velocity matching at both budgets, on UAV-pillars as on D3IL-avoiding, and per-step projection is again what turns plans that violate the declared constraints in almost every episode into plans that violate none.

Where the flown outcome departs from the table, the cause is the plant and its controller, and what they change against the manipulator is twofold. The vehicle tracks more closely: its flown position stays within about 0.007 of the commanded one in the table's units, against the manipulator's own 0.02 to 0.04 and well inside the tightening margin of 0.025, so the flown path keeps the constraint satisfaction the projection gives the plan. And an obstacle costs more. Success within the constraints falls to 0.600–0.700 against 0.967–1.000, entirely through collisions with the five pillars the constraint sets leave out: the plans pass them as closely as the demonstrations passed the posts, at the rod's radius, which the scale of the scene makes the rotor reach — on 50 of the 51 collisions the commanded path itself is inside the reach — and for a quadrotor a collision is catastrophic, a rotor strike that in most cases ends in lost control, so the flight ends at the first contact. Both models lose the same share on the same geometry, so the collisions do not re-order them. The declared constraint set transfers exactly; freedom from collision with the obstacles it leaves out does not. Only the Pareto-optimal operating points of D3IL-avoiding were flown on this scene, since it tests what the change of plant does to them; the diffusion baseline and instantaneous-velocity matching were not flown, and the dominance over the baseline remains a result of D3IL-avoiding.

6.3.2. UAV-corridor

The two constraints are read one at a time, as the avoiding geometries are: every table of this scene has a *tilt* block and a *hump* block, and nothing is averaged across them. A flight succeeds when it reaches the end of the corridor, the scene's finish line, and since almost no flight of this scene is free of violations, the constraint is read as the number

of violating control steps per flight (Subsection 5.4.3). Every value is the mean over the ten flights of a cell with the sample standard deviation over them.

6.3.2.1. Generative Models

Without projection every model reaches the end of the corridor on every flight at every budget, and every flight violates the constraint (Table 6.11): a flight violates the tilt on 74.5 to 99.4 control steps on average and the hump on 28.6 to 36.6. The plans follow the level lanes of the demonstrations, which were flown before either boundary existed (Subsection 5.3.3), so the two blocks are the same flights read against two constraints: they share their steps, their time per action differs by at most 1.5 ms between the two runs, and they differ in what they violate. The flow-based models lie within one standard deviation of one another in violating steps; the diffusion baseline has the most under the tilt and the fewest over the hump. The flights take 242 to 251 control steps to the end of the corridor, and one action costs 9.1 to 9.4 ms at one evaluation, 170.8 to 172.3 ms at twenty and about 178 ms for the baseline. As before projection on D3IL-avoiding, every configuration succeeds, and no cost frontier is drawn for it.

6. Results

Table 6.11.: UAV-corridor with the test-time constraints, **before projection**: the plan flown as generated, ten flights per row spread over the three routes, one training seed, one block per constraint. The grid: the two average-velocity models at $K \in \{1, 2, 3\}$, instantaneous-velocity matching at $K \in \{1, 2, 3, 5, 20\}$, the diffusion baseline at its trained budget. Each value mean $\pm$ the sample standard deviation over the flights; *Success*: reaches the end of the corridor (Subsection 5.4.3); *Violating steps*: control steps of the flight that violate the constraint; *Steps*: control steps to the end of the corridor. • the diffusion model of DPCC.

Model	K	Success	Violating steps	Steps	ms/step
<i>tilt</i>					
MeanFM	1	1.000 ± 0.000	91.3 ± 34.8	246.4 ± 7.0	9.4 ± 0.4
MeanFM	2	1.000 ± 0.000	92.9 ± 30.2	247.3 ± 4.8	18.1 ± 0.4
MeanFM	3	1.000 ± 0.000	92.0 ± 30.4	246.5 ± 5.0	27.0 ± 0.4
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	1	1.000 ± 0.000	90.0 ± 32.6	242.4 ± 4.3	9.3 ± 0.4
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	2	1.000 ± 0.000	93.8 ± 30.0	246.2 ± 5.5	18.4 ± 0.4
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	1.000 ± 0.000	93.1 ± 30.1	245.4 ± 4.9	27.3 ± 0.4
FM	1	1.000 ± 0.000	74.5 ± 25.6	250.9 ± 5.2	9.1 ± 0.4
FM	2	1.000 ± 0.000	83.9 ± 28.4	251.2 ± 2.9	17.7 ± 0.4
FM	3	1.000 ± 0.000	87.0 ± 28.3	251.4 ± 2.7	25.9 ± 0.5
FM	5	1.000 ± 0.000	89.9 ± 28.4	249.7 ± 2.5	43.4 ± 0.4
FM	20	1.000 ± 0.000	94.7 ± 27.0	249.5 ± 2.9	170.8 ± 0.4
• Diffusion	20	1.000 ± 0.000	99.4 ± 27.0	242.4 ± 4.2	178.1 ± 0.8
<i>hump</i>					
MeanFM	1	1.000 ± 0.000	36.0 ± 8.9	246.4 ± 7.0	9.4 ± 0.4
MeanFM	2	1.000 ± 0.000	35.6 ± 8.1	247.3 ± 4.8	18.2 ± 0.4
MeanFM	3	1.000 ± 0.000	35.7 ± 7.8	246.5 ± 5.0	27.2 ± 0.4
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	1	1.000 ± 0.000	36.6 ± 8.4	242.4 ± 4.3	9.3 ± 0.4
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	2	1.000 ± 0.000	35.6 ± 7.9	246.2 ± 5.5	18.0 ± 0.4
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	1.000 ± 0.000	35.6 ± 7.6	245.4 ± 4.9	27.1 ± 0.4
FM	1	1.000 ± 0.000	34.7 ± 6.8	250.9 ± 5.2	9.1 ± 0.4
FM	2	1.000 ± 0.000	32.6 ± 7.8	251.2 ± 2.9	17.5 ± 0.3
FM	3	1.000 ± 0.000	32.6 ± 8.0	251.4 ± 2.7	26.3 ± 0.4
FM	5	1.000 ± 0.000	32.0 ± 7.9	249.7 ± 2.5	43.6 ± 0.4
FM	20	1.000 ± 0.000	31.3 ± 7.6	249.5 ± 2.9	172.3 ± 2.6
• Diffusion	20	1.000 ± 0.000	28.6 ± 7.7	242.4 ± 4.2	178.2 ± 2.7

6.3.2.2. Projection

After projection every flow-based configuration still reaches the end of the corridor on every flight, except per-step projection at one and two evaluations over the hump, whose flights stop short of the apex (Table 6.12). Projection removes most of the violating steps and rarely all of them: a flight's count falls from 74.5 to 99.4 under the tilt and from 28.6 to 36.6 over the hump to 1.4 to 10.4 and 0.2 to 8.3 for the flow-based models, and to 0.8 and 0.0 for the diffusion baseline at its best rule, at 666 and 654 ms per action against 26 to 417 ms for the flow-based models. At the route's goal point, the line these flights were first scored at, only the baseline's success differs, 0.000 under the tilt and 0.300 over the hump at its best rule: 41 of its 60 projected flights had passed the end of the corridor and were still flying towards the goal when the episode limit ended them (Subsection 5.4.3). The budgets are read as one ladder across the scene: the per-step projector is active at every budget, endpoint projection at $K \geq 3$, where it has a guiding step (Subsubsection 6.1.2.6), so at $K = 1$ and $K = 2$ the table carries per-step rows only.

The budget makes the violations fewer and shallower and does not remove them. Wherever the flights reach the end of the corridor, the violating steps per flight fall with the budget, under both constraints and both projection methods. Under the tilt they fall from 9.8 to 11.1 at one and two evaluations to 2.1 to 3.2 at twenty under per-step projection and to 1.4 to 2.5 under endpoint projection, and the deepest of them from 8.8 to 11.9 cm to 1.3 to 2.2 and 1.0 to 1.3 cm. Over the hump they fall from 4.6 to 6.5 under per-step and 8.0 to 10.4 under endpoint projection at three evaluations to 1.8 to 2.2 and 0.2 to 0.7 at twenty. The budget at which a flight stops reaching the finish line lies at the other end of the ladder: under per-step projection at one and two evaluations 177 of the 180 flow-based flights over the hump stop short of the apex, and the other three reach the end of the corridor only after drifting across the boundary of its wall. The baseline, at its one budget, is the slowest planner on the scene: its projected flights need 290 (tilt) and 321 (hump) control steps to the end of the corridor, against 247 to 270 and 277 to 316 for the flow-based ones, and 623 to 680 ms per action, the highest price of the scene.

The cost frontier of this scene is drawn after projection only, on violating steps per flight against the time to compute one action (Figure 6.6, top). A configuration takes part when every one of its flights reaches the end of the corridor — its success within 0.05 of the best, as on D3IL-avoiding — and is on the frontier when no other configuration leaves fewer violating steps at no more time. Both frontiers are staircases along the budget, and the budget is the trade between the two axes: every step up in network evaluations buys fewer violating steps with more time per action. Under the tilt a flight violates on about ten control steps at one and two evaluations (27

6. Results

Table 6.12.: UAV-corridor **after projection**: each model at each budget under the per-step projector of DPCC and, from $K = 3$, under endpoint projection, at its best selection rule (most successes, then fewest violating steps, then least time; every rule in Section B.4); tightened constraints, activation threshold 0.5, ten flights per cell, one training seed. Columns as in Table 6.11; —: no flight reaches the end of the corridor. At the route’s goal point only the baseline’s success differs: 0.000 (tilt) and 0.300 (hump). • the diffusion model of DPCC.

Model	K	Rule	Success	Violating steps	Steps	ms/step
<i>tilt, per-step projection</i>						
MeanFM	1	<i>c</i>	1.000 ± 0.000	10.2 ± 1.1	253.2 ± 4.4	27.4 ± 0.5
MeanFM	2	<i>c</i>	1.000 ± 0.000	10.3 ± 0.9	253.9 ± 4.2	36.4 ± 0.8
MeanFM	3	<i>c</i>	1.000 ± 0.000	6.6 ± 0.8	256.4 ± 2.9	93.6 ± 1.1
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	1	<i>c</i>	1.000 ± 0.000	10.3 ± 0.9	252.0 ± 3.7	27.9 ± 0.5
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	2	<i>c</i>	1.000 ± 0.000	10.4 ± 0.7	251.9 ± 2.4	36.1 ± 0.3
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	<i>c</i>	1.000 ± 0.000	5.0 ± 0.8	254.0 ± 1.8	93.4 ± 1.2
FM	1	<i>c</i>	1.000 ± 0.000	10.2 ± 1.5	259.8 ± 9.2	27.6 ± 1.5
FM	2	<i>c</i>	1.000 ± 0.000	9.8 ± 1.4	253.9 ± 9.4	34.7 ± 0.5
FM	3	<i>c</i>	1.000 ± 0.000	5.8 ± 1.5	247.2 ± 3.4	79.2 ± 3.2
FM	5	<i>c</i>	1.000 ± 0.000	4.4 ± 1.5	247.6 ± 2.0	116.9 ± 2.4
FM	20	<i>c</i>	1.000 ± 0.000	2.1 ± 0.7	252.6 ± 2.7	360.4 ± 1.5
• Diffusion	20	<i>c</i>	1.000 ± 0.000	0.8 ± 1.8	290.3 ± 15.5	666.3 ± 16.7
<i>tilt, endpoint projection</i>						
MeanFM	3	<i>c</i>	1.000 ± 0.000	7.6 ± 1.3	255.2 ± 4.5	73.5 ± 0.8
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	<i>c</i>	1.000 ± 0.000	7.1 ± 1.0	253.2 ± 3.5	72.7 ± 0.5
FM	3	<i>c</i>	1.000 ± 0.000	6.9 ± 1.8	257.8 ± 3.3	71.6 ± 0.6
FM	5	<i>c</i>	1.000 ± 0.000	4.9 ± 1.9	256.6 ± 2.9	115.2 ± 1.4
FM	20	<i>c</i>	1.000 ± 0.000	1.4 ± 0.7	269.8 ± 10.5	416.9 ± 0.8
<i>hump, per-step projection</i>						
MeanFM	1	<i>c</i>	0.000 ± 0.000	0.0 ± 0.0	—	26.4 ± 0.0
MeanFM	2	<i>t</i>	0.100 ± 0.316	17.9 ± 30.0	387.0	42.3 ± 10.9
MeanFM	3	<i>c</i>	1.000 ± 0.000	4.9 ± 0.9	286.3 ± 4.3	91.1 ± 1.8
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	1	<i>t</i>	0.000 ± 0.000	0.0 ± 0.0	—	26.6 ± 0.3
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	2	<i>t</i>	0.200 ± 0.422	19.8 ± 32.1	388.0 ± 2.8	43.0 ± 12.2
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	<i>c</i>	1.000 ± 0.000	4.6 ± 0.8	282.7 ± 3.6	89.2 ± 1.9
FM	1	<i>c</i>	0.000 ± 0.000	0.0 ± 0.0	—	26.2 ± 0.2
FM	2	<i>r</i>	0.000 ± 0.000	0.0 ± 0.0	—	34.5 ± 0.4
FM	3	<i>c</i>	1.000 ± 0.000	5.6 ± 0.5	277.0 ± 7.3	77.1 ± 2.7
FM	5	<i>c</i>	1.000 ± 0.000	3.9 ± 0.7	277.8 ± 7.0	114.1 ± 2.3
FM	20	<i>c</i>	1.000 ± 0.000	1.8 ± 0.4	285.1 ± 5.6	355.2 ± 2.2
• Diffusion	20	<i>r</i>	1.000 ± 0.000	0.0 ± 0.0	320.8 ± 53.6	654.2 ± 89.7
<i>hump, endpoint projection</i>						
MeanFM	3	<i>c</i>	1.000 ± 0.000	8.0 ± 1.5	287.7 ± 4.9	72.1 ± 0.3
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	<i>c</i>	1.000 ± 0.000	8.0 ± 1.2	284.6 ± 2.8	71.9 ± 0.6
FM	3	<i>c</i>	1.000 ± 0.000	8.3 ± 1.1	299.3 ± 4.7	71.9 ± 1.0
FM	5	<i>c</i>	1.000 ± 0.000	3.9 ± 0.3	292.8 ± 3.7	112.4 ± 0.6
FM	20	single 12200	1.000 ± 0.000	0.2 ± 0.4	316.2 ± 6.3	291.0 ± 0.8

to 36 ms per action), on four to eight at three and five (72 to 117 ms) and on one to two at twenty (360 to 417 ms); over the hump on about eight at three evaluations under endpoint projection and four to six under per-step projection (72 to 91 ms), about four at five (112 to 114 ms) and under two at twenty (291 to 355 ms). Under the tilt it runs from analytic average-velocity matching at one evaluation, 10.2 violating steps at 27.4 ms, through instantaneous-velocity matching at two, three, five and twenty evaluations and consistency-interpolated average-velocity matching at three, to the baseline, 0.8 at 666.3 ms. Over the hump it runs from consistency-interpolated average-velocity matching at three evaluations under endpoint projection, 8.0 at 71.9 ms, through instantaneous-velocity matching and the consistency-interpolated model at three evaluations under per-step projection and instantaneous-velocity matching at five under endpoint projection, to instantaneous-velocity matching at twenty with a single candidate under endpoint projection, 0.2 at 291.0 ms, and the baseline, 0.0 at 654.2 ms. No model is ahead at every budget. Where the three flow-based models share a budget, the average-velocity objectives are on the frontier at its cheap end: under the tilt at one evaluation analytic average-velocity matching leaves the same 10.2 violating steps as instantaneous-velocity matching 0.2 ms sooner, and at three evaluations the consistency-interpolated model leaves the fewest under per-step projection on both constraints, 5.0 and 4.6 against 5.8 and 5.6, at about a sixth more time, and over the hump under endpoint projection 8.0 against 8.3 at the same 71.9 ms. Each of these differences is about one standard deviation of the flights. Per-step projection at one and two evaluations over the hump is off the frontier: its flights stop before the apex, most of them with no violating step.

The configurations on that frontier fall into groups that violate about equally, and within a group the comparison is the one of D3IL-avoiding: which of them reaches the finish line in fewer control steps at no more time (Figure 6.6, bottom). The groups are where the violating steps jump by more than two: under the tilt about ten (one and two evaluations), four to eight (three and five) and up to two (twenty, and the baseline); over the hump about eight (endpoint projection at three evaluations), four to six (per-step projection at three, and five) and up to two (twenty, and the baseline). Among the one- and two-evaluation configurations under the tilt the average-velocity objectives reach the line first: analytic average-velocity matching at one evaluation in 253.2 control steps at 27.4 ms, against 259.8 at 27.6 ms for instantaneous-velocity matching, and the consistency-interpolated model in 252.0 and 251.9 steps at one and two evaluations. In the middle group under the tilt instantaneous-velocity matching under per-step projection at three evaluations is the shortest, 247.2 steps at 79.2 ms, and the endpoint cells of it and of the consistency-interpolated model are about 7 ms cheaper at 253.2 to 257.8 steps. Over the hump the consistency-interpolated model under endpoint projection dominates its group, 284.6 steps against 287.7 and

299.3 at 72 ms, and instantaneous-velocity matching under per-step projection at three evaluations dominates the next, 277.0 steps at 77.1 ms. Among the configurations with the fewest violating steps, instantaneous-velocity matching at twenty evaluations is on the frontier and the baseline is not: under the tilt per-step projection, 252.6 steps at 360 ms, dominates endpoint projection (269.8, 417 ms) and the baseline (290.3, 666 ms); over the hump endpoint projection with one candidate is the cheaper, 316.2 steps at 291 ms, per-step projection the shorter, 285.1 at 355 ms, and the baseline, 320.8 steps at 654 ms, is dominated. Most of these differences are about one standard deviation of the flights; the largest, over the hump between the consistency-interpolated model and instantaneous-velocity matching under endpoint projection, is three.

Where the violating steps lie is seen in the flights themselves (Figure B.3, Section B.3), drawn for instantaneous-velocity matching, the one flow-based model flown at twenty evaluations, without projection and under per-step projection at one and at twenty evaluations. Under the tilt the unprojected flights stay level, so their margin to the leaned plane shrinks steadily along the corridor, to as much as 0.37 m inside the plane at its end. The projected flights move sideways and down until they hold a few centimetres clear of the plane, and every one of them cuts into it in the last 0.3 m of the plane's span: every violating step of every projected flight, on every model, budget and projection method, lies between $x = 1.71$ and 2.00 m. At the first violating step of each projected flow-based flight the commanded position is already past the end of the plane, at $x = 2.27$ to 2.52 m and 0.49 to 0.61 m ahead of the vehicle, and at every violating step the commanded position itself is clear of the constraint. The plane is enforced over its span of x , and on this scene the span is checked at the commanded position: once that position has left the span, the plan is released from the plane and returns toward its route, and the vehicle, half a metre behind, follows it into the plane. The violating steps under the tilt are thus the vehicle catching up with a plan that has already left the constraint, which is why the budget makes them shallower without removing them. The baseline's projected flights leave the corridor further to the side of their routes — a median of 0.37 m, against 0.19 m for the flow-based flights — and stay there, and three to eight of ten of them fly the scene without a violation.

Over the hump the unprojected flights stay level and pass through the roof. Under per-step projection at one evaluation every flight climbs along the boundary and stops 0.12 to 0.13 m short of the apex, at 1.42 to 1.43 m, where it stays until its 396 control steps run out. At one and two evaluations all three flow-based models stop this way, on 174 of their 180 flights, 0.01 to 0.17 m short of the apex; the other six clear the apex and then drift sideways across the boundary of the corridor wall. Two properties of the setup meet at the apex. The plans never rise above 1.30 m, the top of the altitude range of the demonstrations (dotted in Figure B.3), while the apex asks for 1.48 m. And each face of the roof, like the tilt, is enforced over its span of x checked at the commanded

6. Results

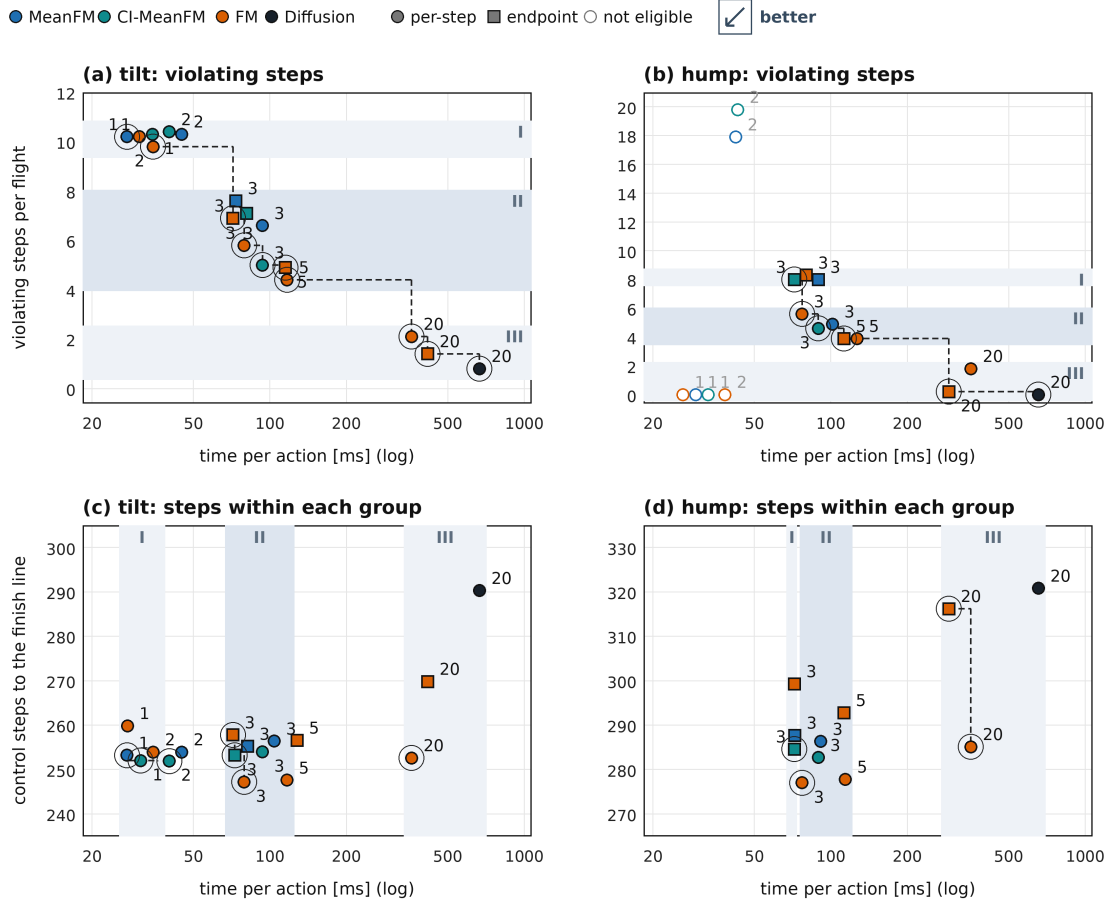


Figure 6.6.: UAV-corridor after projection, one point per model, budget and projection method at its best rule (Table 6.12); colour the model, circles per-step projection, squares endpoint projection, the number its budget. (a, b) Violating steps per flight against the time to compute one action; rings and the dashed staircase mark the non-dominated points among those reaching the end of the corridor on every flight (hollow: not). Bands I–III group configurations separated by more than two violating steps. (c, d) Within each group, control steps to the finish line against time, its non-dominated points ringed. Ten flights per point, one training seed.

position, so while that position is short of the apex the rising face binds the whole plan, including its part beyond the apex, where the face keeps rising. Both hold at every budget. At one and two evaluations the corrected plan settles short of the apex; from three on it does not, and at twenty the flights cross the apex grazing the boundary by about a centimetre and descend toward their route altitude before the line. In altitude, projection does what the two constraints ask: the lowest altitude under the tilt drops on 540 of the 560 projected flights, and the highest over the hump rises on all 560, while the unprojected flights fly level.

UAV-pillars is not drawn as a cost frontier: its result is the difference between a plan's outcome on D3IL-avoiding and the outcome of the same plan flown, which is a comparison and not a trade-off (Subsubsection 6.3.1.1). UAV-s-curve is not drawn either: most of its flights end inverted (Table 6.13), and a frontier over its configurations would rank how early they fail, so that scene gets the abort count printed beside each of its rows instead.

6.3.2.3. Projection Methods

Per-step and endpoint projection separate on this scene only at twenty evaluations (Table 6.12; every selection rule in Section B.4). At three and five evaluations endpoint projection is as cheap or cheaper, 44 to 74 ms per action against 77 to 94 at three evaluations and 73 to 115 against 112 to 118 at five, and it leaves more violating steps: at three evaluations about two more per flight under the tilt (6.9 to 9.2 against 5.0 to 7.5) and about four more over the hump (8.0 to 10.4 against 4.6 to 6.5), at five about one more. At twenty evaluations endpoint projection leaves fewer and shallower violating steps than per-step projection under both constraints: under the tilt 1.4 to 2.5 per flight, at most 1.3 cm deep, against 2.1 to 3.2 and 2.2 cm, and over the hump 0.2 to 0.7 against 1.8 to 2.2. The bar this thesis set for endpoint projection, to overtake the per-step projector of DPCC at a lower cost, is met on those two measures over the hump at twenty evaluations with a single candidate, 0.2 against 1.8 violating steps per flight at 291 against 355 ms per action, though on flights about thirty control steps longer, 316 against 285, so that neither projector dominates the other; with four candidates it leaves 0.4 to 0.7 at 422 to 425 ms. Under the tilt its fewest violating steps come with four candidates at 417 ms, more time than the per-step projector's 360 ms, and with a single candidate it is cheaper, 289 ms, but leaves 2.3.

Endpoint projection has no row at $K = 1$ or $K = 2$ on this scene and will not get one: at the activation threshold of 0.5 the only active sampling step there is the terminal one, so there is no guiding step for the method to act on (Subsubsection 6.1.2.6). That is a property of the method at those budgets, not a pending evaluation. The baseline has no endpoint rows for another reason: endpoint projection adds its control to a velocity

field (Subsection 4.5.3), which the diffusion model does not have.

The projection configuration of this scene differs from that of the other quadrotor scene (Subsubsection 5.6.3.3), so the scenes are not pooled.

6.3.2.4. Conclusion

UAV-corridor shows two things the manipulation benchmarks could not. The first is that the concept of DPCC, a generative planner whose plans a projector holds inside the constraints, carries over to a new, three-dimensional environment, against constraints that leave the horizontal plane and that no demonstration satisfies. Projected, the flights descend under the tilt (540 of 560) and climb over the hump (all 560), where the unprojected ones fly level through both, and the violating steps of a flight fall from 74.5 to 99.4 (tilt) and 28.6 to 36.6 (hump) to 1.4 to 10.4 and 0.2 to 8.3 for the flow-based models and to 0.8 and 0.0 for the baseline. What remains lies where each constraint ends, and the budget shrinks it. Under the tilt it is the vehicle catching up with a plan the constraint has already released: the plane is enforced over its span along the corridor at the commanded position, half a metre ahead of the vehicle, and once that position has left the span the plan turns back toward its route while the vehicle is still inside.

The second is what the expert data decide. The demonstrations carry little information — straight lines on three lanes at one altitude per episode, a single path forward from any position (Table 5.5) — and on them the generative model does not matter; the budget does. Without projection the flights of the three flow-based objectives cannot be told apart; with it they leave the same violating steps at a shared budget and projection method, to within 1.6 per flight, while under the tilt the budget takes them from about ten at one evaluation to one or two at twenty. The one place where projection falls short lies at the edge of the data: per-step projection at one and two evaluations does not carry a plan over the apex of the hump, at 1.48 m, above the 1.30 m that neither the plans nor the demonstrations exceed, and those flights stop short of it. Against the baseline the flow-based models trade: a twenty-fourth (tilt) and a ninth (hump) of its time per action at their cheapest, for more violating steps, at best 1.4 and 0.2 per flight against its 0.8 and 0.0. The baseline’s projected flights leave the corridor further to the side of their routes, a median of 0.37 m against 0.19 m, and take more control steps to its end, 290 and 321 against 247 to 316.

Between the two projection methods the scene shows a trade-off, and neither dominates. At twenty evaluations endpoint projection leaves fewer violating steps under both constraints: over the hump with a single candidate also at less time per action, 0.2 against 1.8 violating steps at 291 against 355 ms, but on flights about thirty control steps longer; under the tilt at more time, 1.4 against 2.1 at 417 against 360 ms. At three and

five evaluations it is as cheap or cheaper but leaves as many violating steps or more, and at one and two it has no guiding step.

6.3.3. UAV-s-curve

UAV-s-curve is read in the order of the two other scenes, with one step added. The generative models fly the single route without projection first, each flow-based model at one, two and twenty network evaluations and the diffusion baseline at the twenty it was trained with (Table 6.13). The configuration whose plans cross the finish line on the most flights is then the one the projector is applied to (Table 6.14), and its plans, unprojected and projected, are what the controller comparison of Subsubsection 6.3.3.3 flies under the two tracking controllers. Every cell is ten flights of the one route, one training seed, tracked by the cascaded geometric controller of Subsection 4.3.5 unless the row says otherwise. A flight ends in one of three ways: it crosses the finish line, it reaches the episode limit, or the divergence guard ends it because the vehicle inverted (Subsection 5.4.3). The last is the *aborted* column, and an aborted flight is never a success.

6.3.3.1. Generative Models

No flight of any cell is free of violations: the flow-based flights violate on 13.0 to 26.1 control steps on average, counted until the flight ends, and the baseline's on 119.4, two of its flights ending on the floor with close to five hundred. Instantaneous-velocity matching at one evaluation crosses the finish line on nine flights of ten, one flight ending inverted, and ends 0.571 m from the goal point on average; it is the configuration with the most crossings, followed by the same model at two evaluations (seven) and at twenty and CI-MeanFM at one (six each). For every flow-based model the crossings fall and the inversions rise from one to two to twenty evaluations: nine, seven and six crossings for instantaneous-velocity matching, six, one and one for CI-MeanFM, three, none and none for analytic average-velocity matching, whose ten flights at twenty evaluations all end inverted. The diffusion baseline at its own twenty steps crosses on none, with eight inverted. The inversions come 11.6 to 15.9 s into the flight, at control steps 386 to 530, and at the second turn: 48 of the 54 flow-based inversions lie between $x = 0.2$ and 1.2 m, just past the crossover, where the route turns into its second straight. The flights that cross reach the line at step 590 to 635. Every flight that crosses violates the constraints on the way, and the reason is in the plans. On every flight of every configuration the commanded path cuts the second inside corner of the crossover, passing 8 to 19 cm inside its keep-out, where the demonstrated route clears the constraints by 0.121 m, and that corner is where the violations lie. The plans cross

Table 6.13.: UAV-s-curve, every flow-based model at one, two and twenty network evaluations and the diffusion baseline at its own twenty, executed without projection under the cascaded geometric controller: ten flights of the single route per cell, one training seed. *Success*: crosses the finish line; *violating steps*: per flight, counted until the flight ends, and *distance*, the final distance to the goal point of the route, each mean $\pm$ the sample standard deviation over the flights; *aborted*: ended by the divergence guard; *ms/step*: time to compute one action. $\blacktriangleright$ the configuration selected for projection; $\bullet$ the baseline of record.

Model	K	Success	Violating steps	Distance [m]	Aborted	ms/step
MeanFM	1	3/10	18.5 ± 10.3	2.023 ± 1.193	7/10	9.2
MeanFM	2	0/10	16.4 ± 17.7	2.515 ± 0.796	9/10	17.9
MeanFM	20	0/10	13.0 ± 5.1	2.772 ± 0.264	10/10	176.4
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	1	6/10	23.3 ± 11.6	1.306 ± 1.319	4/10	9.3
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	2	1/10	21.3 ± 22.3	2.082 ± 1.236	7/10	18.0
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	20	1/10	16.3 ± 13.6	2.511 ± 0.843	9/10	180.4
$\blacktriangleright$ FM	1	9/10	23.0 ± 5.4	0.571 ± 0.868	1/10	8.9
FM	2	7/10	24.3 ± 13.4	1.110 ± 1.328	3/10	17.2
FM	20	6/10	26.1 ± 20.0	1.193 ± 1.257	4/10	168.3
$\bullet$ Diffusion	20	0/10	119.4 ± 199.4	2.432 ± 0.674	8/10	174.3

the gap about 0.2 m late: their commanded paths cross its middle at $x = 0.21$ to 0.27 m, where the Z-shaped route of the demonstrations crosses at $x = 0$, half a metre from both corners. The cascaded geometric controller follows the commanded path to within about a centimetre sideways on average, and it never comes more than 1.4 cm closer to the corner than the command does; the 0.30 to 0.33 m between a flow-based flight and its commanded position is the lead of that position along the path, not a sideways error.

The plans themselves show a second side of the budget (Figure 6.7). At one and two evaluations the four candidates of a plan scatter and zig-zag from one control step to the next, for both models; at twenty they lie smooth along the route. The rougher plans are not the worse ones on this scene: instantaneous-velocity matching crosses the finish line on nine flights with its roughest plans and on six with its smoothest, and every flow-based model crosses most often at one evaluation. A plan of lower quality, re-drawn at every control step, varies more from step to step, and on this route that variation appears to help the controller through the turns rather than hinder it — although, as the two panels of analytic average-velocity matching show, it does not save every model’s flights at the second corner.

6.3.3.2. Projection

The configuration Table 6.13 selects is instantaneous-velocity matching at one evaluation, and it is flown under per-step projection on the tightened constraint set. Endpoint projection guides the sample only on the sampling steps before the terminal one that the activation threshold selects (Subsection 5.6.4): at the default threshold of 0.5 that is none at one evaluation, so the method has nothing to act on at the selected budget and is not flown there.

Table 6.14.: UAV-s-curve, FM at $K = 1$, the configuration selected in Table 6.13, executed without projection and under per-step projection on the tightened constraint set, the cascaded geometric controller, ten flights per cell, one training seed. Columns as in Table 6.13; *ms/step* includes the projection. Per-step projection at random selection, the rule carried into Table 6.15: the three rules cross on five flights each, and random selection aborts least and violates least of the rules that abort least. Endpoint projection has no guiding step at $K = 1$.

Projection	Selection rule	Success	Violating steps	Distance [m]	Aborted	ms/step
none	—	9/10	23.0 ± 5.4	0.571 ± 0.868	1/10	8.9
per-step	random	5/10	71.3 ± 66.6	0.572 ± 0.345	3/10	157.1
endpoint	—	— no guiding step at $K = 1$				

- a plan: four candidates of eight waypoints
- the waypoint it is anchored at
- ✗ the flight lost control (inverted)

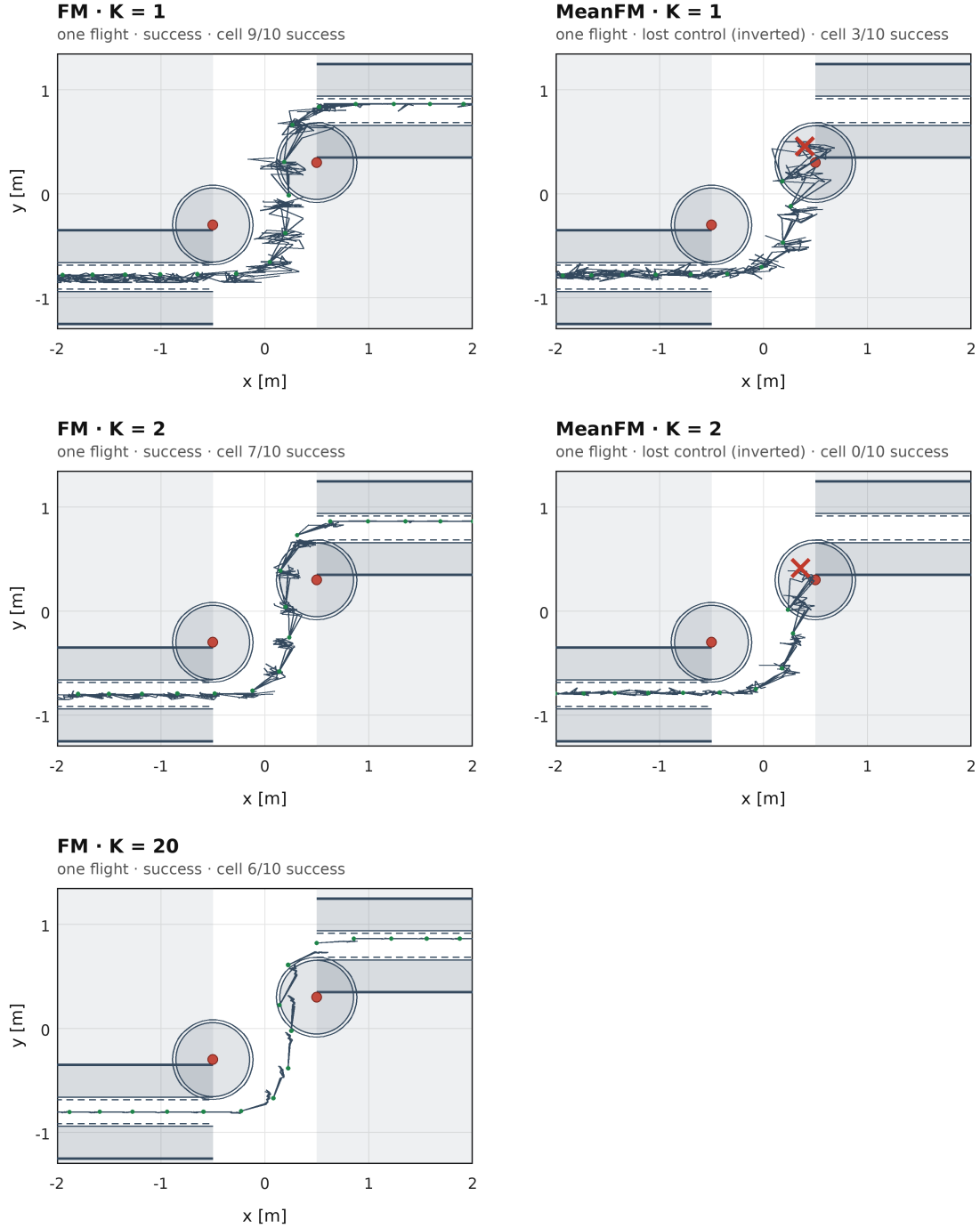


Figure 6.7.: UAV-s-curve, the plans without projection, one flight per panel, drawn like Figure 6.1 but sampled: the plan the model emitted at every twentieth control step, all four candidates, each drawn as the measured planar position of its eight waypoints, green at the waypoint it is anchored at; the executed path is not drawn. Left, FM at $K = 1, 2$ and 20 ; right, MeanFM at $K = 1$ and 2 ; the same flight of the route in every panel, the cascaded geometric controller, one training seed. A cross marks where the flight lost control.

Per-step projection does not keep the nine crossings. Under every selection rule the selected configuration crosses on five flights of ten, the aborts rise from one to three or five, and the violating steps rise from 23.0 to between 37.2 and 74.7 per flight, while the time per control step rises from 8.9 to between 136 and 164 ms, 126 to 155 of it the projection. Under random selection and cumulative projection cost two flights each drop to the floor after crossing, which is where their spread comes from. The projector does not reach what causes the violations: on this scene it binds the plan's measured positions, as DPCC does, and not the commanded position the vehicle is sent to, which UAV-corridor binds as well (Subsubsection 5.6.3.3), and the projected commanded paths still cut the second inside corner, 8 to 9 cm inside its keep-out. A per-step projector that also binds the commanded position, as on UAV-corridor, was not flown on this scene.

6.3.3.3. Caveat: The Tracking Controller

On the manipulator, inverse kinematics and the joint-space controller lie between a generated plan and the executed motion. The quadrotor is underactuated and open-loop unstable (Section 2.7), and a plan reaches the rotors only through the cascaded geometric controller that tracks it (Subsection 4.3.5). The result of a flight is therefore a property of the planner and the controller together, and UAV-s-curve, with its two sharp turns, is where the choice of controller shows. Under the cascaded geometric controller most flights of analytic average-velocity matching, of the consistency-interpolated model and of the baseline end inverted (Table 6.13), while instantaneous-velocity matching at one evaluation crosses the finish line on nine flights of ten, each with 17 to 37 violating steps. What the controller changes is tested by keeping the plan and changing the controller: MuJoCo MPC (Subsection 4.3.6) replaces the cascaded geometric controller here and nowhere else.

The comparison is made on the configuration of Table 6.14, instantaneous-velocity matching at one evaluation, with ten flights per cell (Table 6.15, Figure 6.8): its plans, unprojected and under per-step projection with random selection, under each controller. On the unprojected plans the two controllers succeed on the same nine flights of ten. MuJoCo MPC removes the one inversion and brings every flight within 0.3 m of the goal point, a mean distance of 0.297 against 0.571 m, the tenth flight being scored unsafe for contact; but it follows the commanded path less closely, 5 cm on average and up to 29 cm, the plans generated from its states cut the corner deeper, and the violating steps rise from 23.0 to 41.5 per flight.

Ten flights per cell on the selected configuration. The gains are held at their nominal values in every evaluation of this thesis and were not tuned; the two alternative settings that exist scale the position loop by $\pm 20\%$ and leave the attitude loop and the

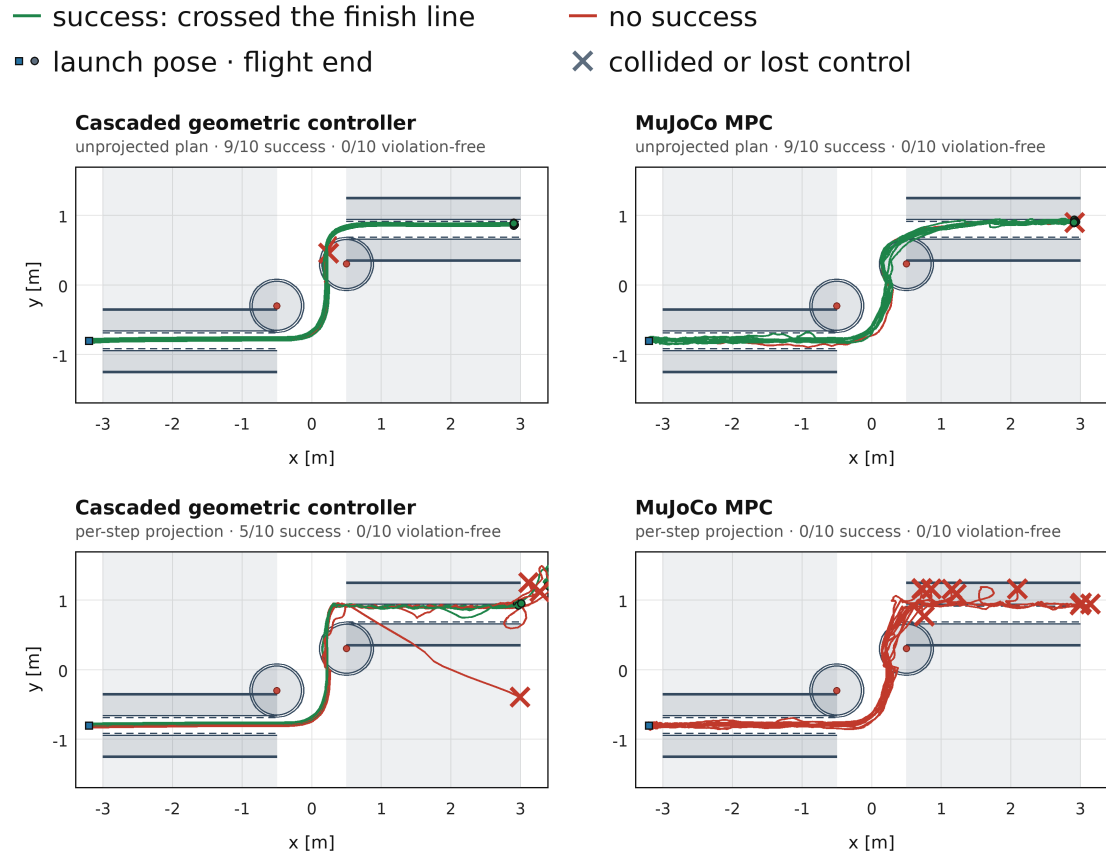


Figure 6.8.: UAV-s-curve from above, drawn like Figure 5.8: FM at $K = 1$, ten flights per panel; top, the unprojected plans; bottom, the plans under per-step projection (random selection, tightened constraint set); left, the cascaded geometric controller; right, MuJoCo MPC. Green: success; red: no success; a cross marks a flight that collided or that the divergence guard ended.

Table 6.15.: UAV-s-curve, FM at $K = 1$, the configuration selected in Table 6.13, the same plans under the two tracking controllers, ten flights per cell, one training seed. *Cascaded geometric* is the controller behind every other result in this chapter; *MuJoCo MPC* (Subsection 4.3.6) replaces it here and nowhere else. *Per-step*: the per-step projector on the tightened constraint set with random selection, first of the rules that tie on crossings in Table 6.14 by fewest aborts, then fewest violating steps. Columns as in Table 6.13.

Projection	Controller	Success	Violating steps	Distance [m]	Aborted
none	cascaded geometric	9/10	23.0 ± 5.4	0.571 ± 0.868	1/10
none	MuJoCo MPC	9/10	41.5 ± 18.4	0.297 ± 0.002	0/10
per-step	cascaded geometric	5/10	71.3 ± 66.6	0.572 ± 0.345	3/10
per-step	MuJoCo MPC	0/10	105.5 ± 46.0	1.626 ± 0.988	7/10

thrust limit untouched. What the comparison supports is that the inversions are the controller's limit, not that no tuning of this controller could fly the scene.

Under projection the cascaded geometric controller is the better of the two. It brings five of ten flights across the finish line, and MuJoCo MPC none: seven of its flights invert 15.7 to 20.7 s into the flight, pressed against the outer wall of the second straight, and the other three reach the goal region with more wall contact than the scene allows. MuJoCo MPC, which keeps every unprojected flight upright, loses every projected one. No flight of the forty is free of violations under either controller, because the corner cut is in the plans; MuJoCo MPC, following the projected plans less closely, 12 cm on average against 8.5 cm for the cascaded law, violates on 105.5 control steps per flight against 71.3.

The other reason MuJoCo MPC replaces the controller here and nowhere else is what it costs to run. The time reported in every other table of this chapter is measured around the planner and stops there (Subsection 5.4.1), so it prices no controller at all; pricing one means reading the elapsed time of the flights themselves and dividing it by the control steps that executed (Table 6.16). On the selected plans without projection, where nothing lies between the plan and the controller, a control step takes 14.0 ms under the cascaded geometric controller and 135.8 ms under MuJoCo MPC. The planner's share of that is 8.9 and 10.4 ms, so the difference belongs to the controller: about 5 ms per step for the cascaded geometric law against about 125 ms for MuJoCo MPC, which solves an optimisation problem at every control step where the other evaluates an algebraic law. The controller that flies this scene is about twenty-four times the price of the one that does not, and by itself it costs twelve times as much per control step as generating

the plan — some seven times what the operating point of D3IL-avoiding spends on an entire action (Table 6.4).

Table 6.16.: UAV-s-curve, FM at $K = 1$ without projection, the configuration selected in Table 6.13: the elapsed time of a flight divided by the control steps it executed, in milliseconds, under the two tracking controllers — the one quantity in this chapter that contains the controller. *The planner*: the planner time reported elsewhere; *the controller and the simulator step*: the remainder. Ten flights per column, one training seed.

Time per executed control step [ms]	Cascaded geometric	MuJoCo MPC
The control loop	14.0	135.8
the planner	8.9	10.4
the controller and the simulator step	5.2	125.4

This is a decomposition of recorded elapsed times and not a measurement taken around the controller call, which the evaluation does not instrument. The remainder row therefore carries the simulator step with the controller, and it is only their *difference* that isolates the controller. One training seed, ten flights against ten.

What the comparison shows is that the controller matters. In this framework a plan reaches the rotors only through the tracking controller, and the same projected plans succeed on five flights of ten or on none, and the same unprojected plans violate on 23 control steps per flight or on 42, depending on which controller flies them. On the configuration of record the cascaded geometric controller used throughout this thesis is the better one: as many successes as MuJoCo MPC without projection, five against none with it, fewer violating steps in both, at a twenty-fourth of the cost per control step. MuJoCo MPC’s one gain, keeping the unprojected flights upright and closer to the goal point, does not survive projection, and it is not adopted.

6.3.3.4. Conclusion

UAV-s-curve shows that in this framework the controller matters. A plan reaches the rotors only through the tracking controller, and on a route of two sharp turns the same plans succeed or fail by the controller that flies them. On the configuration of record the cascaded geometric controller, the one every result of this thesis is measured with, is the better of the two: without projection it succeeds on as many flights as MuJoCo MPC, nine of ten, with fewer violating steps, and under projection on five flights where MuJoCo MPC succeeds on none, at a twenty-fourth of MuJoCo MPC’s cost per control step. MuJoCo MPC’s one gain, keeping the unprojected flights upright and within

0.3 m of the goal point, does not survive projection. The violations themselves are the plan's: every commanded path cuts the second inside corner of the crossover, crossing the gap about 0.2 m later than the demonstrations do and passing 8 to 19 cm inside the corner's keep-out, and neither controller nor per-step projection, as configured, removes that cut.

Among the models instantaneous-velocity matching leads: at one evaluation it crosses the finish line on nine flights of ten, the consistency-interpolated model on six and analytic average-velocity matching on three, while most flights of the other two models and of the baseline invert at the second turn. That the simplest objective leads follows from the data: the demonstrations are one route of straight segments joined by arcs (Table 5.5), too simple to need what the average-velocity objectives are built for. The scene adds a caveat on the budget: where a flight is easy to lose, more network evaluations are not always better. For every flow-based model the crossings fall as the budget grows — from nine to seven to six for instantaneous-velocity matching, from six to one for the consistency-interpolated model and from three to none for analytic average-velocity matching — although the plans grow smoother with it (Figure 6.7): the rougher plans of one evaluation, re-drawn at every control step, vary more from step to step, and on this route that variation appears to help the controller through the turns. Instantaneous-velocity matching is sampled from initial noise of half its training scale (Subsubsection 5.6.3.1); that bears on its order against the average-velocity models, not on the controller comparison, which flies the same plans under both controllers.

6.4. Comparison across Environments

This section recaps the preceding ones and adds no new result. It answers, per environment, the first two research questions (Section 1.3): how the flow-based models compare with the diffusion baseline and with one another, and whether endpoint projection improves on the per-step projection of DPCC.

6.4.1. Generative Models

The results follow the lines along which the study was designed. On the two D3IL tasks the flow-based models Pareto-dominate the diffusion model of DPCC with its own temporal U-Net of 4.0 M parameters; on the quadrotor they carry that result to a new plant, and each scene adds what the plant or the data change.

D3IL-avoiding: Pareto dominance over the baseline. The average-velocity models Pareto-dominate the baseline. CI-MeanFM at one evaluation holds its success with constraint satisfaction of 1.000 in 59.2 control steps against 70.1, at 18.1 against

553.4 ms per action; MeanFM trades one episode in thirty (0.967) for the same saving; FM at one evaluation dominates the baseline too (1.000, 67.0 steps, 17.3 ms). Against the baseline retrained at two denoising steps the saving is still twelvefold.

D3IL-aligning: the dominance under vision and contact. On camera observations and contact, analytic average-velocity matching under per-step projection dominates the baseline’s best projected row: nine violation-free contexts against four, a median box 0.179 against 0.462 m from its target, 266 against 809 ms. Without projection every non-dominated configuration is analytic average-velocity matching, which closes 84 % of the distance at twenty evaluations against 14, 10 and -2% for the others.

UAV-pillars: transfer of the D3IL-avoiding result to the quadrotor. The Pareto-optimal operating points of D3IL-avoiding, flown again by the quadrotor: the declared constraints transfer exactly and the two average-velocity models keep their order, and the outcome departs from the table where the plant and its controller differ from the manipulator. The vehicle tracks more closely than the arm, and collisions with pillars outside the constraint sets, which end a quadrotor’s flight, cost a third of the flights: success within the constraints 0.600 to 0.700.

UAV-corridor: projection in three dimensions and the role of the demonstrations. The projection scheme of DPCC carries over to three dimensions: the violating steps of a flight fall from 29–99 to 0–10. On its straight-line demonstrations the three objectives cannot be told apart and the budget decides; against the baseline the flow-based models trade a twenty-fourth (tilt) and a ninth (hump) of its time for more violating steps.

UAV-s-curve: the role of the tracking controller. The same plans succeed or fail by the controller that flies them, and the cascaded geometric controller is the better one: as many successes as MuJoCo MPC without projection, five of ten against none with it, at a twenty-fourth of the cost. Instantaneous-velocity matching leads on this single, simple route, and more evaluations are not always better where a flight is easy to lose.

6.4.2. Projection Methods

Endpoint projection is the better projector on the constraint where the task needs a budget that gives it steps to guide, as on D3IL-aligning; on UAV-corridor the two projectors trade against each other, and where one or two evaluations solve the task endpoint projection has no step to guide.

D3IL-avoiding (operated at $K = 1$). Endpoint projection has no guiding step at the operating budget. Where it can act, with matched candidates, it is level or ahead on the constraint and cheaper on every model except instantaneous-velocity matching at three evaluations; on the average-velocity models at three it halves the time per control step.

D3IL-aligning ($K = 20$). Endpoint projection is the better projector on the constraint: all ten contexts free of violations at the time per control step of per-step projection, and more contexts violation-free in seven of nine comparisons. Its advantage is on the constraint, not the task.

UAV-pillars ($K = 1$ and 2). Per-step projection only, at the D3IL-avoiding operating points: as there, it turns plans that violate the declared constraints into plans that violate none.

UAV-corridor ($K = 1$ to 20). A trade-off, with no clear dominance. At twenty evaluations endpoint projection leaves fewer violating steps under both constraints: over the hump with one candidate also at less time, 0.2 against 1.8 at 291 against 355 ms, but on flights about thirty control steps longer; under the tilt at more time, 1.4 against 2.1 at 417 against 360 ms. At three and five evaluations it leaves as many violating steps or more.

UAV-s-curve ($K = 1$). Only per-step projection runs. It binds the measured position, leaves the corner cut of the plans in place and costs four of the nine crossings, at 126 to 155 ms of projection per step.

6.4.3. Conclusion

The study is built in three steps, and its results follow them. The main claim is made and tested on D3IL-avoiding, the benchmark of DPCC, on which the endpoint-projection work was also evaluated: there the average-velocity models Pareto-dominate the diffusion model of DPCC with its own temporal U-Net, and no endpoint-projection configuration, which needs at least three evaluations to guide the sample, improves on their operating point at one evaluation with per-step projection. D3IL-aligning extends the claim to a more complex task, camera observations and an alignment through contact, and reaffirms it. The quadrotor scenes are mainly a test of transfer: they carry the framework to a new plant and its tracking controller, and show where the result holds and what the plant, the controller and the demonstrations change.

Within that structure the results form one picture. Where the planner learns from human demonstrations, the average-velocity models lead: on D3IL-avoiding they Pareto-dominate the diffusion model of DPCC with its own temporal U-Net, in fewer

control steps at a thirtieth of its time per action, and on D3IL-aligning analytic average-velocity matching does so again, closer to the target at a third of the time. On these harder tasks the model and the projection matter together: per-step projection at one evaluation completes D3IL-avoiding, and on D3IL-aligning, which needs twenty evaluations, endpoint projection helps as the better projector on the constraint. The quadrotor scenes show the role of the demonstrations: generated for this thesis and simple — straight lanes on UAV-corridor, a single route on UAV-s-curve — they leave the objectives little to separate, so that instantaneous-velocity matching is enough; on UAV-corridor the three objectives cannot be told apart, as expected from what the average-velocity objectives are built for, and on UAV-s-curve instantaneous-velocity matching leads. UAV-corridor also shows that the projection scheme of DPCC carries over to a three-dimensional environment against constraints no demonstration satisfies, where the two projectors trade against each other without one dominating. And the quadrotor shows that the plant and its controller matter: on UAV-pillars the vehicle, tracking more closely than the arm, flies into the pillars the declared constraints leave out, and a collision ends its flight; on UAV-s-curve the same plans succeed or fail by the controller that flies them, the cascaded geometric controller used throughout the thesis being the better of the two.

Three findings cut across the environments.

1. **The price of a projection is set by what the projector is handed.** A plan close to the feasible set is cheap to repair, which is why the flow-based models at one evaluation and endpoint projection’s predicted endpoint are cheap, and the two-step diffusion baseline, handed a plan one step from noise, is not.
2. **A task saturated at one evaluation hides what a task that rewards more evaluations exposes.** The self-referential target of the consistency-interpolated model, invisible on D3IL-avoiding, stops it improving on D3IL-aligning.
3. **The demonstrations bound what a model can show.** On the straight lanes of UAV-corridor the objectives do not differ, and the plans stay within the altitudes the demonstrations cover, which is where per-step projection at the smallest budgets falls short.

The combination each environment supports, and what it achieves:

D3IL-avoiding. MeanFM or CI-MeanFM at one evaluation, per-step projection, temporal consistency: success with constraint satisfaction of 0.967 and 1.000 in 58.6 and 59.2 control steps at 18.3 and 18.1 ms per action, against the baseline’s 1.000 in 70.1 at 553.4 ms; five training seeds, the protocol of DPCC.

D3IL-aligning. MeanFM at twenty evaluations, endpoint projection, random selection: all ten contexts free of violations. The box ends in position, within the 1.8 cm of the task's own test, in none of them, but it comes close in two, 2.2 and 4.7 cm from its target; in six more it closes 40 to 75 % of the distance and ends 13 to 27 cm away, and in the last two it does not move (41 and 45 cm). The median context ends 0.197 m from the target, 57 % of the initial distance closed. The unprojected plan does not reach the tolerance either: its best two contexts end 2.8 and 2.9 cm away.

UAV-pillars. The D3IL-avoiding configurations, MeanFM or CI-MeanFM at one or two evaluations, per-step projection, temporal consistency: no violating step, and success within the constraints of 0.600 to 0.700, every failure a collision with a pillar outside the constraint sets.

UAV-corridor. Any flow-based objective, the budget chosen for the trade: violating steps per flight from 10.2 at 27.4 ms (one evaluation, per-step) to 1.4 at 416.9 ms (twenty, endpoint) under the tilt, and from 8.0 at 71.9 ms (three, endpoint) to 0.2 at 291.0 ms (twenty, endpoint, one candidate) over the hump; every flight reaches the end of the corridor.

UAV-s-curve. FM at one evaluation without projection, under the cascaded geometric controller: nine of ten flights cross the finish line, none free of violations; MuJoCo MPC crosses as often without projection and on none with it, at twenty-four times the controller's cost per step.

7. Conclusion

7.1. Summary

This thesis replaced the diffusion model of DPCC with three flow-based objectives, instantaneous-velocity matching, analytic average-velocity matching and consistency-interpolated average-velocity matching, and set endpoint projection beside the per-step projection of DPCC, keeping the temporal U-Net, the projection program, the data and the evaluation of the baseline. The step budget thereby moved from training to inference. The comparison was made in three steps: on the obstacle-avoidance benchmark of DPCC with its setup unchanged, on a vision-conditioned alignment task built for this thesis, and on a quadrotor benchmark of three scenes built for it as well.

On the benchmark of DPCC the average-velocity models Pareto-dominate its diffusion model with its own U-Net: the same success with constraint satisfaction, fewer control steps, and about a thirtieth of the time per action, twelve times less than a baseline retrained at two denoising steps. Instantaneous-velocity matching dominates the baseline as well, so the saving belongs first to a flow-based sampler at a small budget and then to the average-velocity objective, which adds the fewest control steps. On D3IL-aligning the dominance repeats where the task separates the models: analytic average-velocity matching is the only model that moves the box in every context, it closes most of the distance to the target, and it is the only objective that improves with the budget, while the consistency-interpolated model, instantaneous-velocity matching and the baseline barely move the box. Endpoint projection has no step to guide at the one-evaluation operating point of D3IL-avoiding and nothing to win there; on D3IL-aligning, run at twenty evaluations, it is the better projector on the constraint, keeping every context violation-free at the price of per-step projection.

The quadrotor scenes carry the framework to a plant that holds nothing by itself. UAV-pillars flies the operating points of D3IL-avoiding and keeps their result on the declared constraints, and shows what the plant changes: the vehicle tracks more closely than the arm, and a collision with a pillar the constraints leave out ends the flight. UAV-corridor shows that the projection scheme of DPCC carries over to three dimensions, against constraints that leave the horizontal plane and that no demonstration satisfies, and that on straight-line demonstrations the generative model does not matter, the budget does; there the two projectors trade against each other without one dominating.

UAV-s-curve shows that the controller matters: the same plans succeed or fail by the controller that flies them, and the cascaded geometric controller used throughout is the better of the two, at a twenty-fourth of the cost.

With human demonstrations the average-velocity models lead, on the harder tasks the model and the projection matter together and endpoint projection helps where the task needs a large budget; with simple generated demonstrations instantaneous-velocity matching is enough and the objectives are equivalent; and on the quadrotor the plant and its controller matter as much as the planner.

7.2. Answers to the Research Questions

RQ1 — Generative model. Yes, with a margin. With the backbone, the data and the evaluation held fixed, the flow-based models reach the diffusion baseline’s success with constraint satisfaction at one network evaluation, where the diffusion model, run away from its trained budget, loses it: on D3IL-avoiding consistency-interpolated average-velocity matching holds 1.000 in 59.2 control steps at 18.1 ms per action against the baseline’s 1.000 in 70.1 at 553.4, and instantaneous-velocity matching 1.000 in 67.0 at 17.3. Among the objectives the order, wherever the task separates them, is the average-velocity models, then instantaneous-velocity matching, then the diffusion model. On D3IL-aligning analytic average-velocity matching closes 84 % of the distance to the target against 14, 10 and -2% for the others, and under projection keeps nine contexts violation-free against the baseline’s four at a third of its time. Instantaneous-velocity matching and the diffusion model are level on outcome there and on D3IL-avoiding, and differ in cost. Between the two average-velocity objectives the consistency-interpolated one is ahead by one episode in thirty on D3IL-avoiding and behind by most of the task on D3IL-aligning, where it does not improve with the budget; the analytic objective leads or holds in every environment. Where the demonstrations are simple, on UAV-corridor and UAV-s-curve, the objectives are equivalent or instantaneous-velocity matching leads, and the budget orders the configurations instead. One evaluation saturates D3IL-avoiding, D3IL-aligning needs twenty, and on the s-curve more evaluations lose flights.

RQ2 — Constraints. Endpoint projection guides the sample only where the budget and the threshold leave a sampling step before the terminal one, $n_{\text{guide}} = \lceil \eta K \rceil - 1$: at one evaluation never, at two at the default threshold not, and there it computes the same plan as per-step projection at 2.45 times the cost. Where it has steps to guide it is the better projector on the constraint at a comparable price: on D3IL-aligning at twenty evaluations it keeps all ten contexts violation-free where

per-step projection keeps nine, in seven of nine matched comparisons, and at ten evaluations at 37 % of per-step projection's time; on D3IL-avoiding at three evaluations it halves the time per control step of the two average-velocity models at 1.000. It does not reach the goal in fewer control steps, and on UAV-corridor it trades: fewer and shallower violations at twenty evaluations, cheaper over the hump with one candidate and dearer under the tilt, on longer flights. It helps on the hard task that needs the budget, D3IL-aligning, and not on D3IL-avoiding, which is solved at a budget where it has no step to act.

RQ3 — Transfer. The answers hold where the demonstrations are of the same kind, and the plant and the demonstrations change the rest. Analytic average-velocity matching leads again on camera observations and contact, and the declared constraints transfer exactly from the manipulator to the quadrotor on UAV-pillars, where the two average-velocity models keep their order. The quadrotor adds what the manipulator could not show: an undeclared obstacle ends a flight, the controller decides whether a plan on a route of sharp turns is flown at all, and on demonstrations that carry one path forward from every position the objectives cannot be told apart. The projection scheme itself carries over to three dimensions and to constraints that leave the horizontal plane.

8. Discussion

What the results of Chapter 6 do not cover is stated here: the limits of the framework (Section 8.1), what a deployment has to add to it (Section 8.2), and the work that follows from where the results stop (Section 8.3).

8.1. Limitations

The plan is a sequence of positions. The generative model plans position increments and never an actuator command; everything below the increment is the plant's controller (Subsection 4.3.3). The projection's dynamics model is a first-order integrator over the commanded quantity, so feasibility is a statement about that model and its mismatch bound, which the tightening absorbs, and not about the plant. On the manipulator this holds, because the end effector tracks its setpoint within the margin; on the quadrotor whether a plan can be flown is the controller's, which is what UAV-s-curve shows.

A constraint is what is declared. The projector holds a plan inside the declared geometry, halfspaces and keep-out regions, and protects nothing else. On UAV-pillars five of the six pillars are in no constraint set and a third of the flights end on them. Whether a scene is safe follows from its constraint set and not from the projection; the remedy is a constraint per obstacle, or a margin beyond the rotor reach, and it is a decision of scene design. The span over which a wall or a plane acts is checked at the commanded position, half a metre ahead of the vehicle on the corridor, which leaves the hand-over residue at the end of every plane.

Endpoint projection is defined on a velocity field. It adds its control to the field of a flow-based sampler and has no counterpart for the diffusion model, whose reverse chain has none (Subsection 4.5.3). The comparison of the two projection methods is therefore made within the flow-based family, and the diffusion baseline is read under per-step projection alone.

Plan quality is read through violating steps and the drawn plans. The plans are scored by success, constraint satisfaction, control steps and time, and inspected as

drawn (Figure 6.1, Figure 6.7). No curvature, jerk or path-length measure is taken over the stored plans, so the ribbon of the average-velocity models against the scribble of the one-step baseline, and the rough low-budget plans that cross the s-curve most often, are read from the drawings and the counts beside them.

Scope of the transfer. The vision-conditioned task adds camera observations and contact to the manipulator in one step; the state-observed form of the alignment task was not run, so what D3IL-aligning shows is that the result survives a harder, differently observed task, and not the step from state to vision alone. The quadrotor is observed through its state, at one altitude on UAV-pillars, where the plan carries no altitude, and within the 0.90 to 1.30 m of the demonstrations elsewhere. D3IL-aligning, UAV-corridor and UAV-s-curve are evaluated on one training seed, D3IL-avoiding and UAV-pillars on five (Table 5.10). No loop of this thesis was run in real time, and no environment is a physical system.

8.2. Towards Deployment

Three properties of the framework carry into practice as it stands. The constraints enter only at sampling time, through the projection; nothing about them reaches training, so a new restriction is a change of the constraint set and not a training run, and the feasible set can be replaced between two replans, as the switched walls of the quadrotor scenes are (Subsection 4.6.3). The step budget is a deployment choice: the same checkpoint of a flow-based model is read at one evaluation on D3IL-avoiding and at twenty on D3IL-aligning, where a diffusion model needs one checkpoint per budget. And the plan is a sequence of positions that a tracking controller follows, which is what let one planner fly the quadrotor unchanged.

Two quantities bound that use. A plan is held inside the declared geometry only, so the constraint set has to name every obstacle that matters, with a margin that covers the vehicle's extent, and the channel it leaves has to be flyable by the controller: on the corridor the commanded position runs 0.49 to 0.61 m ahead of the vehicle, on the s-curve about 0.3 m, and the vehicle follows the commanded path to within about a centimetre sideways. And the time to compute one action, 18 ms at the operating point of D3IL-avoiding and 266 to 275 ms on D3IL-aligning on the node of record, prices the planner alone; the execution adds its own, 5 ms per control step for the cascaded geometric controller and 125 ms for MuJoCo MPC on the s-curve, and the manipulator's inverse kinematics and joint controller were not timed. No loop of this thesis was run in real time; the simulation waits for the planner (Subsection 4.3.5).

What a deployment can leave out is the candidate machinery of DPCC. With a single

candidate plan the three selection rules select the same plan, and success and control steps are identical in all fifteen seed-geometry cells of every model on D3IL-avoiding; with four candidates the generation time changes by at most six per cent, because the four are one batched network evaluation, and the projection time grows by a factor of 3.2 to 4.9, because the four programs are solved one after another on the CPU. On UAV-corridor endpoint projection with one candidate costs 291 ms per action against 422 to 425 ms with four, and leaves 0.2 violating steps against 0.4 to 0.7. Where four candidates are kept, the rule matters by model: instantaneous-velocity matching holds 1.000 under all three rules, while the two average-velocity models are operated with temporal consistency, since cumulative projection cost, the rule of DPCC, occasionally selects a plan that stalls and costs them control steps, 72.4 and 97.2 against 58.6 and 59.4 for analytic average-velocity matching at one and two evaluations. Solving the four programs in parallel would take that model at one evaluation from 18.1 to about 11.7 ms per control step, a prediction from two measured points.

8.3. Future Work

Demonstrations for the quadrotor. The demonstrations of UAV-corridor and UAV-s-curve are generated, one path forward from every position, and the s-curve's plans cut the corner that the demonstrated route clears. Human demonstrations of the quadrotor scenes, from a pilot in the loop of a simulator or from flight logs, would give the average-velocity objectives the multimodal data on which they separate and the s-curve a route its plans do not cut; NVIDIA Isaac Sim, a simulator with a rendered scene and a pilot interface, is the way to record them without a vehicle.

The controller. On the s-curve the projector binds the measured position while the corridor's binds the commanded one; the projector that binds both was not flown there. A plan that carries a velocity setpoint, or a controller that reads the next positions of the plan as feed-forward, would close the lead between the commanded and the flown position from which the hand-over residue and the corner cut come.

Real-time execution. The execution stack has to be timed and the loop run without waiting for the planner: the planner at one evaluation, the projector solved in parallel over the candidates, or with a single candidate where the rule selects the same plan, and the controller at its own rate.

Constraints from perception. The projector consumes a short list of primitives with metric extent, halfspaces, keep-out disks and spheres and a workspace box, each

in the planner’s frame and each with its feasible side. A perception front-end that emits that list from depth or range sensing, and a map that keeps it beyond the field of view, is what a deployment outside the simulator needs; the uncertainty of the detector belongs in the inflation margin the projector already carries. The first experiment needs no detector: perturb the geometry the projector receives, by an offset, a radius error, a missing or a spurious primitive, and record where success with constraint satisfaction breaks.

Re-tasking at deployment. Because the constraints never enter training, the same checkpoint can be asked to satisfy a new set: a closed passage, a keep-out volume, a tighter clearance. What has to be measured is the envelope, the slack a new set may leave before the projector pulls the plan off the data and success collapses rather than degrades, and the transfer of one checkpoint to a geometry its demonstrations never contained.

The alignment target. The optional terminal cost of endpoint projection (Subsection 4.5.3), zero in this thesis, is where the target of D3IL-aligning could enter the projection; constraint satisfaction would then stop costing completion.

The sampler of instantaneous-velocity matching. Sampling it from the scale it was trained at needs no retraining and would settle its order against the average-velocity models on D3IL-aligning and the quadrotor scenes.

Seeds. More training seeds on D3IL-aligning and the quadrotor scenes.

Appendix

A. Supplementary Material

A.1. Training-Time Sampling Laws

The diffusion baseline draws a discrete denoising index uniformly, $k \sim \text{Uniform}\{0, \dots, K - 1\}$, where K is the number of diffusion steps. This is not a draw from the continuous transport time used by the flow-based models.

For instantaneous-velocity matching, let $U \sim \text{Beta}(1.5, 1)$ and set the transport time to $\tau = 1 - U$, on the axis where $\tau = 0$ is noise and $\tau = 1$ is data. Equivalently,

$$\tau \sim \text{Beta}(1, 1.5), \quad p(\tau) = 1.5(1 - \tau)^{1/2}, \quad 0 < \tau < 1. \quad (\text{A.1})$$

Thus the implemented draw weights the noise end of this transport axis. The expression $\text{Beta}(1.5, 1)$ in the training configuration describes U *before* the reflection, not the time passed to the network.

Analytic and consistency-interpolated average-velocity matching each draw two independent standard normal variables Z_1, Z_2 and transform them by

$$T_i = \frac{1}{1 + \exp[-(P_{\text{std}}Z_i - P_{\text{mean}})]}, \quad P_{\text{mean}} = -0.4, \quad P_{\text{std}} = 1.0, \quad i \in \{1, 2\}. \quad (\text{A.2})$$

On the same noise-at-zero axis, $\text{logit}(T_i)$ therefore has mean $+0.4$ and standard deviation 1.0 . The ordered pair is $r = \min(T_1, T_2)$ and $t = \max(T_1, T_2)$. For each example, independently with probability 0.5 , the implementation then sets $r = t$, giving a zero-length interval and an instantaneous-velocity anchor; otherwise the interval has length $t - r > 0$. Before ordering, the logit-normal density of each T_i is

$$p_T(u) = \frac{1}{u(1-u)\sqrt{2\pi}} \exp\left[-\frac{(\text{logit}(u) - 0.4)^2}{2}\right], \quad 0 < u < 1. \quad (\text{A.3})$$

These laws are the samplers behind the entries in Table 5.8; the average-velocity pair and the losses to which it is supplied are defined in Section 4.4.

A.2. Quadrotor Dimensions

The dimensions of the Skydio X2 that enter the quadrotor scenes, the rotor disks, the collision footprint and the rotor reach of 0.31 m by which every constraint surface is inflated (Subsection 5.2.3), are drawn to scale in Figure A.1.

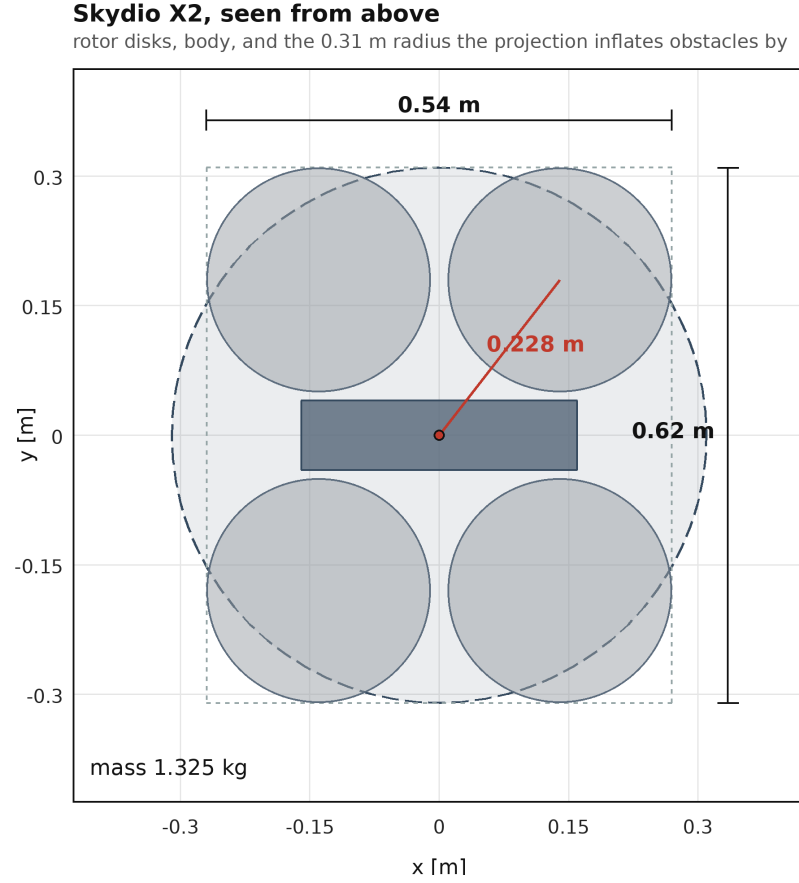


Figure A.1.: The Skydio X2 rotor disks (radius 0.13 m), body and 0.54×0.62 m collision footprint (dotted), drawn to scale. The dashed circle is the 0.31 m rotor reach used to inflate the projection constraints.

B. Extended Results

This appendix records the data behind Appendix 6 that the chapter reads without printing: the executed paths of D3IL-avoiding (Section B.1) and D3IL-aligning (Section B.2), the flights of UAV-corridor along the corridor (Section B.3), UAV-corridor under every selection rule (Section B.4), and D3IL-avoiding at twenty episodes per seed (Section B.5). Section B.4 is a data record and keeps its reading here.

B.1. D3IL-avoiding: Executed Paths under Per-Step Projection

Figure B.1 shows the executed end-effector path of episode 2 for each of the two average-velocity models at $K = 1$ and $K = 2$. The four paths share an initial pose, training seed and constraint geometry. All eight episodes across the four cells reach the goal without a violating control step; the figure draws one matched episode per cell. Each displayed path bends around the keep-out disk and crosses the goal line. The unprojected plans for these model-budget cells are shown in Figure 6.1; their constraint violations are reported in Table 6.1.

One training seed and one geometry. The protocol runs these two episodes for each of five training seeds on each of three geometries (Table 5.10); only one seed’s rollout artefacts are on disk, so this figure draws two of the thirty episodes a row of Table 6.2 is averaged over, and the counts beside the panels are not that row’s rates.

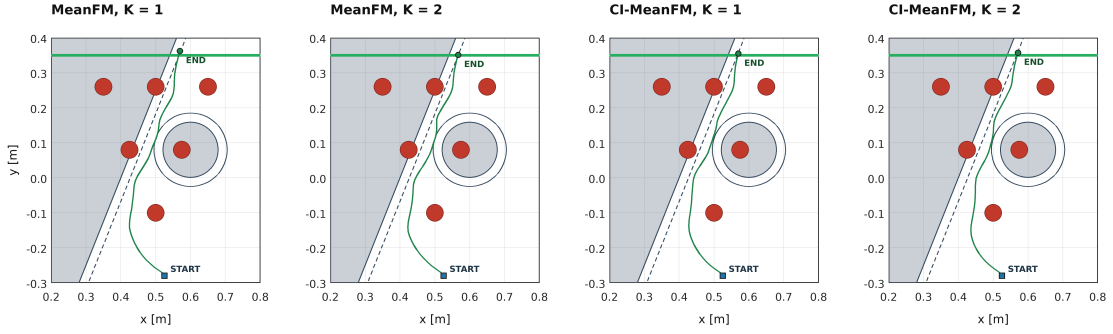


Figure B.1.: D3IL-avoiding, the executed end-effector path of episode 2 in each model-budget cell. Left to right: MeanFM at $K = 1$ and 2, then CI-MeanFM at $K = 1$ and 2. The blue square marks the common start; the green circle marks where each episode ended beyond the goal line. Green paths have no violating control step. Per-step projection with temporal consistency, tightened *top-right-hard* constraints and one training seed; both evaluated episodes reached the goal without a violation in every cell. Shading shows the excluded halfspace and keep-out disk; the dashed line and circle are the tightened boundaries, as in Figure 5.9. The view is x - y only.

B.2. D3IL-aligning: Executed Paths under Each Projection Method

Figure B.2 shows nine of the ten evaluated contexts under each projection setting; the tenth, context 7, is the one no configuration moves (Subsection 6.2.2) and its logged path has no extent. Without projection eight of the nine paths enter the keep-out region or cross the tightened boundary; under per-step projection one still does, and under endpoint projection none. Over all ten contexts the unprojected setting has two violation-free paths, per-step projection nine and endpoint projection ten (Table 6.7), and the panel counts agree with those once the unmoved context is added back. The projected paths are visibly longer than their unprojected counterparts.

This draws the end effector, not the box. The constraint set of this task applies to the planned end-effector position, so the drawn quantity is the one the projector acts on — but the quadrotor figures draw the constrained *body*, and the equivalent here would be the box. The evaluation does not log a box pose: `eval_mix_visual_aligning.py` writes a six-dimensional `obs_all` that is the commanded and the measured end-effector position, and the box enters the record only as the distances of Table 6.7. Drawing the box would need an evaluation re-run with that pose recorded, which is

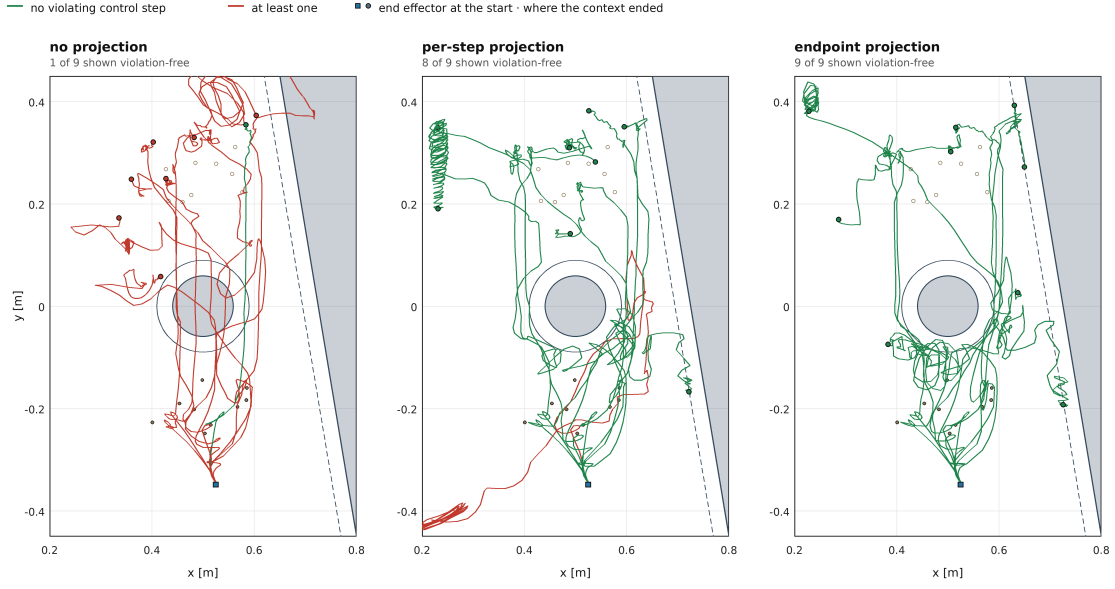


Figure B.2.: D3IL-aligning, the executed end-effector path in nine of the ten contexts under the three projection settings of Table 6.7, over the constraint set of Table 5.3; analytic average-velocity matching at $K = 20$, random selection, tightened constraints, one training seed. Context 7, whose path has no extent because the arm never moves, is left out of every panel; the full ten-context counts are in Table 6.7. Only the x - y position is plotted; height z is omitted. Green: no violating control step in that context; red: at least one. The filled square is the end effector at the start, common to every context, and a dot marks where a context ended. The small marks are the nine box start poses (filled) and their targets (hollow), as in Figure 5.6. Shaded: the region excluded for the planned position; dashed: the tightened boundary. Panel counts refer only to the paths shown.

data_status/PENDING_20260922_all_lacking_runs.md, D10c.

B.3. UAV-corridor: The Flights along the Corridor

Figure B.3 draws the flights of instantaneous-velocity matching at one and twenty network evaluations along the corridor, without projection and under per-step projection, against both test-time constraints; Subsubsection 6.3.2.2 reads them.

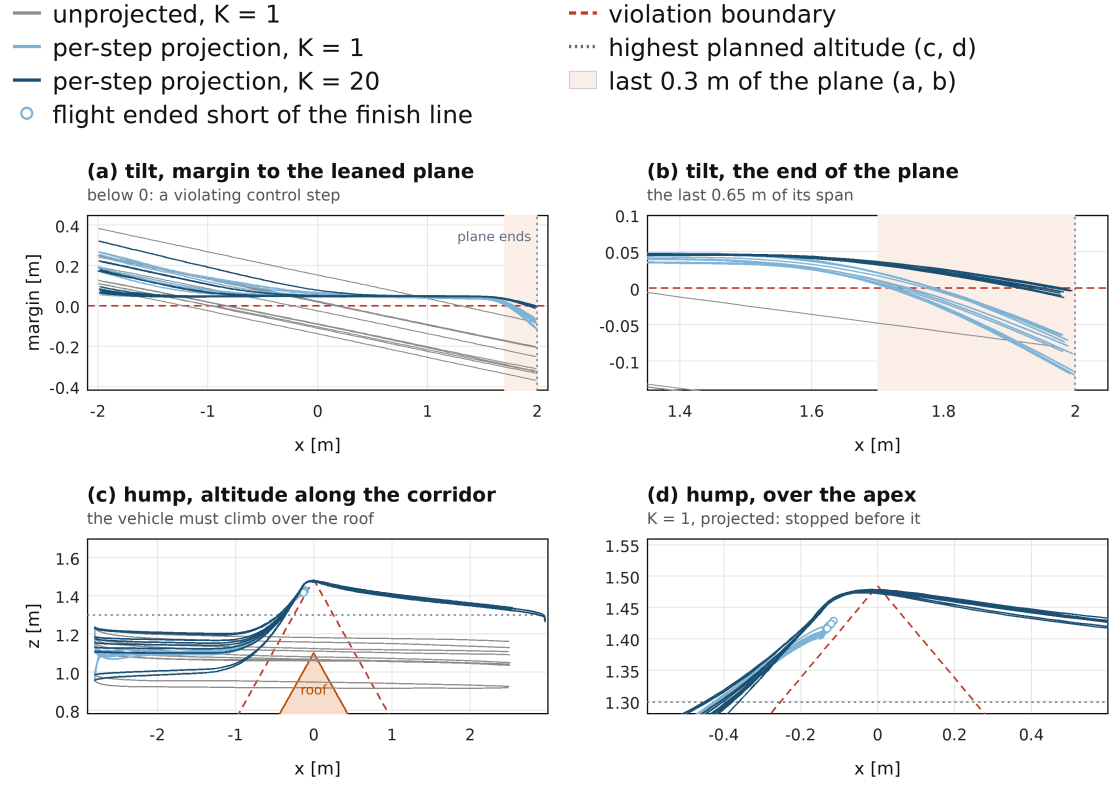


Figure B.3.: UAV-corridor along the corridor, instantaneous-velocity matching, ten flights per series, one training seed: without projection at $K = 1$ (grey) and under per-step projection with cumulative-cost selection at $K = 1$ and $K = 20$ (blue). (a, b) Tilt: the margin of the flown position to the leaned plane, less the rotor reach, over the plane's span of x ; a control step below the dashed line violates. (c, d) Hump: altitude over x , the roof shaded, the violation boundary dashed, the highest planned altitude dotted; a circle ends a flight short of the finish line.

B.4. UAV-corridor under Every Selection Rule

Table 6.12 prints each model at each budget at its best selection rule under each projection method. Under every constraint, projection method and budget the three rules of a flow-based model lie within two violating steps per flight and within about fifteen control steps of one another, cumulative projection cost leaving the fewest violating steps in most cells; the baseline’s rules differ by up to five violating steps under the tilt, 0.8 under cumulative projection cost against 6.1 under temporal consistency, and its cumulative-cost rule over the hump loses one flight of ten. Every flight reaches the end of the corridor except under per-step projection at one and two evaluations over the hump. Table 6.12 prints each model at each budget at its best selection rule under each projection method: the rule with the most successes, then the fewest violating steps, then the least time. The four tables below print every rule, per constraint and projection method. Success is reaching the end of the corridor within the episode limit, and the constraint is read as violating control steps per flight (Subsection 5.4.3).

B. Extended Results

Table B.1.: UAV-corridor, *tilt*, per-step projection of DPCC under each selection rule (r random, c cumulative projection cost, t temporal consistency; four candidate plans), every budget of the grid; tightened constraints, activation threshold 0.5, ten flights per cell, one training seed. Columns as in Table 6.12, each mean $\pm$ the sample standard deviation over the flights. At the route’s goal point the baseline’s success is 0.200, 0.000 and 0.300. • the diffusion model of DPCC.

Model	K	Rule	Success	Violating steps	Steps	ms/step
MeanFM	1	r	1.000 ± 0.000	10.4 ± 1.1	260.2 ± 4.9	28.5 ± 0.8
MeanFM	1	c	1.000 ± 0.000	10.2 ± 1.1	253.2 ± 4.4	27.4 ± 0.5
MeanFM	1	t	1.000 ± 0.000	10.3 ± 1.1	249.2 ± 5.0	28.7 ± 1.0
MeanFM	2	r	1.000 ± 0.000	11.0 ± 1.2	262.1 ± 4.4	37.6 ± 0.9
MeanFM	2	c	1.000 ± 0.000	10.3 ± 0.9	253.9 ± 4.2	36.4 ± 0.8
MeanFM	2	t	1.000 ± 0.000	10.6 ± 0.8	255.6 ± 5.1	37.6 ± 1.0
MeanFM	3	r	1.000 ± 0.000	7.5 ± 0.8	257.1 ± 2.7	94.3 ± 1.5
MeanFM	3	c	1.000 ± 0.000	6.6 ± 0.8	256.4 ± 2.9	93.6 ± 1.1
MeanFM	3	t	1.000 ± 0.000	7.2 ± 1.2	261.4 ± 1.8	93.4 ± 1.3
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	1	r	1.000 ± 0.000	10.5 ± 1.2	259.2 ± 3.5	29.4 ± 0.8
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	1	c	1.000 ± 0.000	10.3 ± 0.9	252.0 ± 3.7	27.9 ± 0.5
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	1	t	1.000 ± 0.000	10.3 ± 1.3	247.2 ± 3.0	29.0 ± 0.8
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	2	r	1.000 ± 0.000	10.5 ± 1.4	258.7 ± 4.4	37.6 ± 0.9
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	2	c	1.000 ± 0.000	10.4 ± 0.7	251.9 ± 2.4	36.1 ± 0.3
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	2	t	1.000 ± 0.000	10.6 ± 1.4	253.6 ± 4.6	37.3 ± 0.7
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	r	1.000 ± 0.000	6.6 ± 1.0	255.3 ± 3.7	93.6 ± 1.8
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	c	1.000 ± 0.000	5.0 ± 0.8	254.0 ± 1.8	93.4 ± 1.2
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	t	1.000 ± 0.000	7.0 ± 1.2	258.0 ± 2.4	93.0 ± 1.2
FM	1	r	1.000 ± 0.000	10.2 ± 0.8	257.9 ± 9.6	28.1 ± 2.0
FM	1	c	1.000 ± 0.000	10.2 ± 1.5	259.8 ± 9.2	27.6 ± 1.5
FM	1	t	1.000 ± 0.000	10.9 ± 1.0	256.9 ± 21.3	27.5 ± 1.2
FM	2	r	1.000 ± 0.000	10.4 ± 1.1	257.2 ± 8.4	34.8 ± 1.1
FM	2	c	1.000 ± 0.000	9.8 ± 1.4	253.9 ± 9.4	34.7 ± 0.5
FM	2	t	1.000 ± 0.000	11.1 ± 0.9	246.6 ± 12.3	34.6 ± 0.4
FM	3	r	1.000 ± 0.000	6.7 ± 1.5	247.1 ± 3.2	78.8 ± 3.2
FM	3	c	1.000 ± 0.000	5.8 ± 1.5	247.2 ± 3.4	79.2 ± 3.2
FM	3	t	1.000 ± 0.000	6.7 ± 1.2	241.1 ± 2.3	78.9 ± 2.9
FM	5	r	1.000 ± 0.000	5.4 ± 1.4	248.1 ± 2.0	117.8 ± 2.5
FM	5	c	1.000 ± 0.000	4.4 ± 1.5	247.6 ± 2.0	116.9 ± 2.4
FM	5	t	1.000 ± 0.000	5.3 ± 1.3	242.7 ± 1.3	116.8 ± 2.8
FM	20	r	1.000 ± 0.000	3.0 ± 0.9	253.3 ± 3.7	360.0 ± 1.1
FM	20	c	1.000 ± 0.000	2.1 ± 0.7	252.6 ± 2.7	360.4 ± 1.5
FM	20	t	1.000 ± 0.000	3.2 ± 0.8	247.7 ± 4.4	358.4 ± 1.3
• Diffusion	20	r	1.000 ± 0.000	3.2 ± 3.3	267.7 ± 37.7	679.7 ± 30.9
• Diffusion	20	c	1.000 ± 0.000	0.8 ± 1.8	290.3 ± 15.5	666.3 ± 16.7
• Diffusion	20	t	1.000 ± 0.000	6.1 ± 4.6	257.2 ± 45.7	650.6 ± 24.2

Table B.2.: UAV-corridor, *tilt*, endpoint projection with one candidate plan (*single*) and under the three rules over four, at the budgets at which it has a guiding step; tightened constraints, activation threshold 0.5, ten flights per cell, one training seed. Columns as in Table 6.12, each mean $\pm$ the sample standard deviation over the flights.

Model	K	Rule	Success	Violating steps	Steps	ms/step
MeanFM	3	single	1.000 $\pm$ 0.000	9.2 $\pm$ 1.4	256.6 $\pm$ 4.7	46.2 $\pm$ 0.6
MeanFM	3	<i>r</i>	1.000 $\pm$ 0.000	8.5 $\pm$ 0.8	257.3 $\pm$ 5.3	73.9 $\pm$ 0.5
MeanFM	3	<i>c</i>	1.000 $\pm$ 0.000	7.6 $\pm$ 1.3	255.2 $\pm$ 4.5	73.5 $\pm$ 0.8
MeanFM	3	<i>t</i>	1.000 $\pm$ 0.000	9.0 $\pm$ 1.2	261.6 $\pm$ 5.1	72.5 $\pm$ 0.2
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	single	1.000 $\pm$ 0.000	8.8 $\pm$ 0.8	254.4 $\pm$ 5.0	46.2 $\pm$ 0.4
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	<i>r</i>	1.000 $\pm$ 0.000	9.0 $\pm$ 0.9	254.6 $\pm$ 4.4	73.2 $\pm$ 0.6
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	<i>c</i>	1.000 $\pm$ 0.000	7.1 $\pm$ 1.0	253.2 $\pm$ 3.5	72.7 $\pm$ 0.5
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	<i>t</i>	1.000 $\pm$ 0.000	8.7 $\pm$ 1.5	259.2 $\pm$ 3.9	72.8 $\pm$ 0.2
FM	3	single	1.000 $\pm$ 0.000	8.0 $\pm$ 1.6	254.8 $\pm$ 3.2	43.8 $\pm$ 0.2
FM	3	<i>r</i>	1.000 $\pm$ 0.000	8.2 $\pm$ 1.5	256.6 $\pm$ 1.6	71.4 $\pm$ 0.5
FM	3	<i>c</i>	1.000 $\pm$ 0.000	6.9 $\pm$ 1.8	257.8 $\pm$ 3.3	71.6 $\pm$ 0.6
FM	3	<i>t</i>	1.000 $\pm$ 0.000	8.2 $\pm$ 1.4	248.2 $\pm$ 5.2	71.4 $\pm$ 0.9
FM	5	single	1.000 $\pm$ 0.000	6.0 $\pm$ 1.7	255.6 $\pm$ 4.2	73.6 $\pm$ 0.2
FM	5	<i>r</i>	1.000 $\pm$ 0.000	6.0 $\pm$ 1.4	257.4 $\pm$ 3.9	113.0 $\pm$ 0.7
FM	5	<i>c</i>	1.000 $\pm$ 0.000	4.9 $\pm$ 1.9	256.6 $\pm$ 2.9	115.2 $\pm$ 1.4
FM	5	<i>t</i>	1.000 $\pm$ 0.000	6.3 $\pm$ 1.3	250.3 $\pm$ 5.3	115.2 $\pm$ 0.3
FM	20	single	1.000 $\pm$ 0.000	2.3 $\pm$ 0.5	269.9 $\pm$ 10.5	288.9 $\pm$ 1.4
FM	20	<i>r</i>	1.000 $\pm$ 0.000	2.3 $\pm$ 0.9	269.3 $\pm$ 10.7	416.8 $\pm$ 1.4
FM	20	<i>c</i>	1.000 $\pm$ 0.000	1.4 $\pm$ 0.7	269.8 $\pm$ 10.5	416.9 $\pm$ 0.8
FM	20	<i>t</i>	1.000 $\pm$ 0.000	2.5 $\pm$ 0.7	260.5 $\pm$ 10.6	417.2 $\pm$ 0.6

B. Extended Results

Table B.3.: UAV-corridor, *hump*, per-step projection of DPCC under each selection rule (r random, c cumulative projection cost, t temporal consistency; four candidate plans), every budget of the grid; tightened constraints, activation threshold 0.5, ten flights per cell, one training seed. Columns as in Table 6.12, each mean $\pm$ the sample standard deviation over the flights. —: no flight reaches the end of the corridor. At the route’s goal point the baseline’s success is 0.300, 0.300 and 0.700. • the diffusion model of DPCC.

Model	K	Rule	Success	Violating steps	Steps	ms/step
MeanFM	1	r	0.000 ± 0.000	0.0 ± 0.0	—	26.5 ± 0.2
MeanFM	1	c	0.000 ± 0.000	0.0 ± 0.0	—	26.4 ± 0.0
MeanFM	1	t	0.000 ± 0.000	0.0 ± 0.0	—	26.6 ± 0.2
MeanFM	2	r	0.000 ± 0.000	0.0 ± 0.0	—	35.0 ± 0.4
MeanFM	2	c	0.000 ± 0.000	0.0 ± 0.0	—	35.2 ± 0.3
MeanFM	2	t	0.100 ± 0.316	17.9 ± 30.0	387.0	42.3 ± 10.9
MeanFM	3	r	1.000 ± 0.000	5.9 ± 0.7	288.5 ± 3.7	88.9 ± 1.8
MeanFM	3	c	1.000 ± 0.000	4.9 ± 0.9	286.3 ± 4.3	91.1 ± 1.8
MeanFM	3	t	1.000 ± 0.000	6.2 ± 0.6	288.0 ± 4.6	89.8 ± 1.9
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	1	r	0.000 ± 0.000	0.0 ± 0.0	—	26.7 ± 0.3
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	1	c	0.000 ± 0.000	0.0 ± 0.0	—	26.7 ± 0.3
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	1	t	0.000 ± 0.000	0.0 ± 0.0	—	26.6 ± 0.3
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	2	r	0.000 ± 0.000	0.0 ± 0.0	—	35.4 ± 0.2
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	2	c	0.000 ± 0.000	0.0 ± 0.0	—	35.6 ± 0.3
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	2	t	0.200 ± 0.422	19.8 ± 32.1	388.0 ± 2.8	43.0 ± 12.2
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	r	1.000 ± 0.000	5.0 ± 1.1	287.1 ± 4.1	89.6 ± 1.7
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	c	1.000 ± 0.000	4.6 ± 0.8	282.7 ± 3.6	89.2 ± 1.9
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	t	1.000 ± 0.000	6.5 ± 0.5	285.8 ± 2.9	89.2 ± 2.0
FM	1	r	0.000 ± 0.000	0.0 ± 0.0	—	26.3 ± 0.4
FM	1	c	0.000 ± 0.000	0.0 ± 0.0	—	26.2 ± 0.2
FM	1	t	0.000 ± 0.000	0.0 ± 0.0	—	26.3 ± 0.3
FM	2	r	0.000 ± 0.000	0.0 ± 0.0	—	34.5 ± 0.4
FM	2	c	0.000 ± 0.000	0.0 ± 0.0	—	34.6 ± 0.0
FM	2	t	0.000 ± 0.000	0.0 ± 0.0	—	34.7 ± 0.4
FM	3	r	1.000 ± 0.000	5.9 ± 0.3	278.7 ± 6.1	76.8 ± 2.7
FM	3	c	1.000 ± 0.000	5.6 ± 0.5	277.0 ± 7.3	77.1 ± 2.7
FM	3	t	1.000 ± 0.000	5.8 ± 0.4	269.5 ± 4.0	77.2 ± 2.5
FM	5	r	1.000 ± 0.000	4.2 ± 0.4	279.1 ± 4.6	113.2 ± 2.8
FM	5	c	1.000 ± 0.000	3.9 ± 0.7	277.8 ± 7.0	114.1 ± 2.3
FM	5	t	1.000 ± 0.000	4.3 ± 0.5	269.7 ± 3.5	112.1 ± 2.3
FM	20	r	1.000 ± 0.000	2.2 ± 0.8	286.4 ± 3.5	354.3 ± 2.6
FM	20	c	1.000 ± 0.000	1.8 ± 0.4	285.1 ± 5.6	355.2 ± 2.2
FM	20	t	1.000 ± 0.000	2.2 ± 0.4	277.2 ± 2.3	356.3 ± 2.7
• Diffusion	20	r	1.000 ± 0.000	0.0 ± 0.0	320.8 ± 53.6	654.2 ± 89.7
• Diffusion	20	c	0.900 ± 0.316	0.0 ± 0.0	345.8 ± 22.8	623.1 ± 87.8
• Diffusion	20	t	1.000 ± 0.000	0.4 ± 1.3	308.8 ± 47.4	625.9 ± 75.0

B. Extended Results

Table B.4.: UAV-corridor, *hump*, endpoint projection with one candidate plan (*single*) and under the three rules over four, at the budgets at which it has a guiding step; tightened constraints, activation threshold 0.5, ten flights per cell, one training seed. Columns as in Table 6.12, each mean $\pm$ the sample standard deviation over the flights.

Model	K	Rule	Success	Violating steps	Steps	ms/step
MeanFM	3	single	1.000 ± 0.000	10.1 ± 2.1	301.3 ± 4.6	45.9 ± 0.2
MeanFM	3	r	1.000 ± 0.000	8.6 ± 2.5	303.0 ± 3.9	72.4 ± 0.6
MeanFM	3	c	1.000 ± 0.000	8.0 ± 1.5	287.7 ± 4.9	72.1 ± 0.3
MeanFM	3	t	1.000 ± 0.000	10.0 ± 0.8	291.3 ± 6.5	72.2 ± 0.3
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	single	1.000 ± 0.000	10.4 ± 1.8	298.0 ± 4.1	45.6 ± 0.2
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	r	1.000 ± 0.000	9.8 ± 1.9	298.0 ± 6.6	72.3 ± 0.7
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	c	1.000 ± 0.000	8.0 ± 1.2	284.6 ± 2.8	71.9 ± 0.6
CI-MeanFM, $\alpha_{\text{end}} = 0.2$	3	t	1.000 ± 0.000	10.1 ± 1.2	288.8 ± 5.9	72.9 ± 0.7
FM	3	single	1.000 ± 0.000	10.1 ± 1.4	310.3 ± 7.7	44.2 ± 0.5
FM	3	r	1.000 ± 0.000	9.3 ± 1.3	310.2 ± 6.8	72.1 ± 0.8
FM	3	c	1.000 ± 0.000	8.3 ± 1.1	299.3 ± 4.7	71.9 ± 1.0
FM	3	t	1.000 ± 0.000	9.8 ± 0.8	288.8 ± 6.0	71.9 ± 1.2
FM	5	single	1.000 ± 0.000	4.9 ± 0.6	297.4 ± 4.1	72.9 ± 0.3
FM	5	r	1.000 ± 0.000	4.8 ± 0.6	296.7 ± 3.7	112.2 ± 0.5
FM	5	c	1.000 ± 0.000	3.9 ± 0.3	292.8 ± 3.7	112.4 ± 0.6
FM	5	t	1.000 ± 0.000	5.6 ± 0.5	283.6 ± 4.3	112.4 ± 0.7
FM	20	single	1.000 ± 0.000	0.2 ± 0.4	316.2 ± 6.3	291.0 ± 0.8
FM	20	r	1.000 ± 0.000	0.5 ± 0.7	317.3 ± 7.2	422.3 ± 1.2
FM	20	c	1.000 ± 0.000	0.4 ± 0.5	311.4 ± 4.9	424.6 ± 2.5
FM	20	t	1.000 ± 0.000	0.7 ± 0.7	301.8 ± 4.8	422.3 ± 0.9

B.5. D3IL-avoiding at Twenty Episodes per Seed

Analytic average-velocity matching and instantaneous-velocity matching were evaluated a second time at the budgets they are operated at, $K = 1$ and $K = 2$, with twenty episodes per training seed and geometry in place of two: one hundred episodes per geometry and three hundred per configuration. Everything else is the configuration of Table 6.2: the same five trained checkpoints per model, four candidate plans, tightened constraints, the per-step projection of DPCC and the three selection rules. The comparison is within each model, the same configuration at two and at twenty episodes. Table B.5 prints it under projection and Table B.6 with projection switched off. These are the twenty-episode evaluations that are complete on all five training seeds and all three geometries. Consistency-interpolated average-velocity matching and the diffusion baseline have no twenty-episode evaluation that is complete in this sense, and at $K = 20$, where one control step takes about half a second (Table 6.2), none was completed for any model.

Success with constraint satisfaction. The two protocols agree. The operating point, analytic average-velocity matching at $K = 1$ under temporal consistency, reaches the goal within the constraints in 298 of 300 episodes at twenty episodes per seed and in 29 of 30 at two, and at $K = 2$ the counts are the same. Instantaneous-velocity matching under cumulative projection cost reaches 299 of 300 at $K = 1$ and 300 of 300 at $K = 2$, against 30 of 30 at both. In all sixteen configurations of the two tables, the count at two episodes lies inside the range of counts that 95 % of two-episode evaluations drawn from the twenty-episode one return.

What twenty episodes resolve. At two episodes per seed the count moves in steps of one episode in thirty, and the three selection rules of each model read within two episodes of each other. At twenty they separate. Analytic average-velocity matching under temporal consistency keeps 298 of 300 at both budgets, against 279 under random selection and 283 under cumulative projection cost at $K = 1$, and 280 and 278 at $K = 2$: the rule it is operated with is also the one with the highest success with constraint satisfaction. The three rules of instantaneous-velocity matching, all at 30 of 30 with two episodes, read 299, 299 and 297 of 300 at $K = 1$ and 299, 300 and 295 at $K = 2$. Drawn from these twenty-episode evaluations, a two-episode evaluation returns 30 of 30 with probability 0.81 for the operating point and 0.58 for instantaneous-velocity matching at $K = 2$ under temporal consistency. The spread over the five seeds narrows: for the operating point, the standard deviation of the success rate with constraint satisfaction falls from 0.075 at two episodes to 0.015 at twenty.

Control steps and time. The step counts behind Subsubsection 6.1.2.2 are reproduced. Under cumulative projection cost analytic average-velocity matching takes 72.0 control steps at $K = 1$ and 98.0 at $K = 2$, against 72.4 and 97.2 at two episodes, and under temporal consistency 61.0 and 60.4, against 58.6 and 59.4. Across the sixteen configurations the two protocols differ by at most 2.4 control steps. The time per control step of analytic average-velocity matching agrees to within 4 % under projection and 7 % without it; that of instantaneous-velocity matching is 5 to 13 % lower in the two-episode evaluation. The best rule of each model is the same at both protocols except for instantaneous-velocity matching at $K = 2$, whose three rules are level at two episodes and ordered cumulative projection cost, random selection, temporal consistency at twenty.

Without projection. Both models reach the goal in 297 to 299 of 300 episodes, against 30 of 30 at two episodes, and success with constraint satisfaction stays low: 36 of 300 for analytic average-velocity matching at both budgets and 15 and 30 of 300 for instantaneous-velocity matching at $K = 1$ and $K = 2$, against 1 and 3 of 30.

B. Extended Results

Table B.5.: D3IL-avoiding under per-step projection at twenty and at two episodes per training seed and geometry (columns 20 and 2): five training seeds $\times$ three geometries, 300 and 30 episodes per configuration; tightened constraints, four candidate plans, temporal U-Net of 4.0M parameters. The two-episode columns are the cells of Table 6.2; rules as there. S&C is the number of episodes that reach the goal within the constraints. *Range*: the counts, of 30, that 95 % of two-episode evaluations return when the two episodes of each seed and geometry are drawn without replacement from its twenty, computed exactly from the fifteen seed-geometry counts of the twenty-episode evaluation. Control steps and time per control step are mean $\pm$ standard deviation over the five seeds (Subsection 5.4.4). $\blacktriangleright$ the operating point.

Model	K	Rule	S&C, episodes			Control steps		ms per control step	
			20	2	Range	20	2	20	2
MeanFM	1	<i>r</i>	279/300	27/30	25–30	64.7 ± 1.8	64.3 ± 2.6	18.4 ± 0.1	19.2 ± 1.6
		<i>c</i>	283/300	28/30	26–30	72.0 ± 4.1	72.4 ± 8.4	18.0 ± 0.4	18.3 ± 0.5
		$\blacktriangleright t$	298/300	29/30	29–30	61.0 ± 1.0	58.6 ± 2.2	18.1 ± 0.3	18.3 ± 0.2
MeanFM	2	<i>r</i>	280/300	29/30	25–30	65.9 ± 1.4	65.1 ± 2.6	27.3 ± 0.2	27.6 ± 0.5
		<i>c</i>	278/300	28/30	25–30	98.0 ± 10.4	97.2 ± 4.8	26.8 ± 0.4	27.1 ± 0.2
		<i>t</i>	298/300	29/30	29–30	60.4 ± 0.9	59.4 ± 2.0	27.1 ± 0.3	27.7 ± 0.5
FM	1	<i>r</i>	299/300	30/30	29–30	68.3 ± 0.9	67.9 ± 1.2	19.4 ± 0.6	17.4 ± 0.1
		<i>c</i>	299/300	30/30	29–30	67.5 ± 1.2	67.0 ± 0.8	18.9 ± 0.2	17.3 ± 0.1
		<i>t</i>	297/300	30/30	29–30	67.1 ± 1.6	68.2 ± 1.7	20.1 ± 1.9	17.7 ± 0.6
FM	2	<i>r</i>	299/300	30/30	29–30	68.0 ± 1.6	65.7 ± 0.8	27.6 ± 0.5	26.2 ± 0.9
		<i>c</i>	300/300	30/30	30	68.4 ± 3.0	67.0 ± 3.8	27.3 ± 0.3	25.5 ± 0.2
		<i>t</i>	295/300	30/30	28–30	64.9 ± 1.9	63.8 ± 1.9	27.9 ± 0.5	25.9 ± 0.2

Table B.6.: The configurations of Table B.5 with projection switched off; the two-episode columns are the cells of Table 6.1. Success and S&C are counts of episodes, *Range* as in Table B.5, violating steps per episode as mean $\pm$ standard deviation over the five seeds.

Model	K	Success, episodes		S&C, episodes			Violating steps	
		20	2	20	2	Range	20	2
MeanFM	1	297/300	30/30	36/300	1/30	1–7	14.5 ± 0.7	15.5 ± 2.1
MeanFM	2	299/300	30/30	36/300	1/30	1–7	15.4 ± 0.9	15.5 ± 0.4
FM	1	297/300	30/30	15/300	3/30	0–4	17.3 ± 1.0	16.5 ± 3.4
FM	2	299/300	30/30	30/300	3/30	1–6	16.4 ± 1.3	15.6 ± 2.6

C. Compute Environment

Every training and evaluation of this thesis ran on one node of the institute’s cluster, scheduled by SLURM; Table C.1 lists it. The network runs on the GPU; the MuJoCo rollout and the SLSQP projection run on the CPU, which a job shares with the other jobs of the node, as it shares the memory and the I/O. Each job log records the git revision, the node and the GPU it ran on.

Table C.1.: The machine and the software of every reported run.

Component	Configuration
<i>hardware</i>	
Node	one GPU node of the institute cluster, shared
CPU	2× AMD EPYC 7282, 16 cores each
Cores / threads	32 physical / 64 logical, 2 NUMA domains, 1.5–2.8 GHz
Cache	512 KiB L2 per core; 128 MiB L3 total (8 × 16 MiB)
Main memory	252 GiB; no node-local scratch
GPUs on node	8× NVIDIA RTX A5000, 24 GB each
NVIDIA driver	530.41.03
<i>scheduling</i>	
Scheduler	SLURM 21.08.5
Job time limit	24 h
Allocation per job	one GPU, 8 threads, 32 GB memory; 3–24 h per training job
Operating system	Ubuntu, Linux 5.15.0
<i>software</i>	
Python	3.10 (Conda environment)
PyTorch	2.2.2, CUDA 12.1 build
NumPy / SciPy	1.26.4 / 1.13.1
MuJoCo	2.3.7, headless EGL rendering
Projection solver	SciPy SLSQP
ODE integration	torchdiffeq 0.2.3
Diffusers / Transformers	0.31.0 / 4.41.2
Gym / Gymnasium	0.26.2 / 0.29.1

Abbreviations

DPCC Diffusion Predictive Control with Constraints

NFE number of function evaluations

ODE ordinary differential equation

JVP Jacobian–vector product

NLP nonlinear program

SLSQP sequential least-squares quadratic programming

IK inverse kinematics

PD proportional–derivative

UAV unmanned aerial vehicle

FiLM feature-wise linear modulation

MPC model predictive control

List of Figures

4.1. Endpoint projection over the sampling steps	41
5.1. The two simulated platforms	50
5.2. The obstacle-avoidance task in MuJoCo	51
5.3. The three constraint geometries of obstacle avoidance	53
5.4. The vision-conditioned alignment task	54
5.5. The two camera observations used for alignment	55
5.6. The ten evaluated alignment contexts	57
5.7. The quadrotor scenes	59
5.8. The constraint sets of the quadrotor scenes	61
5.9. The demonstrations against the three constraint geometries	62
5.10. The recorded contexts against the alignment constraints	64
5.11. The demonstrations against the quadrotor constraints	67
6.1. Plans without projection, four models across the step budget	87
6.2. Control steps against planner time per control step	93
6.3. Alignment: outcome against cost	102
6.4. Alignment after projection: outcome against cost	111
6.5. UAV-pillars: the flown paths of the selected configuration	117
6.6. UAV-corridor after projection: violating steps and control steps against cost	125
6.7. UAV-s-curve: the plans without projection across the budget	131
6.8. UAV-s-curve: the selected plans under the two tracking controllers . . .	133
A.1. X2 dimensions and projection footprint	150
B.1. D3IL-avoiding: the executed paths under per-step projection	152
B.2. Alignment: the executed end-effector paths under each projection method	153
B.3. UAV-corridor: the flights along the corridor	155

List of Tables

3.1. Planner, loop and constraint mechanism in the closest related work . .	14
4.1. Notation	18
4.2. Method provenance	19
4.3. Embodiment instantiation	22
5.1. Properties of the simulated platforms	50
5.2. Constraint geometries of obstacle avoidance	52
5.3. Constraint set of the alignment task	55
5.4. Quadrotor scenes	60
5.5. Quadrotor demonstrations	65
5.6. The DPCC baseline, published and reproduced	72
5.7. Alignment set-up against D3IL	74
5.8. Training parameters	77
5.9. Evaluation parameters	79
5.10. Evaluation protocol per environment	81
6.1. The four models without projection	85
6.2. The generative models at the protocol of DPCC	91
6.3. Endpoint against per-step projection at matched candidates	95
6.4. Supported combinations on D3IL-avoiding	98
6.5. Alignment: the generative models without projection	100
6.6. Alignment: the activation threshold across the budget ladder	105
6.7. Alignment: the two projection methods	106
6.8. Alignment: the two projection methods across the models	109
6.9. UAV-pillars: the avoiding configurations on D3IL-avoiding and on UAV-pillars	114
6.10. UAV-pillars: the projected flights per geometry	115
6.11. UAV-corridor: the generative models before projection	120
6.12. UAV-corridor: the generative models after projection	122
6.13. UAV-s-curve: the generative models without projection	129
6.14. UAV-s-curve: the selected configuration under projection	130
6.15. UAV-s-curve: the same plans under two tracking controllers	134

6.16. UAV-s-curve: what each tracking controller costs per control step	135
B.1. UAV-corridor, tilt, per-step projection, every rule	157
B.2. UAV-corridor, tilt, endpoint projection, every rule	158
B.3. UAV-corridor, hump, per-step projection, every rule	159
B.4. UAV-corridor, hump, endpoint projection, every rule	160
B.5. D3IL-avoiding under projection at twenty and at two episodes per seed	163
B.6. D3IL-avoiding without projection at twenty and at two episodes per seed	163
C.1. Compute environment	165

Bibliography

- [BGL25] K. Black, M. Y. Galliker, and S. Levine. “Real-Time Execution of Action Chunking Flow Policies.” In: *arXiv preprint* (2025). arXiv: 2506.07339.
- [Bla+24] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, et al. “ π_0 : A Vision-Language-Action Flow Model for General Robot Control.” In: *arXiv preprint* (2024). Physical Intelligence. arXiv: 2410.24164.
- [Car+25] J. Carvalho, A. T. Le, P. Kicki, D. Koert, and J. Peters. “Motion planning diffusion: Learning and adapting robot motion planning with diffusion models.” In: *IEEE Transactions on Robotics* (2025).
- [CBF24] J. K. Christopher, S. Baek, and N. Fioretto. “Constrained synthesis with projected diffusion models.” In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 89307–89333.
- [Chi+23] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. “Diffusion policy: Visuomotor policy learning via action diffusion.” In: *The International Journal of Robotics Research* (2023). arXiv: 2303.04137.
- [DN21] P. Dhariwal and A. Nichol. “Diffusion models beat GANs on image synthesis.” In: *Advances in Neural Information Processing Systems* 34 (2021), pp. 8780–8794.
- [Ess+24] P. Esser, S. Kulal, A. Blattmann, R. Entezari, J. Müller, H. Saini, Y. Levi, D. Lorenz, A. Sauer, F. Boesel, et al. “Scaling rectified flow transformers for high-resolution image synthesis.” In: *Proceedings of the 41st International Conference on Machine Learning*. 2024.
- [Gao+26] D. Gao, B. Zhao, A. Lee, I. Chuang, H. Zhou, H. Wang, Z. Zhao, J. Zhang, and I. Soltani. “VITA: Vision-to-Action Flow Matching Policy.” In: *International Conference on Learning Representations (ICLR)*. 2026. arXiv: 2507.13231.
- [Gen+25a] Z. Geng, M. Deng, X. Bai, J. Z. Kolter, and K. He. “Mean Flows for One-step Generative Modeling.” In: *arXiv preprint* (2025). Carnegie Mellon University and MIT. arXiv: 2505.13447.

- [Gen+25b] Z. Geng, Y. Lu, Z. Wu, E. Shechtman, J. Z. Kolter, and K. He. “Improved Mean Flows: On the Challenges of Fastforward Generative Models.” In: *arXiv preprint* (2025). arXiv: 2512.02012.
- [He+16] K. He, X. Zhang, S. Ren, and J. Sun. “Deep Residual Learning for Image Recognition.” In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778.
- [HJA20] J. Ho, A. Jain, and P. Abbeel. “Denoising diffusion probabilistic models.” In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 6840–6851.
- [How+22] T. Howell, N. Gileadi, S. Tunyasuvunakool, K. Zakka, T. Erez, and Y. Tassa. “Predictive Sampling: Real-time Behaviour Synthesis with MuJoCo.” In: *arXiv preprint* (2022). Stanford University, DeepMind and UC Berkeley. arXiv: 2212.00541.
- [HS22] J. Ho and T. Salimans. “Classifier-free diffusion guidance.” In: *arXiv preprint arXiv:2207.12598* (2022).
- [Hua+25] T.-Y. Huang, A. Lederer, D.-J. Wu, X. Dai, S. Zhang, S. Sosnowski, S.-H. Sun, and S. Hirche. “SAD-Flower: Flow Matching for Safe, Admissible, and Dynamically Consistent Planning.” In: *arXiv preprint* (2025). arXiv: 2511.05355.
- [Jan+22] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine. “Planning with Diffusion for Flexible Behavior Synthesis.” In: *Proceedings of the 39th International Conference on Machine Learning*. Vol. 162. Proceedings of Machine Learning Research. PMLR, 2022, pp. 9902–9915. arXiv: 2205.09991.
- [Jia+24] X. Jia, D. Blessing, X. Jiang, M. Reuss, A. Donat, R. Lioutikov, and G. Neumann. “Towards Diverse Behaviors: A Benchmark for Imitation Learning with Human Demonstrations.” In: *The Twelfth International Conference on Learning Representations (ICLR)*. 2024. arXiv: 2402.14606.
- [Jia+25a] X. Jia, A. Donat, X. Huang, X. Zhao, D. Blessing, H. Zhou, H. A. Wang, H. Zhang, Q. Wang, R. Lioutikov, and G. Neumann. “X-IL: Exploring the Design Space of Imitation Learning Policies.” In: *arXiv preprint* (2025). arXiv: 2502.12330.
- [Jia+25b] S. Jiang, X. Fang, N. Roy, T. Lozano-Pérez, L. Kaelbling, and S. Ancha. “Streaming Flow Policy: Simplifying Diffusion/Flow-Matching Policies by Treating Action Trajectories as Flow Trajectories.” In: *arXiv preprint* (2025). arXiv: 2505.21851.

- [Kon+24] K. Kondo, A. Tagliabue, X. Cai, C. Tewari, O. Garcia, M. Espitia-Alvarez, and J. P. How. “CGD: Constraint-Guided Diffusion Policies for UAV Trajectory Planning.” In: *arXiv preprint* (2024). arXiv: 2405.01758.
- [Kra88] D. Kraft. *A software package for sequential quadratic programming*. Tech. rep. DFVLR-FB 88-28. Oberpfaffenhofen, Germany: DFVLR, 1988.
- [LAA25] Z. Li, K. Alim, and N. Azizan. “HardFlow: Hard-Constrained Sampling for Flow-Matching Models via Trajectory Optimization.” In: *arXiv preprint* (2025). Massachusetts Institute of Technology. LaTeX source and bibliography in PAPERS/Recommand_Paper/HF/. arXiv: 2511.08425.
- [LGL23] X. Liu, C. Gong, and Q. Liu. “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow.” In: *The Eleventh International Conference on Learning Representations (ICLR)*. 2023.
- [Lip+23] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. “Flow Matching for Generative Modeling.” In: *The Eleventh International Conference on Learning Representations (ICLR)*. 2023. arXiv: 2210.02747.
- [Lip+24] Y. Lipman, M. Havasi, P. Holderrieth, N. Shaul, M. Le, B. Karrer, R. T. Q. Chen, D. Lopez-Paz, H. Ben-Hamu, and I. Gat. “Flow Matching Guide and Code.” In: *arXiv preprint* (2024). FAIR at Meta, MIT CSAIL and Weizmann Institute of Science. arXiv: 2412.06264.
- [LLM10] T. Lee, M. Leok, and N. H. McClamroch. “Control of Complex Maneuvers for a Quadrotor UAV using Geometric Methods on SE(3).” In: *arXiv preprint* (2010). arXiv: 1003.2005.
- [Man+21] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín. “What Matters in Learning from Offline Human Demonstrations for Robot Manipulation.” In: *5th Annual Conference on Robot Learning (CoRL)*. 2021.
- [MK11] D. Mellinger and V. Kumar. “Minimum Snap Trajectory Generation and Control for Quadrotors.” In: *IEEE International Conference on Robotics and Automation (ICRA)*. NO LOCAL COPY – metadata verified against the ICRA 2011 proceedings record; cited for the cascaded position/attitude structure and the fixed rotor-allocation matrix. IEEE, 2011, pp. 2520–2525.
- [ML99] M. Morari and J. H. Lee. “Model predictive control: past, present and future.” In: *Computers & Chemical Engineering* 23.4-5 (1999), pp. 667–682.

- [ND21] A. Q. Nichol and P. Dhariwal. “Improved Denoising Diffusion Probabilistic Models.” In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research. PMLR, 2021, pp. 8162–8171.
- [NV25] K. Nguyen and M. N. Vu. “XFlowMP: Task-Conditioned Motion Fields for Generative Robot Planning with Schrödinger Bridges.” In: *arXiv preprint* (2025). arXiv: 2512.00022.
- [Oel+26] T. Oelerich, G. Ebmer, C. Hartl-Nesic, and A. Kugi. “SafeFlowMPC: Predictive and Safe Trajectory Planning for Robot Manipulators with Learning-based Policies.” In: *IEEE International Conference on Robotics and Automation (ICRA)*. TU Wien (ACIN); code at github.com/ThiesOelerich. 2026. arXiv: 2602.12794.
- [Per+17] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. “FiLM: Visual Reasoning with a General Conditioning Layer.” In: *arXiv preprint* (2017). arXiv: 1709.07871.
- [Pow+23] T. Power, R. Soltani-Zarrin, S. Iba, and D. Berenson. “Sampling constrained trajectories using composable diffusion models.” In: *IROS 2023 Workshop on Differentiable Probabilistic Robotics*. 2023.
- [PX22] W. Peebles and S. Xie. “Scalable Diffusion Models with Transformers.” In: *arXiv preprint* (2022). arXiv: 2212.09748.
- [RRS25] R. Römer, A. v. Rohr, and A. P. Schoellig. “Diffusion Predictive Control with Constraints.” In: *Proceedings of the 7th Annual Learning for Dynamics & Control Conference*. Ed. by N. Ozay, L. Balzano, D. Panagou, and A. Abate. Vol. 283. Proceedings of Machine Learning Research. PMLR, 2025, pp. 791–803. arXiv: 2412.09342.
- [SME21] J. Song, C. Meng, and S. Ermon. “Denoising Diffusion Implicit Models.” In: *The Ninth International Conference on Learning Representations (ICLR)*. 2021.
- [Soh+15] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli. “Deep unsupervised learning using nonequilibrium thermodynamics.” In: *Proceedings of the 32nd International Conference on Machine Learning*. PMLR. 2015, pp. 2256–2265.
- [Son+23] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever. “Consistency Models.” In: *Proceedings of the 40th International Conference on Machine Learning*. Proceedings of Machine Learning Research. PMLR, 2023.

- [TET12] E. Todorov, T. Erez, and Y. Tassa. “MuJoCo: A physics engine for model-based control.” In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2012, pp. 5026–5033.
- [Utk+25] Utkarsh, P. Cai, A. Edelman, R. Gomez-Bombarelli, and C. V. Rackauckas. “Physics-Constrained Flow Matching: Sampling Generative Models with Hard Constraints.” In: *arXiv preprint* (2025). arXiv: 2506.04171.
- [VGO+20] P. Virtanen, R. Gommers, T. E. Oliphant, et al. “SciPy 1.0: fundamental algorithms for scientific computing in Python.” In: *Nature Methods* 17.3 (2020), pp. 261–272.
- [Wan+24] Z. Wang, Z. Li, A. Mandlekar, Z. Xu, J. Fan, Y. Narang, L. Fan, Y. Zhu, Y. Balaji, M. Zhou, M.-Y. Liu, and Y. Zeng. “One-Step Diffusion Policy: Fast Visuomotor Policies via Diffusion Distillation.” In: *arXiv preprint* (2024). arXiv: 2410.21257.
- [Wan+25] X. Wang, D. Yang, Y. Liao, W. Zheng, B. Dai, W. Wu, H. Li, and S. Liu. “UAV-Flow Colosseo: A Real-World Benchmark for Flying-on-a-Word UAV Imitation Learning.” In: *arXiv preprint* (2025). arXiv: 2505.15725.
- [WH18] Y. Wu and K. He. “Group Normalization.” In: *Proceedings of the European Conference on Computer Vision (ECCV)*. 2018, pp. 3–19.
- [Yan+25a] G. Yan, J. Zhu, Y. Deng, S. Yang, R.-Z. Qiu, X. Cheng, M. Memmel, R. Krishna, A. Goyal, X. Wang, and D. Fox. “ManiFlow: A General Robot Manipulation Policy via Consistency Flow Training.” In: *arXiv preprint* (2025). arXiv: 2509.01819.
- [Yan+25b] Z. Yang, X. Dai, D. Yu, Z. Li, M. Khadiv, S. Hirche, and S. Haddadin. “Uni-ConFlow: A Unified Constrained Flow-Matching Framework for Certified Motion Planning.” In: *arXiv preprint* (2025). arXiv: 2506.02955.
- [Yi+26] B. Yi, H. Choi, H. G. Singh, X. Huang, T. E. Truong, C. Sferrazza, Y. Ma, R. Duan, P. Abbeel, G. Shi, K. Liu, and A. Kanazawa. “Flow Policy Gradients for Robot Control.” In: *arXiv preprint* (2026). arXiv: 2602.02481.
- [YJH25] J. Yang, S. Jang, and S. Han. “SafeFlowMatcher: Safe and Fast Planning using Flow Matching with Control Barrier Functions.” In: *arXiv preprint* (2025). Korea Advanced Institute of Science and Technology. arXiv: 2509.24243.
- [Zha+24] Q. Zhang, Z. Liu, H. Fan, G. Liu, B. Zeng, and S. Liu. “FlowPolicy: Enabling Fast and Robust 3D Flow-based Policy via Consistency Flow Matching for Robot Manipulation.” In: *arXiv preprint* (2024). arXiv: 2412.04987.

- [Zha+25a] X. Zhai, Q. Zhao, Q. Yu, and C. Hao. “VFP: Variational Flow-Matching Policy for Multi-Modal Robot Manipulation.” In: *arXiv preprint* (2025). arXiv: 2508.01622.
- [Zha+25b] H. Zhang, A. Siarohin, W. Menapace, M. Vasilkovsky, S. Tulyakov, Q. Qu, and I. Skorokhodov. “AlphaFlow: Understanding and Improving MeanFlow Models.” In: *arXiv preprint* (2025). Snap Inc. and University of Michigan. arXiv: 2510.20771.
- [ZTM22] K. Zakka, Y. Tassa, and MuJoCo Menagerie Contributors. *MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo*. https://github.com/google-deepmind/mujoco_menagerie. 2022.